

RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM)

SECTION 1

GROUP 3

NAME	MATRIC NUMBER
CHEW CHIU XIAN	A23CS0061
CHUA JIA LIN	A23CS0069
DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
ANIS SAFIYYA BINTI JANAI	A23CS0049
TAN YI YA	A23CS0187
GOE JIE YING	A23CS0224



UNIVERSITI TEKNOLOGI MALAYSIA

DECLARATION OF THESIS/PROJECT REPORT

Author's Full Name :
Student's Matric No. : Academic Session :
Date : UTM Email :
Thesis/Project Report Title :

I declare that this thesis is classified as : Please tick one (/)

☐

OPEN ACCESS

I agree that my thesis/project report to be published as hard copy or online open access.

☐

RESTRICTED

Contain restricted information as specified by the organization/institution where research was done.

(The Library will block access for up to five (5) years).

☐

CONFIDENTIAL

Contain confidential information under Official Secret Act 1972.

(If none of the options are selected, the first option will be carried out by default)

I acknowledge the intellectual property in the thesis/project report belonged to Universiti Teknologi Malaysia and I agree to allow this thesis/project report to be placed at the library under the following terms:

1. This thesis/project report is the property of Universiti Teknologi Malaysia.
2. The library of Universiti Teknologi Malaysia has the right to make copies for educational purposes only.
3. The library of Universiti Teknologi Malaysia is allowed to make copies of this thesis for academic exchange.

Signature of Student :

Full Name :

Date :

Approved by Supervisor :

Signature of University Coach :

Signature of Industry Coach :

Full Name of University Coach:

Full Name of Industry Coach:

Date :

Date

:

NOTES: If the thesis is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restriction.

“I hereby declare that we have read this thesis and in my
opinion this thesis is sufficient in term of scope and quality for the
award of the degree of Bachelor of Computer Science (Data Engineering)”

Signature :
Industry Coach :
Date :

Signature :
University Coach :
Date :

RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM)

SECTION 1

GROUP 3

NAME	MATRIC NUMBER
CHEW CHIU XIAN	A23CS0061
CHUA JIA LIN	A23CS0069
DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
ANIS SAFIYYA BINTI JANAI	A23CS0049
TAN YI YA	A23CS0187
GOE JIE YING	A23CS0224

A thesis submitted in fulfilment of the
requirements for the award of the degree of

Bachelor of Computer Science (Data Engineering)

Faculty of Computing
Universiti Teknologi Malaysia

JUNE 2026

DECLARATION

I declare that this thesis entitled “*title* “ is the result of my own research except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature :
Name :
Date :

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to our project lecturer, Dr. Aryati Binti Bakri, for the invaluable guidance, continuous support, and constructive feedback provided throughout the completion of this project. The knowledge, advice, and encouragement given have greatly contributed to the successful development of this study.

We would also like to extend our appreciation to the Faculty of Computing, Universiti Teknologi Malaysia, for providing us with the opportunity, facilities, and resources necessary to carry out this project. The knowledge and experiences gained throughout our studies have provided a strong foundation for the completion of this work.

Our heartfelt thanks also go to all group members for their commitment, cooperation, and dedication throughout every stage of the project. The teamwork, mutual support, and exchange of ideas among us played an important role in overcoming challenges and ensuring the successful completion of this project.

Finally, we would like to express our deepest appreciation to our parents, family members, and friends for their continuous encouragement, patience, and moral support throughout this journey. Their understanding and motivation have inspired us to persevere and successfully complete this project.

We are sincerely grateful to everyone who has directly or indirectly contributed to the completion of this thesis.

ABSTRACT

Inventory management plays a crucial role in ensuring operational efficiency, cost control, and customer satisfaction in manufacturing organizations. However, many organizations still face challenges due to fragmented data sources and limited analytical capabilities, resulting in poor visibility into inventory performance and delayed decision-making. This study aims to develop an end-to-end data engineering pipeline and business intelligence solution for Recoverable Assets (RA) and Inventory Risk Management (IRM) in Pittsburgh Plate Glass (PPG). The objectives of the study are to automate the identification of excess inventory, obsolete materials, financial provisions, stockout risks, and the impact of material shortages on customer sales orders. The study is academically significant as it demonstrates the application of modern cloud-based data engineering practices while practically providing a scalable solution for inventory monitoring and decision support in manufacturing environments. The project adopts the Waterfall methodology and implements a Medallion Architecture consisting of Bronze, Silver, and Gold layers using Microsoft Azure services, including Azure Data Factory, Azure Data Lake Storage Gen2, and Azure Synapse Analytics. Six synthetic CSV datasets representing operational inventory data are processed and transformed using business rules related to Recoverable Assets (RA) and Magna Carta (MC). The processed data are modelled into a galaxy schema and visualized through interactive dashboards developed in Microsoft Power BI. The findings demonstrate that the proposed system successfully automates inventory risk analysis by identifying recoverable assets, dead stock, expired materials, financial exposure, and stockout risks while enabling the tracing of affected customer orders. Testing results indicate that the pipeline operates accurately and reliably in processing and transforming inventory data. In conclusion, the study contributes a cloud-based and scalable data engineering framework that transforms fragmented inventory data into actionable business intelligence. The proposed solution improves inventory visibility, supports proactive decision-making, reduces potential financial losses, and provides organizations with a centralized and analytics-driven approach to inventory risk management.

ABSTRAK

Pengurusan inventori memainkan peranan penting dalam memastikan kecekapan operasi, kawalan kos dan kepuasan pelanggan dalam organisasi pembuatan. Walau bagaimanapun, banyak organisasi masih menghadapi cabaran akibat sumber data yang berpecah dan keupayaan analitik yang terhad, sekali gus menyebabkan kekurangan keterlihatan terhadap prestasi inventori dan kelewatan dalam membuat keputusan. Kajian ini bertujuan membangunkan satu saluran paip kejuruteraan data hujung ke hujung serta penyelesaian perisikan perniagaan bagi Pengurusan Aset Boleh Dipulihkan (Recoverable Assets, RA) dan Pengurusan Risiko Inventori (Inventory Risk Management, IRM) di Pittsburgh Plate Glass (PPG). Objektif kajian adalah untuk mengautomasikan pengenalan inventori berlebihan, bahan usang, peruntukan kewangan, risiko kehabisan stok serta impak kekurangan bahan terhadap pesanan jualan pelanggan. Kajian ini penting dari sudut akademik kerana menunjukkan aplikasi amalan kejuruteraan data berasaskan awan moden, dan dari sudut praktikal kerana menyediakan penyelesaian yang berskala untuk pemantauan inventori dan sokongan membuat keputusan dalam persekitaran pembuatan. Projek ini menggunakan metodologi Waterfall dan melaksanakan Seni Bina Medallion yang terdiri daripada lapisan Bronze, Silver dan Gold menggunakan perkhidmatan Microsoft Azure, termasuk Azure Data Factory, Azure Data Lake Storage Gen2 dan Azure Synapse Analytics. Enam set data CSV sintetik yang mewakili data operasi inventori diproses dan ditransformasikan menggunakan peraturan perisikan berkaitan Recoverable Assets (RA) dan Magna Carta (MC). Data yang telah diproses dimodelkan ke dalam skema galaksi dan divisualisasikan melalui papan pemuka interaktif yang dibangunkan menggunakan Microsoft Power BI. Hasil kajian menunjukkan bahawa sistem yang dicadangkan berjaya mengautomasikan analisis risiko inventori dengan mengenal pasti aset boleh dipulihkan, stok mati, bahan tamat tempoh, pendedahan kewangan dan risiko kehabisan stok, di samping membolehkan pengesanan pesanan pelanggan yang terjejas. Keputusan pengujian turut membuktikan bahawa saluran paip data beroperasi dengan tepat dan boleh dipercayai dalam memproses serta mentransformasikan data inventori. Kesimpulannya, kajian ini menyumbang kepada pembangunan rangka kerja kejuruteraan data berasaskan awan yang berskala dan mampu menukarkan data inventori yang berpecah kepada perisikan perniagaan yang boleh diambil tindakan. Penyelesaian yang dicadangkan meningkatkan keterlihatan inventori, menyokong pembuatan keputusan secara proaktif, mengurangkan potensi kerugian kewangan dan menyediakan pendekatan pengurusan risiko inventori yang berpusat serta dipacu oleh analitik.

TABLE OF CONTENTS

DECLARATION	6
ACKNOWLEDGEMENT	7
ABSTRACT	8
ABSTRAK	9
CHAPTER 1	20
1.1 Introduction	20
1.2 Problem Background	21
1.3 Project Aim	24
1.4 Project Objectives	24
1.5 Project Scope	24
1.6 Project Importance	25
1.7 Report Organization	26
CHAPTER 2	27
2.1 Introduction	27
2.2 Enterprise Resource Planning	27
2.3 Supply Chain Management	28
2.4 Medallion Data Architecture	28
2.5 Dimensional Modelling and Galaxy-Schema	29
2.6 Microsoft Azure	29
2.7 Dashboard Visualization	30
2.8 Related Previous Study	31
2.9 Summary	33
CHAPTER 3	34
3.1 Introduction	34
3.2 Methodology Selection	34
3.3 Phases of Methodology	35
3.3.1 Planning	35
3.3.2 Analysis	36
3.3.3 Design	37
3.3.4 Implementation	38
3.3.5 Testing	39
3.3.6 Gantt Chart	40
3.4 System Requirement Analysis	40
3.4.1 Software Requirements	41
3.4.1.1 Microsoft Azure Services Required	41
3.4.1.2 Azure Data Lake Storage Gen2 (ADLS Gen2)	41
3.4.1.3 Azure Data Factory (ADF)	41
3.4.1.4 Azure Synapse Analytics	42
3.4.1.5 Data Warehouse Layer	42
3.4.1.6 Power BI	42
3.4.1.7 Security and Access Management	42
3.4.2 Hardware Requirements	43

3.4.2.1 Client Machine Requirements	43
3.4.2.2 Server Infrastructure	43
3.4.2.3 Network Requirements	44
3.5 Summary	44
CHAPTER 4	45
4.1 Introduction	45
4.2 System Analysis	45
4.2.1 Case Study	46
4.2.2 Company Organization Structure	47
4.2.3 System Requirements Gathering Techniques	48
4.3 System Requirements	50
4.3.1. Functional Requirements	50
4.3.2. Non Functional Requirements	52
4.4 Current System Analysis	54
4.4.1 Existing System Overview	54
4.4.2 Current Business Process	55
4.4.3 Problems in Current System	56
4.4.4 Need for Proposed System	57
4.5 System Design	58
4.5.1 System Architecture	58
4.5.2 System Architecture Components	58
4.5.2.1 Data Sources Layer	59
4.5.2.2 Data Orchestration Layer	59
4.5.2.3 Data Storage Layer	59
4.5.2.4 Data Transformation Layer	60
4.5.2.5 Data Modeling Layer	60
4.5.2.6 Data Visualization Layer	60
4.5.2.7 Security and Governance Layer	60
4.5.3 System Design	61
4.5.3.1 Use Case Diagram	61
4.5.3.2 Activity Diagram	62
4.5.3.3 Sequence Diagram	63
4.5.3.4 Data Flow Diagram (DFD)	64
4.5.4 Database Design	66
4.5.4.1 NoSQL Design	67
4.5.4.2 Normalization	68
4.5.4.3 Entity Relationship Diagram (ERD)	69
4.5.4.4 Data Modelling: Galaxy Schema	70
4.5.5 Interface Design	72
4.5.6 Summary	73
CHAPTER 5	74
5.1 Introduction	74
5.2 System Development	74

5.2.1 Source Data Preparation	74
5.2.2 SQL Database Setup	80
5.2.3 Azure Resource Configuration	82
5.2.4 Azure Data Factory Pipeline Development	83
5.2.5 Overall Pipeline Flow	84
5.3 Coding of system's main functions	85
5.3.1 Data Loading from Bronze Layer	85
5.3.2 Data Profiling and Validation	85
5.3.3 Handling Business-Valid Null Values	87
5.3.4 Writing Cleaned Data to Silver Layer	87
5.3.5 Silver to Gold Layer Data Transformation	88
5.3.6 Temporally-Correct Supplier Mapping Engine	89
5.3.7 Lot Expiry Date & Consumption Metrics Calculation	90
5.3.8 Magna Carta Business Flag Implementation	91
5.3.9 Customer Sales Revenue At Risk Mapping	92
5.3.10 Committing Analytical Datasets to Gold Layer Storage	92
5.3.11 Coding of Synapse Logical Data Warehouse External Schema Layer	93
5.3.12 Provisioning the Central Fact and Dimensional Semantic Structures	94
5.3.13 Multi-Table Operational Audit and Pipeline Validation	97
5.4 Dashboard interface	99
5.5 Testing	101
5.5.1 White Box Testing	101
5.5.1.1 Overview	101
5.5.1.2 Executive Summary	102
5.5.1.3 Key Findings	102
5.5.1.4 Defect Register	103
5.5.1.5 Module 1 : Bronze Layer: Schema Validation	104
5.5.1.6 Module 2 : Bronze Layer: Row Count Integrity	104
5.5.1.7 Module 3a : Silver Column Pruning: materials	105
5.5.1.8 Module 3b : Silver Passthrough Tables	105
5.5.1.9 Module 3c : Silver delivery_status Logic	106
5.5.1.10 Module 4a : Gold dim_materials Validation	107
5.5.1.11 Module 4b : Gold dim_suppliers Validation	107
5.5.1.12 Module 5 : Gold bridge_bom Validation	108
5.5.1.13 Module 6a : is_mc_dead_stock Flag Logic	109
5.5.1.14 Module 6b : is_mc_expired Flag Logic	109
5.5.1.15 Module 6c : is_recoverable_asset Flag Logic	110
5.5.1.16 Module 6d : mc_provision_amount Calculation	110
5.5.1.18 Module 7 : Gold fact_sales_impact Validation	111
5.5.1.19 Module 8 : Referential Integrity	111
5.5.1.20 Pipeline Load Summary Evidence	112
5.5.1.21 Note on Final Notebook Summary Inconsistency	112
5.5.1.22 White Box Testing Conclusion	113

5.5.2 Black Box Testing	113
5.5.2.1 Inventory Risk Dashboard Testing	114
5.5.2.2 Supply Chain Impact Dashboard Testing	115
5.5.2.3 System Testing	115
5.5.2.4 Black Box Testing Conclusion	116
5.6 Summary	117
REFERENCES	118
Appendix A	119
Appendix B	120
Appendix C	122
Appendix D	125
Appendix E	127
Appendix F	128
Appendix G	130
Appendix H	132
Appendix I	133

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 3.4.2.1	<u>Machine Requirements</u>	43
Table 4.3.1	<u>Functional Requirements</u>	50
Table 4.3.2	<u>Non Functional Requirements</u>	52
Table 4.5.4.1	<u>Derived column description</u>	71
Table 5.5.2.1	<u>Inventory Risk Dashboard Testing</u>	19
Table 5.5.2.2	<u>Supply Chain Impact Dashboard Testing</u>	21
Table 5.5.2.3	<u>System Testing</u>	21
Table 5.5.1.1.1	<u>White box Testing Details</u>	105
Table 5.5.1.1.2	<u>Testing Modules</u>	105
Table 5.5.1.1.3	<u>Defect Register</u>	106
Table 5.5.1.1.4	<u>Module 1 : Bronze Layer</u>	107
Table 5.5.1.1.5	<u>Module 2 : Bronze Layer: Row Count Integrity</u>	107
Table 5.5.1.1.6	<u>Module 3a : Silver Column Pruning: materials</u>	107
Table 5.5.1.1.7	<u>Module 3b : Silver Passthrough Tables</u>	108
Table 5.5.1.1.8	<u>Module 3c : Silver delivery_status Logic</u>	108
Table 5.5.1.1.9	<u>Module 4a : Gold dim_materials Validation</u>	109
Table 5.5.1.1.10	<u>Module 4b : Gold dim_suppliers Validation</u>	109
Table 5.5.1.1.11	<u>Gold bridge_bom Validation</u>	110
Table 5.5.1.1.12	<u>Module 6a : is_mc_dead_stock Flag Logic</u>	111
Table 5.5.1.1.13	<u>Module 6b : is_mc_expired Flag Logic</u>	111
Table 5.5.1.1.14	<u>Module 6c : is_recoverable_asset Flag Logic</u>	112
Table 5.5.1.1.15	<u>Module 6d : mc_provision_amount Calculation</u>	112
Table 5.5.1.1.16	<u>Gold fact_sales_impact Validation</u>	113
Table 5.5.1.1.17	<u>Referential Integrity</u>	113

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
Figure 3.3.1	<u>Galaxy Schema Design</u>	38
Figure 3.3.2	<u>Project Gantt Chart</u>	40
Figure 4.4.2	<u>Flowchart Diagram for Current Business Process</u>	55
Figure 4.5.1	<u>System Architecture Diagram</u>	58
Figure 4.5.3.1	<u>Use Case Diagram</u>	61
Figure 4.5.3.2	<u>Activity Diagram</u>	63
Figure 4.5.3.3	<u>Sequence Diagram</u>	64
Figure 4.5.3.4	<u>Level 0 DFD</u>	65
Figure 4.5.3.5	<u>Level 1 DFD</u>	66
Figure 4.5.4.1	<u>Bronze layer folder hierarchy structure</u>	67
Figure 4.5.4.2	<u>3NF Normalization on project dataset</u>	69
Figure 4.5.4.3	<u>Entity Relationship Diagram (ERD)</u>	70
Figure 4.5.4.4	<u>Galaxy Schema</u>	72
Figure 4.5.5.1	<u>Dashboard Interface Design</u>	72
Figure 5.2.2.1	<u>Tables Created in PPG Database</u>	80
Figure 5.2.2.2	<u>PPG_PK_FK_constraints SQL Script</u>	81
Figure 5.2.2.3	<u>Database Schema for PPG Database</u>	82
Figure 5.2.2.4	<u>createlogin.sql SQL Script</u>	82
Figure 5.2.3.1	<u>Resources Created in PPG Resource Group</u>	83
Figure 5.2.4.1	<u>Pipeline Diagram of copy_all_tables</u>	84
Figure 5.2.4.2	<u>copy_all_tables Pipeline Execution Result</u>	84

Figure 5.3.1	<u>Load raw operational data from Bronze</u>	85
Figure 5.3.2.1	<u>Inspect All Tables</u>	86
Figure 5.3.2.2	<u>Check Null Value...</u>	86
Figure 5.3.2.3	<u>Check Duplicate for Each Record in Table</u>	87
Figure 5.3.3	<u>Business-Valid Null Values Handling</u>	87
Figure 5.3.4.1	<u>Data Transformation to Silver</u>	88
Figure 5.3.4.2	<u>Inspect Silver Layer</u>	88
Figure 5.3.5	<u>Load raw operational data from Silver</u>	89
Figure 5.3.6.1	<u>Filter and join between Purchase and Inventory</u>	90
Figure 5.3.6.2	<u>Window function</u>	93
Figure 5.3.7.1	<u>Lot Expiry Calculation</u>	93
Figure 5.3.7.2	<u>Consumption Metrics Calculation</u>	94
Figure 5.3.8	<u>Magna Carta Business Flag Computation Code</u>	94
Figure 5.3.9.1	<u>Sales Order Filtering</u>	95
Figure 5.3.9.2	<u>Bridge table between material_id and finished_product</u>	95
Figure 5.3.10.1	<u>Writing fact and dimension tables into Gold</u>	96
Figure 5.3.10.2	<u>Tables display code</u>	96
Figure 5.3.11	<u>Clean State Provisioning Script</u>	96
Figure 5.3.12.1	<u>Master Dimension Table Schema Generation</u>	97
Figure 5.3.12.2	<u>Recipe Mapping Bridge Table Schema Generation</u>	98
Figure 5.3.12.3	<u>Central Fact Schema Provisioning and Validation</u>	99
Figure 5.3.13.1	<u>Cross-Process Star Schema Join Query (Query A)</u>	100
Figure 5.3.13.2	<u>Inventory Provisioning and Supplier Audit Query (Query B)</u>	100
Figure 5.3.13.3	<u>Chronological Time-Series Integrity Spot-Checks</u>	101
Figure 5.4.1	<u>Dashboard 1: Inventory RA/MC Overview</u>	104

Figure 5.4.2	<u>Dashboard 2: Material Restock Alert</u>	105
Figure 5.5.1.1.1	<u>Bronze Schema Validation</u>	106
Figure 5.5.1.1.2	<u>Bronze Row Count Integrity</u>	107
Figure 5.5.1.1.3	<u>Silver Column Pruning</u>	107
Figure 5.5.1.1.4	<u>Silver Passthrough Tables</u>	108
Figure 5.5.1.1.5	<u>Silver Delivery Status Logic</u>	108
Figure 5.5.1.1.6	<u>Gold dim_materials Validation</u>	110
Figure 5.5.1.1.7	<u>Gold dim_suppliers Validation</u>	110
Figure 5.5.1.1.8	<u>Gold bridge_bom validation</u>	110
Figure 5.5.1.1.9	<u>is mc_dead_stock Flag Logic</u>	111
Figure 5.5.1.1.10	<u>is mc_expired Flag Logic</u>	111
Figure 5.5.1.1.11	<u>is_recoverable asset Flag Logic</u>	112
Figure 5.5.1.1.12	<u>mc_provision_amount Calculation</u>	112
Figure 5.5.1.1.13	<u>Gold Fact Sales Validation</u>	113
Figure 5.5.1.1.14	<u>Refrential Integrity</u>	114
Figure 5.5.1.1.15	<u>Tables loaded</u>	114
Figure 5.5.1.1.16	<u>White Box Testing - Final Summary</u>	115

LIST OF ABBREVIATIONS

IRM	-	Inventory Risk Management
MC	-	Magna Carta
PPG	-	Pittsburgh Plate Glass
RA	-	Recoverable Assets

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
Appendix A	<u>Gantt Chart</u>	119
Appendix B	<u>expand_inventory_snapshot.py</u>	120
Appendix C	<u>expand_materials.py</u>	122
Appendix D	<u>expand_production_orders.py</u>	125
Appendix E	<u>expand_purchase_orders.py</u>	127
Appendix F	<u>expand_sales_orders.py</u>	128
Appendix G	<u>expand_suppliers.py</u>	130
Appendix H	<u>Activity Diagram</u>	132
Appendix I	<u>Sequence Diagram</u>	133

CHAPTER 1

INTRODUCTION

1.1 Introduction

Inventory management is a key part of any manufacturing organization because it affects how smoothly operations run, how costs are controlled and how well customer demands are met. Companies need to maintain the right balance of stock which will be enough to support production, but not so much that it leads to unnecessary storage costs or wasted resources.

Today, many organizations use data and digital systems to manage their inventory more efficiently. These systems help track stock levels, understand usage patterns and support better decision making. However, in reality, not all companies have fully connected systems, which can make it difficult to manage inventory effectively.

Pittsburgh Plate Glass (PPG) is one example of a company facing such challenges. Due to disconnected data sources and limited analytical capabilities, it becomes harder for the organization to get a clear picture of its inventory. Without proper visibility, identifying potential issues early becomes a challenge.

When data is not properly integrated or analyzed, problems like excess stock or shortages can easily go unnoticed. This often leads to decisions being made too late, which can affect both operations and overall business performance.

Because of this, there is a clear need for a more structured and data-driven approach to inventory management. Therefore, this project focuses on developing an integrated solution using Microsoft Azure and Microsoft Power BI to improve inventory visibility, automate analysis and support better decision making within PPG.

1.2 Problem Background

Effective inventory management is very important for every manufacturing organization to ensure operational efficiency, budgeting, and customer satisfaction. Sufficient stock level must be maintained by the organization to meet user requirements and prevent financial damage caused by excess inventory. However, challenges in inventory management were met by the Pittsburgh Plate Glass (PPG) company due to disconnected data sources and limited analytical capabilities.

One of the biggest issues is the inability to identify excess materials, known as Recoverable Assets (RA), where stock levels exceeding 12 months of consumption are observed. Working capital is occupied and the storage costs are increased by these materials. Besides, obstacles are faced by the company in detecting obsolete inventory, such as Magna Carta (MC) Dead Stock and expired materials, which require financial write-downs and affect the profitability.

Another major challenge is the lack of visibility into stockout risks, where materials drop under the required levels referring to their production time are observed. Production processes can be disrupted and customer order fulfillment can be delayed by this situation. In addition, the capability to trace how material shortages affect open customer sales orders is lacking in the organization, disabling customer expectations to be managed proactively.

These challenges are worsened by the absence of an end-to-end data pipeline that enables raw data to be integrated, business logic applied, and valuable insights generated. Without this type of system, decision-making remains reactive which causes inefficiencies and increased operational risk.

Therefore, a comprehensive solution that leverages data engineering and business intelligence using tools such as Microsoft Azure and Microsoft Power BI is required to automate inventory analysis, improve visibility, and support data-driven decision-making.

Problem Statement

An integrated system that automatically processes inventory data to identify excess stock, detect obsolete materials, quantify financial provisions, and predict stockout risks is lacking in the organization. In conclusion, inefficient inventory management is faced, financial losses due to write-downs are incurred, and the ability to fulfill customer orders effectively is reduced.

Objectives

The objectives of this project are:

1. To develop an end-to-end data pipeline for ingesting and processing inventory-related data
2. To implement business logic for identifying Recoverable Assets (RA), Magna Carta Dead Stock, and Magna Carta Expired materials
3. To calculate the total financial provision required for obsolete inventory
4. To identify materials at risk of stockout based on consumption and lead time
5. To determine the impact of inventory shortages on production and customer sales orders
6. To design and develop an interactive dashboard for visualization and decision support

Scope and Assumption

This project focuses on designing and implementing a three-layer data pipeline architecture, consisting of:

1. Bronze Layer (Raw Ingestion)

- a. Ingest six synthetic CSV datasets into a data lake.
- b. Perform basic data quality checks including null detection, type validation, and duplicate removals.
- c. Preserve raw data in its original format.

2. Silver Layer (Transformation)

- a. Apply business logic including calculate 12-month consumption, identification of Recoverable Assets (RA), detection of Magna Carta Dead Stock (no consumption for 9+ months, no lot expiry) and Expired materials (lot expiry passed), calculate the 100% MC provision, identify stockout risk, and trace sales order impact.

3. Gold Layer (Analytics-Ready)

- a. Load transformed data into a galaxy schema in a data warehouse.
- b. Develop fact tables that store all RA/MC flags and financial values per material per month.
- c. Prepare data for reporting and dashboard development.

In addition, this project assumes that:

1. All synthetic CSV datasets are correct and complete.
2. The business rules are correct.
3. All required information is available in the datasets.
4. The project tools such as Microsoft Azure and Microsoft Power BI can handle the volume of data provided in the datasets.
5. Operational metrics such as lead times remain consistent during analysis.

Significance of Project

This project provides multiple contributions such as:

1. Improved inventory visibility and management.
2. Reduction of financial losses from obsolete stock.
3. Proactive identification of stockout risks.
4. Better customer service through improved order fulfillment.
5. Implementation of business intelligence solutions.

1.3 Project Aim

The aim of this project is to develop an end-to-end data engineering pipeline and dashboard for inventory risk detection, including excess stock, expired materials and potential stock shortages for customer orders.

1.4 Project Objectives

The project objectives is as follows:

- i. To identify and study the inventory risk management domain, as well as the technologies used and existing systems.
- ii. To analyse the provided datasets by applying the Recoverable Assets (RA) and Magna Carta (MC) business logic to identify the inventory risks, such as excess materials, dead stock and expired stock.
- iii. To design and implement a three-layer data pipeline architecture that ingest raw data from six CSV files and transform the data into star schema for analysis.
- iv. To develop a dashboard using Power BI that provides insights into inventory risks and identifies which customer sales orders are at risk due to potential material shortages.

1.5 Project Scope

The project scopes is as follows:

- (a) This project involves creating a three-layer data pipeline architecture using MS Azure tools dedicated to minimizing inventory obsolescence and proactively flagging materials at risk of stockout for the Pittsburgh Plate Glass (PPG) company.
- (b) This project utilizes six synthetic CSV datasets provided by the company to simulate PPG's operational systems, including materials, inventory_snapshot, purchase_orders, production_orders, suppliers, sales_orders.

- (c) The initial layer of the pipeline is designed and implemented to perform the raw ingestion of all six CSV files into a data lake where basic data quality checks are applied while preserving the source data in its original format.
- (d) The second layer applies the PPG's RA/MC business logic to the data which includes calculating 12-month consumption, flagging Recoverable Assets and Magna Carta Dead Stock and Expired materials, executing 100% MC provision calculation, identifying stockout risk, and trace sales order impact.
- (e) The final process involves loading the transformed data into a Galaxy Schema within a data warehouse. This features fact tables that aggregates all RA/MC flags and financial values per material per month, optimized for high-performance reporting and dashboard development.
- (f) This project also involves producing a clean analytics-ready data model and dashboard using Power BI tool to visualize the insights and facts found from the pipeline for the company, providing PPG stakeholders with a single source of truth for inventory health.

1.6 Project Importance

The importance of this project is rooted in three key areas:

- (a) Obsolete inventory will become a financial loss for a global manufacturer like PPG. By implementing the Magna Carta (MC) and Recoverable Assets (RA) business logic into an automated data pipeline, materials that are nearing their shelf-life limit can be identified. This allows the company to reallocate or sell stock before its shelf-life ends, directly avoiding the company's possible financial loss.
- (b) Depletion of materials stock can result in a suspension of an entire production line which can lead to unfulfilled customer orders. By identifying materials at risk of depletion, it will allow the production team to secure material supply ahead of time, preventing lost revenue and maintaining service levels.

- (c) Manual data handling is insufficient for the size of data generated by PPG's global facilities. By building a three-layer Azure pipeline, the project establishes a resilient framework that can handle increasing data loads. This technical foundation reduces the time spent on manual data preparation and increases the speed at which business insights are delivered to stakeholders.

1.7 Report Organization

This report is organized into distinct chapters to provide a structured presentation of the project.

Chapter 1 provides an overview of the study, including the background, problem statement, project aim, objectives, scope and importance. It introduces the key challenges faced by Pittsburgh Plate Glass (PPG) in inventory management and sets the direction of the project.

Chapter 2 presents the literature review, which discusses relevant theories, technologies and previous studies. This chapter establishes the foundation for the proposed solution.

Chapter 3 describes the methodology used in this project, including the justification of the chosen approach, the phases involved and the technologies applied.

Chapter 4 focuses on the system analysis and design. It explains the system architecture, requirements, database design and interface design to illustrate how the solution is structured.

Chapter 5 covers the implementation and testing of the system. It presents the development of the main functions, system interfaces and the testing process to ensure system reliability.

Finally, chapter 6 concludes the report by evaluating the achievement of the project objectives and providing recommendations for future improvements.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

In order to build an end-to-end data engineering pipeline for PPG, this chapter deeply analyzes various literature, theoretical frameworks and industry standards. This chapter aims to lay the theoretical and technical foundation for the construction of data pipelines, focusing on asset recovery and inventory risks.

First of all, this chapter reviews previous studies to understand how industrial data pipelines have evolved from the traditional and isolated extract-transform-load (ETL) process to a modern cloud integration framework. This chapter covers the basic theories of dimensional modeling, inventory expiration logic (Magna Carta) and Medallion data architecture. By combining these theories, this chapter provides a theoretical basis for determining recyclable assets (RA) and calculating financial risks related to material shortages and expired inventory.

2.2 Enterprise Resource Planning

According to Gupta and Kohli (2004), the Enterprise Resource Planning (ERP) system is an integrated information system that unifies and automates the core business processes across organizational functions such as manufacturing, inventory management, and supply chain management, thus improving the operational efficiency.

ERP systems are the foundation to inventory management and risk analysis, providing the required data and process integration to support business operations. In the context of Recoverable Assets (RA) and Inventory Risk Management (IRM), ERP systems enable the identification of excess inventory, obsolete materials, and stockout risks.

2.3 Supply Chain Management

Supply chain management (SCM) is the process by a business to manage the raw materials sources in order to create and deliver a service or product that produces a specific output of value to the customer (Lambert & Cooper, 2000). Effective SCM integrates multiple activities across the supply chain to keep SCM runs smoothly. According to Jean (2024), four key concepts underpin SCM: **demand forecasting**, which maintain optimal inventory levels to prevent overstocking or stockouts; **inventory management**, which aim to minimize the total inventory costs while ensuring the availability; **logistic management**, which select the optimal transportation, warehousing and distribution methods; and finally **procurement and sourcing**, which focus on evaluating and build long term relationships with suitable supplier and contractor.

Among these concepts, inventory management plays an important role as it directly affects the cost efficiency and operational performance. The organization often faces challenges such as excess inventory and dead or expired stock, which can lead to high holding costs and potential waste. Ineffective inventory management can also result in material shortages, causing production delays and unable to fulfil customer orders. Therefore, identifying and managing inventory risk is important to reduce financial loss impact and support more informed and data-driven decision-making in supply chain operations, particularly in inventory stock management.

2.4 Medallion Data Architecture

The lakehouse paradigm and medallion data architecture serve as the foundation for the pipeline's architecture which structures the data processing into progressive refinement layers consisting of Bronze, Silver, and Gold. Extract-Load-Transform (ELT) techniques are becoming more and more popular in modern data engineering as they allow for scalable transformations while preserving raw data (Armbrust et al., 2021).

The Bronze layer carries out simple validation tasks like type checks and duplicate detection while storing raw source data with little modification. Business logic and domain-specific transformations, such as risk scoring, inventory classification, and consumption aggregation, are applied by the Silver layer. The Gold layer is the presentation layer where data is organized into star schemas for optimized querying and dashboarding.

2.5 Dimensional Modelling and Galaxy-Schema

The Gold layer's analytical data model follows the dimensional modeling principles by Kimball and Ross (2013), which arranges data into dimension tables that offer descriptive context and fact tables with quantifiable measures. In online analytical processing (OLAP) contexts, this structure enhances performance and optimizes analytical queries.

Inventory metrics like quantity levels, consumption rates, financial provisions, and risk indicators are recorded in a central fact table that links supporting dimensions such as material, time, supplier, and customer information. In contrast to fully normalized relational schemas, the star-schema structure maintains a business-friendly model that is simpler for analysts to comprehend while enabling effective querying and supporting the analytical requirements of the dashboard (Codd, 1970).

2.6 Microsoft Azure

Microsoft Azure serves as the cloud platform that supports the storage, processing, and integration of data throughout this project. In modern data engineering, Azure provides a **scalable and reliable environment** for building end-to-end pipelines, enabling organizations to manage large volumes of data efficiently while ensuring flexibility, accessibility, and performance.

For the **Recoverable Assets (RA)** and **Inventory Risk Management (IRM)** project, Azure is used as the **main cloud infrastructure** to host the processed data.

and support analytical operations. The gold layer of the pipeline is stored in Azure, allowing it to act as the centralized source for downstream reporting and dashboard development. This cloud-based setup improves data availability and ensures that business users can access updated and consistent information.

One of the main advantages of Azure is its seamless integration with Microsoft Power BI. This integration enables automatic data refresh, ensuring that inventory records, shortage indicators, and expired stock analysis remain current and relevant for decision-making. In addition, Azure supports scalable data handling, which is important for managing multiple datasets and performing transformations across the Bronze, Silver, and Gold layers of the Medallion architecture.

Azure also strengthens the **overall efficiency** of the system by providing a secure and structured environment for data storage and processing. By hosting analytics-ready data in the cloud, the platform allows stakeholders to monitor inventory status, identify stockout risks, detect expired or obsolete materials, and evaluate the impact of these risks on customer sales orders. Therefore, Microsoft Azure plays an essential role in enabling a modern, cloud-based, and decision-oriented data engineering solution for PPG.

2.7 Dashboard Visualization

Dashboard visualization is the final and most user-oriented component in the modern data engineering process. It acts as a **presentation layer**, converting processed and suitable for analysis data into graphical form. Effective data visualization allows stakeholders to quickly understand complex data sets, identify trends, and make proactive, data-based decisions (Dipon et al., 2022). In supply chain and inventory management, visual dashboards are crucial for shifting organizations from passive troubleshooting to proactive strategic planning, allowing them to monitor key performance indicators such as the risk of stockouts and inventory obsolescence in real time.

In the Recyclable Assets (RA) and Inventory Risk Management (IRM) project of Pittsburgh Glass Company (PPG), Microsoft Power BI played a role as a

specialized visualization tool. Power BI was directly connected to the gold layer of the Azure data pipeline, where the data had been optimized and processed in a dimensional star schema format. According to Microsoft (2026), this direct integration can ensure that the dashboard could automatically reflect the latest inventory snapshots and calculation results. By leveraging Power BI, this project established a bridge between the complex backend data transformation and actionable business intelligence (Kolokolov & Zelensky, 2024; O'Connor, 2018).

2.8 Related Previous Study

The application of data-driven techniques in inventory management and risk analysis are widely explored in recent academic literature. Existing studies primarily focus on big data analytics, predictive modeling and supply chain risk management. However, the scope in most studies is limited to specific components of the data lifecycle rather than integrated end-to-end systems.

A study published by IEEE titled Forecasting Obsolescence Risk and Product Life Cycle With Machine Learning, examines the **role of machine learning algorithms in supply chain and inventory management**. The research proposes a proactive forecasting methodology to improve inventory visibility and operational efficiency by predicting when each part will be outdated. The study evaluates supervised learning models, such as Random Forest and Support Vector Machines, to classify parts as active or obsolete and predict specific end-of-life dates.

Research conducted by Institut Teknologi Bandung (2025), titled Ahp-Based Inventory Management Improvement To Reduce Slow-Moving And Dead Stock, investigates **inventory classification and obsolescence detection using data analytics techniques**. The study emphasizes the use of historical consumption patterns and time-based thresholds to identify slow-moving and non-moving inventory. It applies analytical methods to categorize inventory based on usage behavior, demonstrating that temporal consumption analysis is an effective indicator for detecting obsolete or excess stock. This approach aligns with the use of consumption-based rules for inventory evaluation.

An article from the International Journal of Applied Mathematics, titled Data-Driven Decision Making In Manufacturing: Leveraging Power Bi And SQL to Enhance Manufacturing Operations In Erp System, focuses on **integrated data architectures** for inventory risk and financial impact assessment. The study outlines key architectural components, including data ingestion through ETL pipelines, storage and management using SQL databases, and visualization layers built in Power BI. It states that integrating multiple data sources, such as **ERP data, sales, and real-time IoT sensor data**, improves decision-making and enables accurate inventory control. The study discusses how unsold or obsolete inventory contributes to financial loss and highlights the importance of **predictive analytics** to mitigate such risks. It presents models that quantify financial exposure associated with excess inventory and emphasizes the integration of financial metrics into inventory management systems.

Additionally, research published in SciELO Brazil titled Risk assessment model in inventory management using the AHP method, examines supply chain risk and stockout management. The study analyzes factors such as **demand variability, lead time, and inventory policies** in determining stock availability. It identifies that inadequate inventory planning and delayed replenishment can result in stockouts, which directly affect service levels and customer fulfillment. The findings indicate that linking inventory data with operational parameters is necessary to evaluate the impact of stock shortages on supply chain performance.

The studies on big data in inventory management indicate that traditional inventory systems often struggle with dynamic demand conditions, leading to overstock and stockout situations. The integration of large-scale data sources and advanced analytics techniques enables more accurate forecasting and real-time decision-making, thereby improving inventory optimization and reducing operational inefficiencies.

In summary, existing studies demonstrate the importance of big data analytics, consumption-based inventory classification, financial risk evaluation and stockout analysis in supply chain management. However, these works primarily address

individual aspects such as analytics modeling, forecasting, or risk assessment, without providing a fully integrated data engineering pipeline that combines data ingestion, transformation, and business intelligence for comprehensive inventory risk management.

2.9 Summary

This chapter reviews the fundamental concepts and technical frameworks used in the "Recoverable Assets" (RA) and "Inventory Risk Management" (IRM) projects. Firstly, it explores the main challenges in inventory management, including the financial impacts and operational threats arising from obsolescence, slow-moving inventory, and stockouts.

Furthermore, the report details the selected technologies, which are Microsoft Azure and Microsoft Power BI, and explains their capabilities in building a robust and scalable data pipeline. The critical role of dashboard visualization as the final presentation layer is emphasized. By adopting effective visualization strategies in Power BI, such as interactive key performance indicator cards, RA/MC matrices, and sales impact tracking, this project was able to successfully transform complex backend data into actionable business intelligence.

Finally, the theoretical framework, technical insights, and visualization principles established in this literature review lay a solid foundation for the project. These elements will directly guide the development methods of the system and the formulation of the pipeline architecture, and the relevant content will be elaborated in detail in "Methodology" (Chapter 3).

CHAPTER 3

METHODOLOGY

3.1 Introduction

Following the theoretical framework and technological review established in Chapter 2, this chapter will describe the methods adopted for building a data pipeline for the recyclable assets (RA) and inventory risk management (IRM) system of the Pittsburgh Glass Company (PPG). The clear approach is crucial for ensuring that all project goals, from obtaining raw data to creating the final dashboard, can be successfully achieved.

In this chapter, we will explain the adopted methods and break down the project into clear and manageable stages. We will also discuss the main tools that enabled the project to be realized: Microsoft Azure, which handles the three-layer data pipeline, which are bronze, silver, and gold, and Microsoft Power BI, which is used to build interactive dashboards.

3.2 Methodology Selection

The system development approach used in this project is the Waterfall Methodology. The waterfall model is a linear and sequential technique in which the development task of this project is divided into clearly defined phases and each phase needs to be completed before proceeding to the next. This ensures that the business logic for Recoverable Assets (RA) and Inventory Risk Management (IRM) is fully understood before constructing the technical pipeline.

Following the planning and requirements phase, the project moves into design and development phases, where a three-layer Medallion Architecture is developed using Microsoft Azure. By using this architecture, data is planned to flow through the three layer pipeline. Beginning with the Bronze layer for raw data ingestion and data

cleaning, to the Silver layer for applying the Recoverable Assets (RA) and Magna Carta (MC) business logic to the dataset and ending with the Gold layer to load and transform the data into a galaxy schema. Finally, the process concludes with Visualization by Microsoft Power BI to deliver a final dashboard about inventory risks and production prediction for better inventory management. The methodology ensures all project deliverables are met in a structured manner.

3.3 Phases of Methodology

In this project, Waterfall Methodology is applied. The methodology including five phases which are Planning, Analysis, Design, Implementation, and Testing.

3.3.1 Planning

The planning phase defines the project scope, resources, tools, data sources and overall development approach. In this phase, the project objectives, technical requirements, and project workflow were established to ensure the development process aligns with the inventory risk management problem faced by Pittsburgh Plate Glass (PPG).

The project planning began by identifying the business problems related to excess inventory, obsolete materials, stockout risks, and the impact on customer sales orders. Based on these problems, the project objectives were defined, including the development of an end-to-end data pipeline, implementation of Recoverable Assets (RA) and Magna Carta (MC) business logic, and development of a Power BI dashboard for decision support.

During this phase, six synthetic CSV datasets were identified as the main data sources, namely materials, inventory_snapshot, purchase_orders, production_orders, suppliers, and sales_orders. The tools and technologies required for development were also selected, including Microsoft Azure for data engineering and Microsoft Power BI for visualization.

A project methodology based on the Medallion architecture was planned, consisting of Bronze, Silver, Gold and Curated layers to structure data ingestion, transformation, and analytics-ready reporting. The project timeline, system workflow, and deliverables were also outlined to ensure the project can be completed systematically within scope.

3.3.2 Analysis

The analysis phase focuses on studying the datasets, identifying business requirements, and analyzing the logic needed to support inventory risk detection and decision-making.

In this phase, the six provided datasets were analyzed to understand their structure, attributes, relationships, and relevance to the project objectives. Key fields such as material stock quantity, consumption rate, supplier information, lead time, expiry dates, and customer sales orders were examined to determine how they support the implementation of inventory risk logic.

Business rules provided by PPG were analyzed to define the logic for identifying Recoverable Assets (RA), Magna Carta Dead Stock, Magna Carta Expired materials, financial provision calculations, stockout risk detection, and tracing affected sales orders. These rules formed the basis for data transformations performed in the Silver layer.

System requirements were also analyzed from both functional and non-functional perspectives. Functional requirements include data ingestion, inventory classification, risk detection, financial provision calculation, and dashboard reporting. Non-functional requirements include scalability, data quality, performance, and analytical readiness.

Additionally, data relationships between materials, suppliers, dates, and sales orders were analyzed to support the dimensional modeling process and galaxy schema design. This analysis ensures the data model supports efficient querying and dashboard visualization.

3.3.3 Design

The design phase translates requirements into a structured data architecture and model. For this project, the focus is on designing the data pipeline architecture and data model.

The data pipeline is designed using the Medallion architecture to ensure scalability, data quality, and analytical readiness. In the Bronze layer, the six raw datasets in CSV format are ingested and stored in its original form to maintain traceability. The Silver layer focuses on data cleaning and format standardization by removing duplicates, fixing nulls and data types. Then, the Gold layer performs data transformation by applying PPG's key business rules, including Recoverable Assets (RA) classification, Magna Carta (MC) Dead Stock and Expired detection, stockout risk calculation, and sales order impact analysis. Finally, the Curated layer organizes the processed data into a star schema to optimize the reporting and dashboard development. This layered approach supports modern data engineering practices by clearly separating raw data, transformed data, and analytics-ready insights.

To support analytical queries, a star schema is designed with one central fact table and multiple dimension tables. Figure 3.3.1 illustrates the two central fact tables named *fact_inventory_status* and *fact_sales_impact*. The *fact_sales_impact* table stores inventory-related metrics such as stock levels, stock expiry indicators, and consumption data, while the *fact_inventory_status* table stores customers and sales information. Both fact tables are connected through the *dim_material* and *bridge_bom* tables.

The *dim_material* and *dim_supplier* tables provide descriptive information about materials and suppliers. The *bridge_bom* table links finished products to raw materials, allowing analysis of how sales demand affects inventory consumption.

This galaxy schema enables integrated reporting across inventory management, supplier reliability analysis, and sales operations within a scalable enterprise warehouse design.

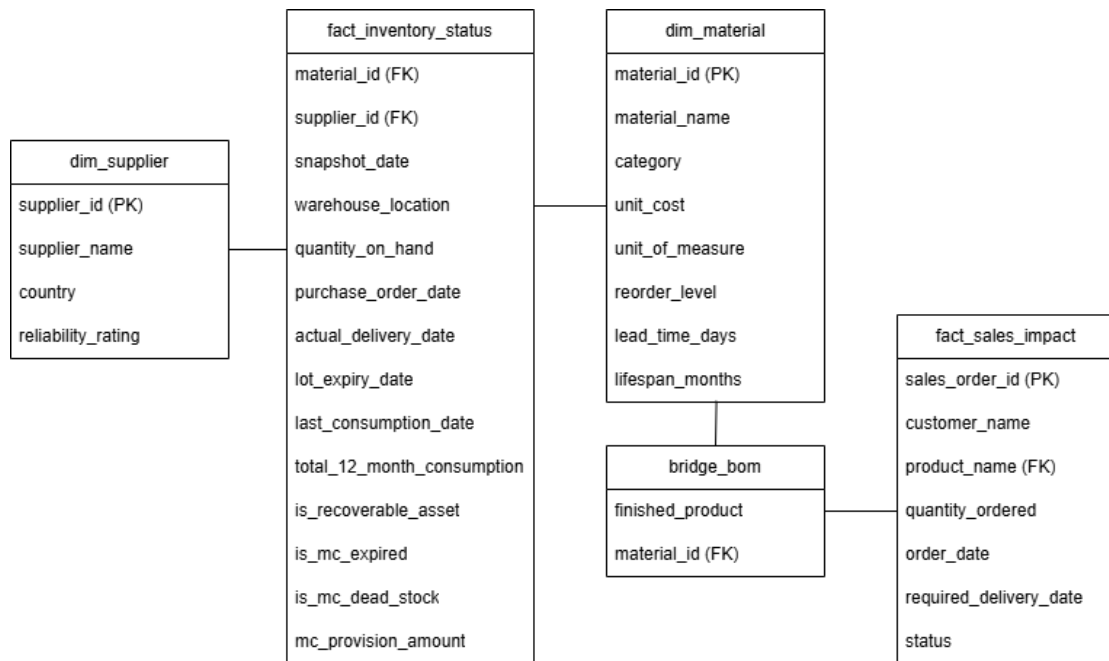


Figure 3.3.1 Galaxy Schema Design

3.3.4 Implementation

The implementation phase transforms the design specifications into a functional three-layer data pipeline. This phase began by setting up the Microsoft Azure environment and securing it through Azure Active Directory for identity management. The foundation of the Medallion Architecture was built by configuring Azure Data Lake Storage Gen2 (ADLS Gen2) with a hierarchical folder structure. This setup is to ensure that data could be stored and moved securely between the Bronze, Silver, and Gold layers of the project.

The development of the data pipeline was executed through three technical stages:

1. **Raw Ingestion** (Bronze): We developed Azure Data Factory (ADF) copy pipelines to ingest six primary CSV files into the data lake. During this stage, we implemented automated data quality checks to detect null values and duplicates, ensuring the raw source data remained intact while maintaining a clean foundation.
2. **Transformation** (Silver): Using Azure Synapse Analytics, we engineered the core transformation logic. The 12-month consumption patterns calculations

were written using Spark-based scripts and the complex flags for Recoverable Assets (RA) and Magna Carta (MC) are applied. This included logic to identify "Dead Stock" and "Expired Materials" to accurately calculate 100% MC financial provisions and stockout risks.

3. **Analytics-Ready (Gold):** The refined data was modeled into a Galaxy Schema within a Synapse SQL Pool. The fact_inventory_status table was constructed to serve as the final repository for all financial values and RA/MC flags, optimized for high-speed retrieval by Power BI.

To complete the implementation, Orchestration and Validation were performed to ensure the pipeline runs automatically and accurately. Automated triggers were set in ADF to schedule daily data refreshes, and end-to-end testing was conducted using sample datasets to verify that the final Power BI reports matched the source data. This structured implementation, as detailed in the Gantt Chart (Figure 3.3.2), successfully transformed raw data into a scalable, automated business intelligence solution.

3.3.5 Testing

The testing phase is to ensure the functionality of the built data pipeline can work normally and accurately. This phase consists of two types of testing:

1. **Black Box Testing:** This test was performed to verify the functionality of the entire pipeline without examining the internal code structure. The testing focused on validating the inputs and outputs of each process, such as data ingestion, transformation, and loading into the final destination.
2. **White Box Testing:** This test was performed to examine the internal logic and code implementation of the pipeline components. The testing focused on validating the transformation scripts, business rules, and calculations implemented in Azure Databricks, such as the Recoverable Assets (RA) calculations, Magna Carta (MC) Dead Stock identification, and financial provision computations.

This testing approach ensures that both the external functionality and internal logic of the data pipeline operate correctly and meet the project requirements.

3.3.6 Gantt Chart

The project gantt chart consist of 11 tasks, which are:

- (a) Requirement Gathering & Analysis
- (b) Project Planning & Risk Assessment
- (c) Pipeline & Data Architecture Design
- (d) Azure Resource Provisioning (AD, ADF, Synapse)
- (e) ADLS Gen2 Data Lake Configuration
- (f) Bronze Layer: Raw Ingestion Development
- (g) Silver Layer: RA/MC Transformation Logic
- (h) Gold Layer: Star Schema & Analytics-Ready
- (i) Visualization: Building Business Dashboard
- (j) Data Orchestration (ADF Triggers/Scheduling)
- (k) Data Validation & End-to-End Testing

The duration of each task is shown in Figure 3.3.2:

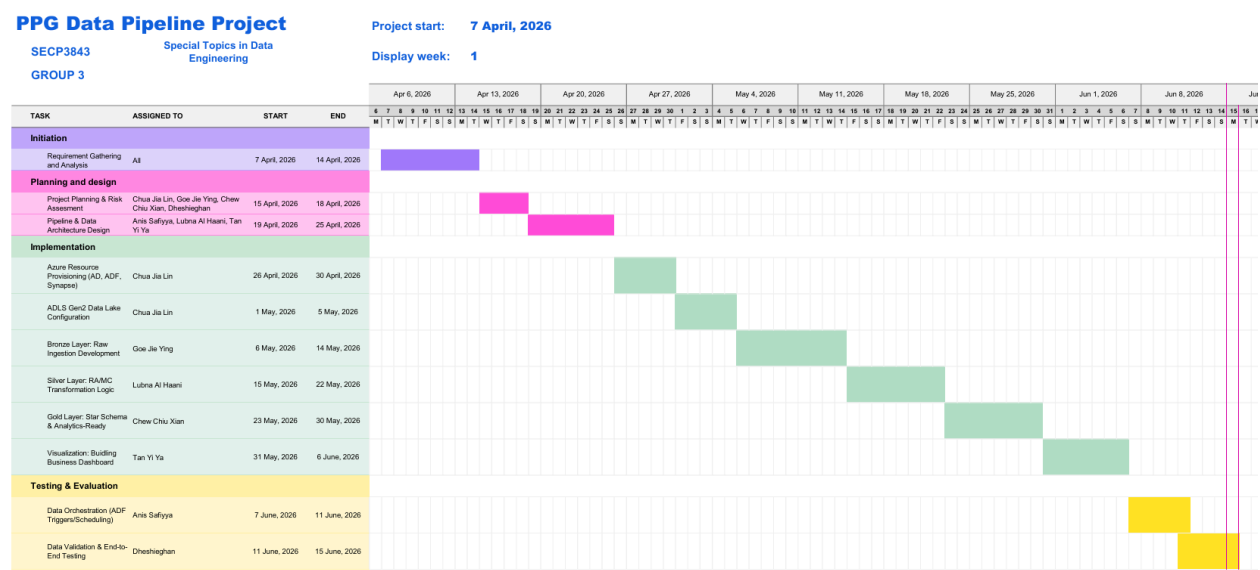


Figure 3.3.2: Project Gantt Chart

3.4 System Requirement Analysis

This section describes the system requirements for developing the cloud-based data pipeline using Microsoft Azure services and Power BI.

3.4.1 Software Requirements

The software requirements consist of cloud services, data integration tools, storage platforms, transformation engines and business intelligence tools needed to support the end-to-end pipeline. Microsoft Azure provides the main infrastructure for data engineering processes, while Power BI is used for reporting and dashboard visualization.

3.4.1.1 Microsoft Azure Services Required

The following Azure services are required for implementing the data pipeline:

- (a) Azure Data Lake Storage Gen2 (ADLS Gen2)
- (b) Azure Data Factory (ADF)
- (c) Azure Synapse Analytics
- (d) Azure SQL Database / Synapse SQL Pool
- (e) Azure Active Directory (Azure AD)
- (f) Microsoft Power BI Service

These services work together to support data ingestion, storage, transformation, warehousing, governance, and reporting.

3.4.1.2 Azure Data Lake Storage Gen2 (ADLS Gen2)

Azure Data Lake Storage Gen2 serves as the main storage platform for the entire data pipeline. It stores all datasets across the Bronze, Silver, and Gold layers in a centralized and scalable environment. This structure improves data traceability and ensures a clear separation between raw and trusted data.

3.4.1.3 Azure Data Factory (ADF)

Azure Data Factory is used as the orchestration and data integration service for the pipeline. It is responsible for extracting data from multiple source systems and

loading it into the Bronze layer of the data lake. ADF also manages scheduling, monitoring, automation, and dependency handling for ETL and ELT workflows. Through trigger-based execution and pipeline monitoring, ADF ensures reliable and consistent movement of data across all layers of the architecture.

3.4.1.4 Azure Synapse Analytics

Azure Synapse Analytics provides the processing and analytical engine required for transforming data across the Bronze, Silver, and Gold layers. Synapse is used to create external tables, views, and transformation logic that convert raw data into structured and optimized datasets. It also supports Delta Lake operations and enhances analytical performance for enterprise reporting.

3.4.1.5 Data Warehouse Layer

The final Gold layer is loaded into a centralized data warehouse that stores curated business data for analytics and reporting. This layer typically consists of fact tables, dimension tables, aggregated KPIs and subject-specific data marts designed using star schema modeling. SQL views and optimized query structures are implemented to improve reporting performance and simplify business analysis.

3.4.1.6 Power BI

Power BI is used as the visualization and business intelligence platform for the project. It connects directly to the Gold layer datasets and enables the development of interactive dashboards, management reports, and KPI monitoring systems. Scheduled refresh features ensure dashboards remain updated with the latest processed data from the warehouse.

3.4.1.7 Security and Access Management

Security and access control are managed through Azure Active Directory, Role-Based Access Control (RBAC), Managed Identity, and storage access

permissions. These mechanisms ensure that only authorized users can access specific services, datasets and reporting layers based on their job responsibilities.

3.4.2 Hardware Requirements

Since the proposed system is cloud-based, most computing resources are provisioned and managed within Microsoft Azure. Only client-side machines are required for development, administration, monitoring, and dashboard access.

3.4.2.1 Client Machine Requirements

Table 3.4.2.1: Machine Requirements

Component	Minimum Requirement
Processor	Intel Core i5 / AMD Ryzen 5
RAM	8 GB (16 GB recommended)
Storage	256 GB SSD
Operating System	Windows 10 / Windows 11
Browser	Google Chrome / Microsoft Edge
Internet Connection	Stable broadband connection

3.4.2.2 Server Infrastructure

Physical servers are not required because Microsoft Azure provides fully managed cloud infrastructure for storage, processing, orchestration, and analytics. Services such as Synapse Spark pools, SQL pools, ADLS Gen2, and ADF dynamically allocate resources based on workload demand. This eliminates the need for organizations to purchase, maintain, and upgrade physical servers manually. It also reduces infrastructure costs, minimizes downtime risks, and improves system scalability.

3.4.2.3 Network Requirements

A stable and secure network connection is essential for accessing Azure services, monitoring pipeline execution, refreshing Power BI datasets and supporting collaboration among team members. A minimum broadband speed of 20 Mbps is recommended to support smooth cloud operations and efficient data transfer between services.

3.5 Summary

This chapter outlines the method of building a cloud-based inventory risk management (IRM) and recoverable asset (RA) system for PPG. The project adopts a waterfall development method to provide a systematic and linear development process from planning to testing. The system architecture adopts the Medallion structure, consists of three layers. The initial inventory data is first received in the bronze layer, then processed and refined in the silver layer, and finally converted for analysis purpose in the golden layer. During the conversion process, the system applies business rules such as recyclable assets (RA), Magna Carta (MC) dead inventory, MC expired materials, financial reserve calculation and out-of-stock risk detection.

The system uses Microsoft Azure services, including Azure Data Factory for arrangement, Azure Data Lake Storage Gen2 for storage, Azure Synapse Analytics for data processing and modeling, and Power BI for dashboard visualization. The structure of Galaxy Schema is designed to promote efficient analytical queries and reports. In order to ensure reliability and accuracy, the system has undergone unit testing, integration testing, end-to-end testing and data verification. In short, this method provides a scalable automated data engineering solution that can transform raw inventory information into important business insights, thus enhancing inventory management and decision-making capabilities.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the system analysis and design of the proposed cloud-based inventory management and business intelligence system. The purpose of this chapter is to explain the current business process, identify the problems in the existing manual inventory monitoring method, and describe the requirements needed for the proposed system.

The proposed system focuses on improving inventory risk detection by integrating multiple operational datasets such as materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders. These datasets are processed through a cloud-based data engineering pipeline using Microsoft Azure services and visualized through Power BI dashboards.

This chapter also discusses the system requirements, current system analysis, proposed system architecture, system design diagrams, database design, and interface design. The overall aim is to show how the proposed system can support better decision-making by automating data ingestion, transformation, reporting, and inventory risk analysis.

4.2 System Analysis

System analysis is an important phase in system development because it helps to understand the current process, user needs, business problems, and system requirements. In this project, system analysis is conducted to study how inventory data is currently managed and how the proposed system can improve the existing workflow.

The current inventory process is highly dependent on separate CSV files and manual reporting. Data from procurement, warehouse, production, and sales departments are not fully integrated, causing delays in analysis and decision-making. Therefore, the proposed system is designed to centralize inventory data, automate

business rule processing, and provide interactive dashboards for management and business users.

The system analysis focuses on the case study, organization structure, and requirements gathering techniques used to identify the functional and non-functional needs of the proposed inventory management system.

4.2.1 Case Study

The case study for this project is based on an inventory management scenario in a manufacturing organization, represented using the PPG Inventory Management System. In manufacturing operations, inventory data is important because it affects production planning, purchasing decisions, customer order fulfilment, and financial reporting.

The existing business environment consists of multiple operational datasets, including material master data, inventory snapshots, supplier records, purchase orders, production orders, and sales orders. These datasets represent the daily activities of departments such as Procurement, Warehouse, Production, and Sales.

However, the current inventory monitoring process is mostly manual. Each department maintains its own records, and users need to manually collect, combine, and analyze CSV files to identify issues such as expired materials, dead stock, recoverable assets, stockout risks, and customer order shortages. This manual process increases the possibility of human error, delays reporting, and makes it difficult for management to obtain real-time visibility into inventory performance.

To solve these issues, the proposed system uses a cloud-based data engineering solution. The system ingests data into Azure Data Lake Storage using Azure Data Factory, processes and transforms the data using Azure Databricks, models the analytics-ready data using Azure Synapse Analytics, and visualizes the results using Power BI. The solution also follows the Medallion Architecture, which separates data into Bronze, Silver, and Gold layers for better data quality, traceability, and scalability.

Through this system, business users can monitor inventory risk, identify

Recoverable Assets (RA), detect Magna Carta (MC) Dead Stock and expired materials, calculate financial exposure, and view stockout risks through interactive dashboards. This helps the organization make faster and more accurate inventory-related decisions. The uploaded Chapter 4 draft also identifies these same key functions, including multi-source ingestion, data validation, RA/MC logic, stockout prediction, star schema loading, and Power BI reporting.

4.2.2 Company Organization Structure

The organization structure for the case study consists of several departments that are involved in inventory management and supply chain operations. Each department plays a different role in generating, maintaining, and using inventory-related data.

The main departments involved are:

1. Procurement Department

The Procurement Department is responsible for managing supplier information, purchase orders, and material purchasing activities. This department ensures that raw materials are ordered from suppliers and delivered according to production requirements.

2. Warehouse Department

The Warehouse Department manages inventory stock levels, material storage, expiry dates, and stock movement. This department plays an important role in ensuring that materials are available when needed and that expired or slow-moving materials are identified.

3. Production Department

The Production Department uses raw materials for manufacturing activities. Production orders help to show the consumption of materials and allow the system to calculate material usage patterns. This information is important for identifying excess stock, dead stock, and production shortage risks.

4. Sales Department

The Sales Department manages customer orders and delivery requirements. Sales order data is important because it helps the system identify whether stock shortages may affect customer order fulfillment.

5. Management / Business Users

Management and business users use reports and dashboards to monitor inventory performance, financial exposure, stockout risks, and customer order impact. They require accurate and timely information to support decision-making.

In the current system, these departments operate separately and maintain their own records. As a result, data becomes scattered across different CSV files, making it difficult to obtain a complete view of inventory performance. The proposed system integrates data from all departments into a centralized data pipeline to improve reporting and decision-making.

4.2.3 System Requirements Gathering Techniques

System requirements gathering is conducted to identify the needs, problems, and expectations of users before designing the proposed system. In this project, several techniques are used to gather requirements for the inventory management and business intelligence solution.

1. Document Analysis

Document analysis is used to study the available CSV datasets, including materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders. These datasets help identify the type of data required by the system and how each dataset contributes to inventory analysis.

Through document analysis, important data fields such as material ID, stock quantity, expiry date, supplier details, purchase order status, production consumption, and customer sales orders are identified. This helps define the data ingestion and transformation requirements.

2. Existing Process Observation

The existing inventory process is analyzed by observing how data flows between departments. The current process shows that Procurement, Warehouse, Production, and Sales departments maintain separate records. Users then manually collect and combine CSV files before performing inventory analysis.

This observation helps identify key problems such as data silos, manual reporting, delayed decision-making, and lack of real-time inventory visibility.

3. User Requirement Analysis

User requirement analysis is conducted by identifying the needs of different users, such as data engineers, inventory managers, business analysts, and management users.

For example, data engineers need a structured pipeline to ingest, clean, transform, and load data. Inventory managers need to monitor stockout risks, expired materials, and dead stock. Business analysts and management users need interactive dashboards to view KPIs, financial exposure, and inventory risk status.

4. Business Rule Analysis

Business rule analysis is used to define the logic required for inventory risk detection. The main business rules include identifying Recoverable Assets (RA), Magna Carta (MC) Dead Stock, expired materials, stockout risks, and financial provision amounts.

These business rules are important because they allow the proposed system to automate analysis that was previously done manually.

5. System Requirement Review

After identifying the user needs and business rules, the requirements are reviewed and categorized into functional and non-functional requirements. Functional requirements describe what the system must do, such as ingesting data, validating data quality, applying RA/MC logic, loading a star schema, and generating dashboards.

Non-functional requirements describe how the system should perform, such as scalability, security, query performance, reliability, and data traceability.

4.3 System Requirements

The system requirements define the explicit capabilities, processes, and constraints that the cloud-based data pipeline and business intelligence solution must satisfy to meet the project objectives. These requirements are categorized into functional requirements (what the system must do) and non-functional requirements (how the system must perform).

4.3.1. Functional Requirements

Functional requirements describe the core data engineering tasks, transformations, and visual outputs that the system must perform automatically to support inventory risk detection and decision-making. Table 4.3.1 shows 10 functional requirements of the system.

Table 4.3.1: Functional Requirements

Requirement ID	Requirement Name	Description
FR-01	Multi-Source Data Ingestion	The system must ingest six synthetic transactional datasets (materials, inventory_snapshot, purchase_orders, production_orders, suppliers, sales_orders) from flat CSV files into the Bronze layer of the data lake.
FR-02	Data Quality Validation	The ingestion pipeline must automatically execute fundamental data quality checks, including null value detection, type validation, and duplicate removal, while preserving the raw historical state.

FR-03	Consumption & Asset Tracking Logic	The system must automatically compute the 12-month trailing material consumption pattern and apply business rules to flag Recoverable Assets (RA) where stock levels exceed 12 months of usage.
FR-04	Obsolescence Risk Identification	The system must process business logic to flag Magna Carta (MC) Dead Stock (no material consumption for 9+ months with no lot expiry) and Expired Materials (lot expiry date passed).
FR-05	Financial Provision Calculation	The transformation engine must calculate the total financial exposure by applying a 100% financial write-down provision to all materials classified under MC Dead Stock or Expired parameters.
FR-06	Stockout Risk Prediction	The system must calculate inventory replenishment lead times and flag materials at risk of dropping below required production safety thresholds.
FR-07	Sales Order Shortage Impact Tracing	The pipeline must correlate material stock shortages directly with active customer sales orders to proactively identify delivery fulfillment risks.
FR-08	Star Schema Dimensional Loading	Transformed analytics-ready data must be loaded into a star schema containing a central fact_inventory_status table linked to

		relevant dimensions (dim_material, dim_supplier, dim_date, dim_sales_order, dim_warehouse).
FR-09	Interactive BI Dashboard Reporting	The visualization tier must deliver an interactive dashboard featuring executive KPI cards, an RA/MC visibility matrix, stockout warnings, and customer order risk drill-downs.
FR-10	Automated Data Refreshes	Orchestration triggers must be established to execute the end-to-end data pipeline sequentially and refresh the dashboard insights automatically on a scheduled daily routine.

4.3.2. Non Functional Requirements

Non-functional requirements outline the operational characteristics, constraints, and quality standards that ensure the system is reliable, secure, and optimized for scalability. Table 4.3.2 shows 5 non functional requirements of the system.

Table 4.3.2: Non Functional Requirements

Requirement ID	Quality Attribute	Constraint / Specification
NFR-01	Scalability	The data architecture must leverage cloud infrastructure (Azure Synapse Spark and SQL Pools) capable of scaling horizontally and vertically to compute expanding transactional data volumes without manual server upgrades.
NFR-02	Query & Reporting	Analytical queries initiated by

	Performance	dashboard interactions must execute efficiently. This is accomplished by utilizing a star-schema architecture optimized for high-performance indexing, keeping visualization load times beneath a strict 5-second threshold.
NFR-03	Data Traceability & Lineage	The system must enforce a strict separation between raw, cleaned, and modeled data layers (Bronze, Silver, Gold). This architectural division ensures full structural audit trails from initial ingestion down to final reporting metrics.
NFR-04	Security & Access Control	Data governance must be managed securely through Azure Active Directory (Azure AD). Role-Based Access Control (RBAC) and Managed Identities must be strictly implemented to prevent unauthorized access to backend storage layers and restricted business insights.
NFR-05	Availability & Reliability	The underlying managed serverless infrastructure must minimize downtime risks, leveraging integrated failover protocols and ADF monitoring pipelines to flag pipeline execution bottlenecks or connection failures.

4.4 Current System Analysis

The current inventory monitoring and recoverable asset tracking process in manufacturing organizations is highly dependent on separate operational systems and manual reporting methods. In the context of inventory risk management, all data related to raw materials, stock levels, supplier performance, production consumption, and customer demand are stored separately across multiple operational systems. This creates difficulties in obtaining a centralized and accurate view of inventory level and financial exposure.

For this project, the existing environment is represented using the six synthetic CSV datasets that simulate operational data from procurement, warehouse, production, and sales systems.

4.4.1 Existing System Overview

The current operational process relies on separate CSV-based transactional records generated from different departments. Each dataset serves a different business function:

- **materials.csv**: Stores basic data for raw materials
- **inventory_snapshot.csv**: Records monthly inventory stock levels and expiry dates
- **purchase_orders.csv**: Tracks purchase orders from suppliers and delivery status
- **suppliers.csv**: Stores supplier information and their reliability ratings
- **sales_orders.csv**: Records customer sales order information
- **production_orders.csv**: Stores production orders details including product and material used

Currently, these files are handled independently without a centralized system. Therefore, inventory analysis is performed manually using spreadsheets or specific reporting tools.

4.4.2 Current Business Process

This section explains the workflow of the current inventory management process in the organization.

The flowchart in Figure 4.4.2 illustrates the current inventory management process, where multiple departments such as Procurement, Warehouse, Production, and Sales departments operate independently in parallel, each maintaining separate data records.

Since these systems are not integrated, employees must manually collect and combine data (CSV files) from different sources. After merging the data, manual analyses such as identifying obsolete stock, calculating inventory provisions, and assessing stockout risks are performed.

Finally, reports are manually generated for decision-making. This fragmented and manual workflow increases the risk of making inaccurate and slow business decisions.

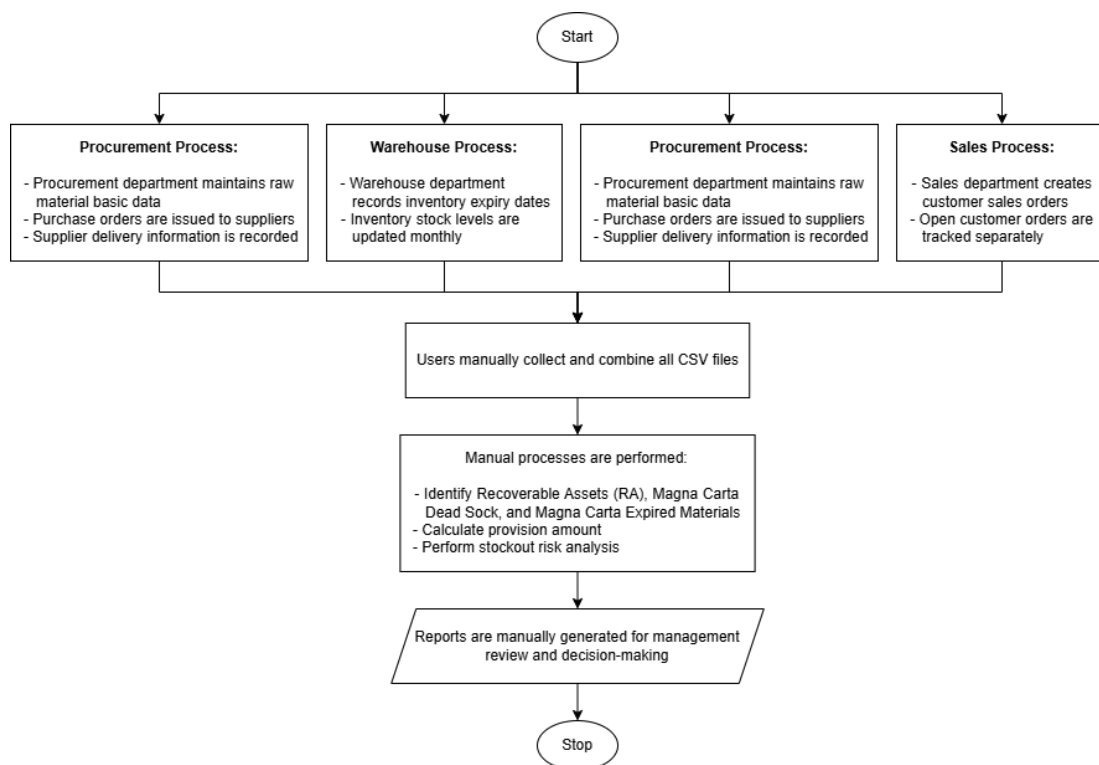


Figure 4.4.2 Flowchart Diagram for Current Business Process

4.4.3 Problems in Current System

Several operational and analytical limitations exist in the current system. These problems include:

- 1. Data Silos**

Data is distributed across multiple CSV files without integration. Employees must manually combine datasets before performing analysis. This increases processing time and causes inconsistency.

- 2. Lack of Real-time Visibility**

The current process does not provide centralized visibility into inventory status. Management cannot quickly identify inventory issues such as excess inventory, expired materials, dead stocks, stockout risk, and customer order impact. This causes late corrective actions to solve these issues.

- 3. Difficulty Identifying Recoverable Assets (RA) and Magna Carta (MC) Materials**

The existing system cannot automatically identify RA, MC Dead Stock (materials with no consumption for more than 9 months), and MC Expired materials (expired materials). Manual identification is very time-consuming and human error may occur.

- 4. Poor Inventory Risk Management**

The current system lacks predictive capabilities for inventory management. Inventory planners cannot easily determine overstocking, understocking, and potential production disruptions. As a result, inventory issues cannot be predicted before the issue occurs.

- 5. Lack of Sales Order Impact Analysis**

When inventory shortages occur, the current system cannot trace which customer sales orders are affected. Employees must manually analyze the open sales orders, required finished products, required raw materials, and inventory availability. This causes delays in customer communication and order fulfillment.

6. Lack of Centralized Reporting

Reporting is generated manually. Therefore, the existing reports are not standardized, require manual updates, and do not support real-time analytics. This causes the lack of timely and accurate insights for decision-making.

4.4.4 Need for Proposed System

To overcome the identified issues, a modern cloud-based data engineering solution is required. The proposed system using Microsoft Azure and Microsoft Power BI provides:

1. Centralized Data Integration

All operational datasets are integrated into a unified data storage in Microsoft Azure to prevent data silos.

2. Automated Data Processing

Business rules for Recoverable Assets (RA), Magna Carta (MC) Dead Stock and MC Expired materials classification are automatically applied to prevent manual processes.

3. Scalable Data Pipeline

The Medallion architecture contains 3 layers, which are the Bronze, Silver, and Gold layer. The Bronze layer stores raw data ingested from source systems. The Silver layer applies business rules to identify RA and MC materials. The Gold layer contains business-ready data optimized for reporting and analytics. These layers improve maintainability and scalability.

4. Real-Time Analytics

Interactive dashboards provide real-time visibility into inventory risk and monitor total financial write-downs.

5. Improved Decision-Making

Management can actively respond to issues such as excess inventory, insufficient stock levels, financial write-down risks, and customer order disruptions with the proposed solution.

4.5 System Design

This system design section illustrates the internal and external design of the inventory management system solution. This involves the system architecture, database design, and the interface design.

4.5.1 System Architecture

The system architecture illustrates the overview of the system as shown in Figure 4.5.1 below.

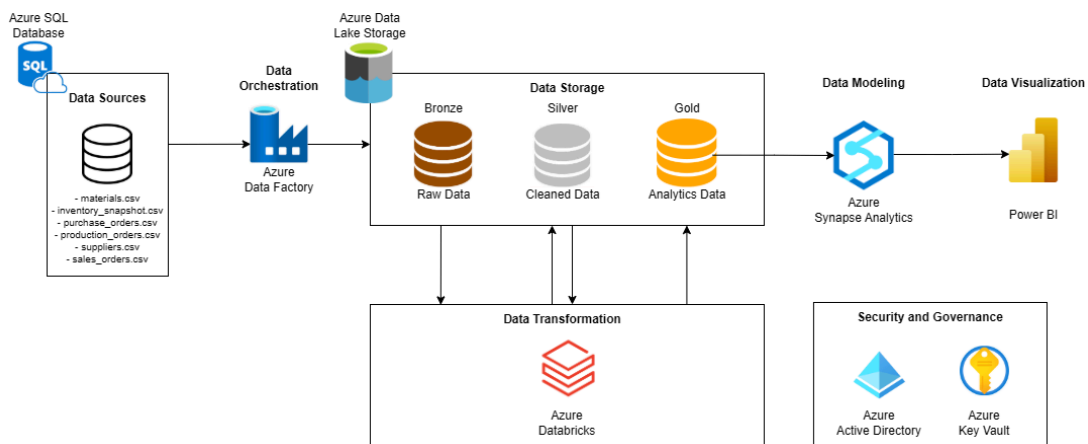


Figure 4.5.1 System Architecture Diagram

4.5.2 System Architecture Components

The system data pipeline is designed using a layered architecture consisting of data sources, data orchestration, data storage, data transformation, data modeling and data visualization layers. Each component plays a critical role in converting raw supply chain data into useful information for inventory risk analysis.

4.5.2.1 Data Sources Layer

The data source layer consists of six csv files of inventory and supply chain related transactional records. These datasets, including materials, inventory snapshots, purchase orders, production orders, suppliers and sales orders, represent raw operational data collected from different departments within the supply chain. In this project, the data is stored in the Azure SQL Database and managed using SQL Server Management Studio (SSMS). This layer serves the starting point for the data pipeline.

4.5.2.2 Data Orchestration Layer

Azure Data Factory (ADF) is used as the data orchestration tool in this system. It is responsible for extracting the data from SQL Database and loading it into Azure Data Lake Storage Gen 2 to be stored. ADF automates the data pipeline through scheduled triggers, monitoring and workflow management. This is to ensure smooth data movement across all stages in the data pipeline.

4.5.2.3 Data Storage Layer

Azure Data Lake Storage Gen2 (ADLS Gen2) serves as the storage platform for our system and implements the Medallion Architecture, which consists of three layers:

1. Bronze Layer (Raw Data)

The Bronze layer stores raw data extracted from the data sources without any transformation. Data recovery using these original data can be done if needed.

2. Silver Layer (Cleaned Data)

The Silver layer contains cleaned and standardized data. After the data is processed, the resulting output is stored in the Silver layer through Databricks transformation processes.

3. Gold Layer (Analytics-Ready Data)

The Gold layer stores curated and business-ready datasets. It contains structured and processed data that has been enriched with business logic, such as Recoverable Assets (RA), Magna Carta (MC) in Databricks. This data are ready for further analysis.

4.5.2.4 Data Transformation Layer

Databricks is used as the primary data transformation engine in this system. It performs two main processing tasks. First, Databricks cleans and standardizes raw data, then writes the output to the Silver layer in ADLS. Second, Databricks applies business rules, including Recoverable Assets (RA) and Magna Carta (MC) logic on the cleaned data. In this stage, the final processed datasets are generated and the analytics-ready data is written to the Gold layer.

4.5.2.5 Data Modeling Layer

For the data modeling layer, Azure Synapse Analytics is used. It reads the curated data, which business logic was successfully applied in the previous stage, from the Gold Layer. Then, it constructs a galaxy schema data model that consists of fact tables, which stores inventory metrics and multiple dimension tables to support analytical processing in the data visualization layer.

4.5.2.6 Data Visualization Layer

Power BI is used as the final visualization layer of the system. It connects to the star schema created in Azure Synapse Analytics to create interactive dashboards. With proper connection in the schema, the dashboard can provide insights into inventory and stockout risks, classify Recoverable Assets and Magna Carta, which enables the decision makers to monitor the inventory performance and make data-driven decisions easier.

4.5.2.7 Security and Governance Layer

Azure Active Directory (AAD) and Azure Key Vault is used to enhance the security and governance of the system. Azure Active Directory manages user authentication and access control across Azure services to ensure only authorized users can access the data pipeline. Azure Key Vault is used to securely store and manage sensitive information such as database credentials and connection strings. It avoids the credentials to be hard-coded within the system to improve overall security and compliance with cloud security best practices.

4.5.3 System Design

This section illustrates the proposed design diagrams of the cloud-based inventory risk management and business intelligence system. Also, the system workflow, data flow, user interaction and database relationship are all presented intuitively in various diagrams, including use case diagram, data flow diagram, sequence diagram, and activity diagram. These diagrams ensure the correct integration of all system components and provide guidance for the development of inventory analysis solutions.

4.5.3.1 Use Case Diagram

Figure 4.5.3.1 demonstrates the use case diagram to show the interaction between user and the proposed inventory management system. The users or primary actors includes data engineer, inventory manager, and business analyst.

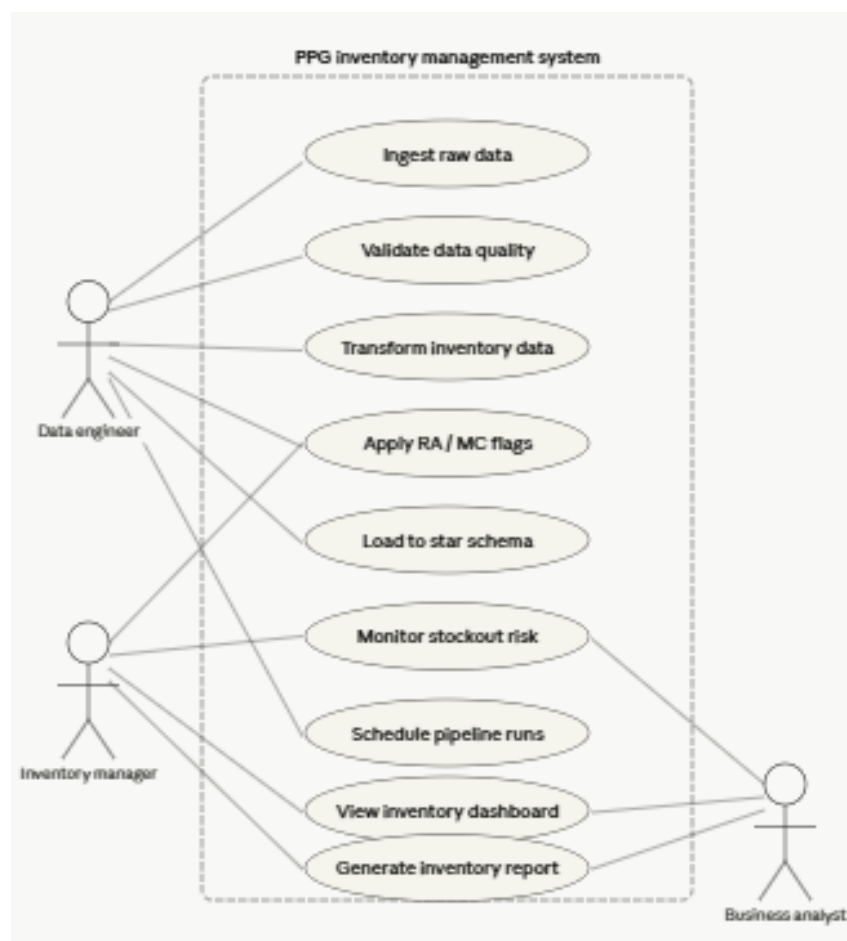


Figure 4.5.3.1: Use Case Diagram

As shown in Figure 4.5.3.1, data engineer is responsible in monitoring data ingestion, validating data quality, executing data transformations, loading data into the schema, and managing the data pipeline.

Meanwhile, the inventory manager will use the system to monitor inventory levels through dashboard and report, identify Recoverable Assets (RA), detect Magna Carta (MC) Dead Stock and Expired Materials, and monitor stockout risks.

Other than that, business analyst use the dashboard to perform inventory analysis and analyse KPI. Then, the business analyst will generate an inventory report based on the analysis done.

4.5.3.2 Activity Diagram

The activity diagram outlines the order of operations recommended by the system, from data input to dashboard visualization. Figure 4.5.3.2 highlights the automation function of the recommended system and reduces the dependence on manual inventory evaluation.

The process begins with obtaining operational data sets from the source system. Azure Data Factory retrieves data and transmits it to the bronze layer. Then, the system performs data quality verification, including detecting duplicates, checking empty values and verifying data types.

The verified data is processed in Databricks, which uses business rules to accurately locate recyclable assets (RA), backlog inventory stipulated in the Magna Carta, expired materials stipulated in the Magna Carta, out-of-stock risks and financial reserves. Then, the modified data is imported into the golden layer and structured in Azure Synapse Analytics.

Finally, Power BI extracts the data ready for analysis and creates dashboards for business users. When the user logs in to the dashboard to track inventory efficiency and risk indicators, the process ends.

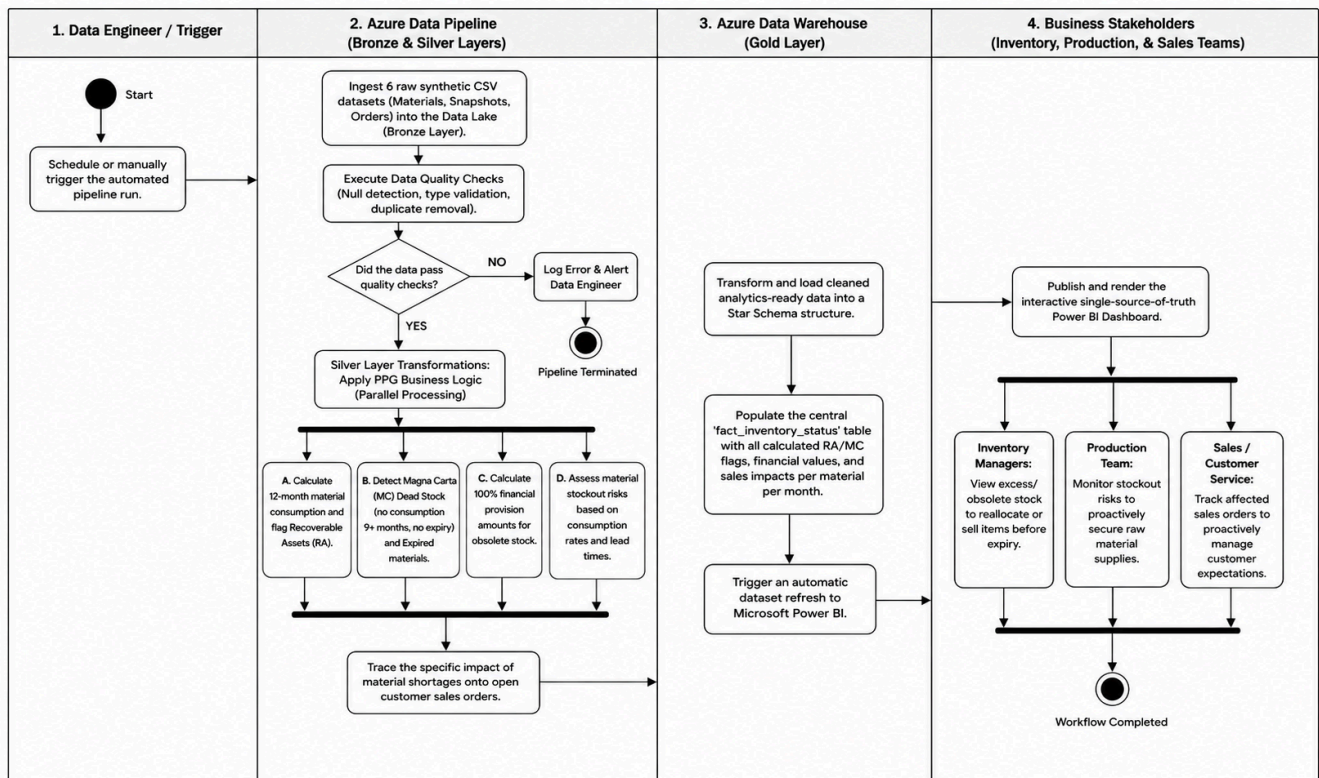


Figure 4.5.3.2: Activity Diagram

4.5.3.3 Sequence Diagram

The sequence diagram depicts the timing interaction between system elements during pipeline execution. The sequence diagram in Figure 4.5.3.3 below, shows the interaction between Azure services and highlights the automated features of the recommended data pipeline.

The process begins with the Azure data factory activation plan pipeline, and then retrieves the source data from the Azure SQL database and saves it to the copper layer of ADLS Gen2. Databricks then obtains the original data, performs cleaning and conversion tasks, and saves the processed results to the silver and gold layers. Azure Synapse Analytics retrieves gold-level data and generates the Galaxy architecture framework required for reports. Power BI then links to Synapse Analytics and updates the dashboard data set. Finally, management users use the updated dashboard to assess inventory risks and acquire business intelligence.

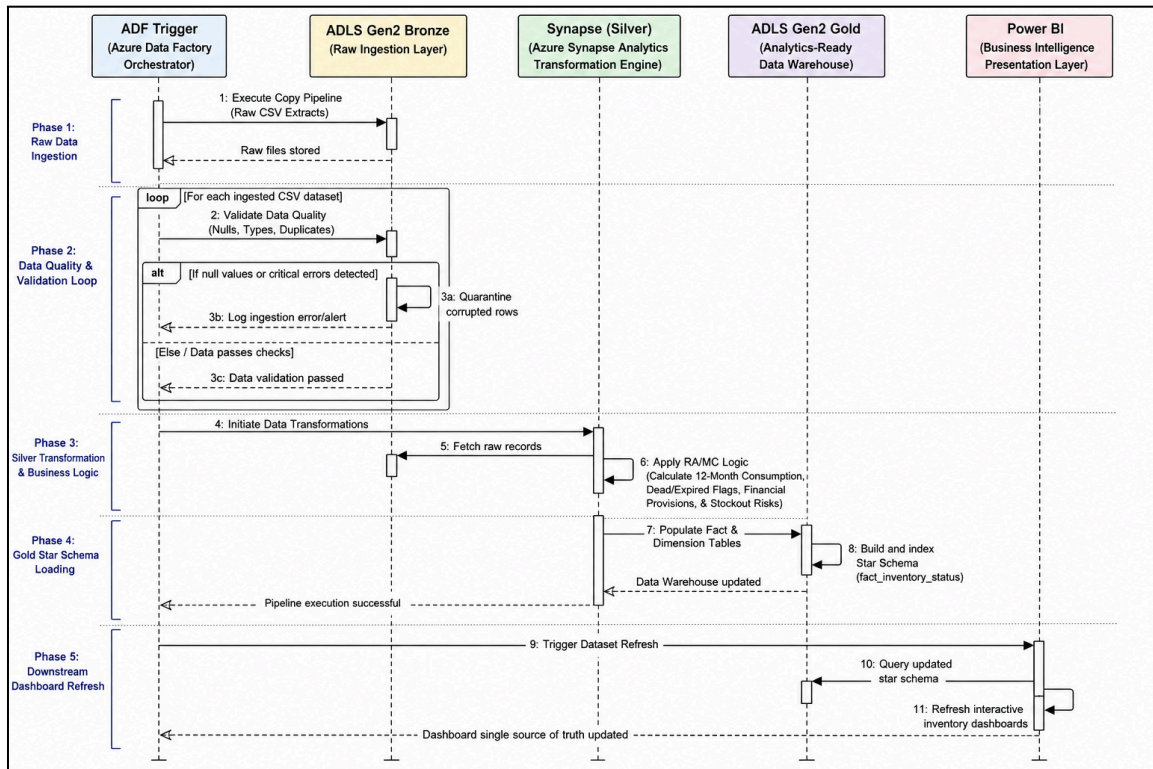


Figure 4.5.3.3: Sequence Diagram

4.5.3.4 Data Flow Diagram (DFD)

The level 0 Data Flow Diagram (DFD) describes how the system interacts with external entities, but does not involve specific details of internal processing. All operational information will be transmitted to the recommended inventory risk management system, which will analyze the information and turn it into valuable business insights. The system will generate reports and dashboards to provide detailed information on the impact of recyclable assets (RA), Magna Carta (MC) backlog inventory, expired materials, out-of-stock risks, financial reserves and sales orders. This Figure 4.5.3.4 aims to show the complete functions of the system and the exchange of information between external stakeholders and the centralized inventory control platform.

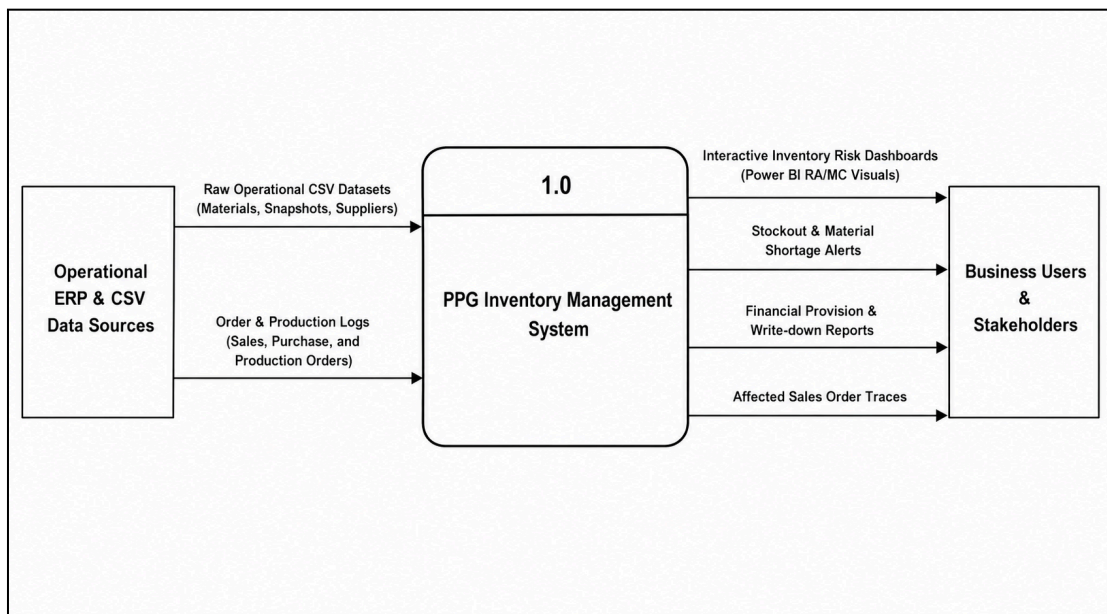


Figure 4.5.3.4: Level 0 DFD

The level 1 data flow diagram elaborates the process described in the level 0 data flow diagram and shows the key activities performed within the system. The process begins with data acquisition. At this stage, operational data sets such as materials, inventory snapshots, suppliers, purchase orders, production orders and sales orders are extracted and imported into the Azure data lake copper layer through the Azure data factory.

Then, data verification and cleaning procedures were conducted. At this stage, the system will verify duplicate entries, identify missing values and correct data type inconsistencies to improve data quality, and then follow-up processing. The cleaned data is then forwarded to the business logic processing module, in which the inventory management rules are implemented. This process evaluates the trend of material consumption in the past 12 months, identifies recyclable assets (RA), finds backlog inventory and expired materials in the Magna Carta (MC), calculates financial reserves, analyzes out-of-stock risks, and studies how inventory shortages affect customer sales orders.

After processing, the modified data will be saved in the golden layer and imported into the Galaxy Schema data warehouse through Azure Synapse Analytics. The warehouse classifies the data into fact tables and dimensional tables for effective analysis and query. Finally, the processed information will be provided to the Power BI dashboard module, where management and business users can view interactive

reports, KPIs, inventory risk indicators, financial risk analysis and sales impact insights.

Figure 4.5.3.5 below shows how to convert unprocessed operational data into information that can be used for analysis through systematic cloud data pipelines, so as to achieve automated inventory tracking and wise decision-making.

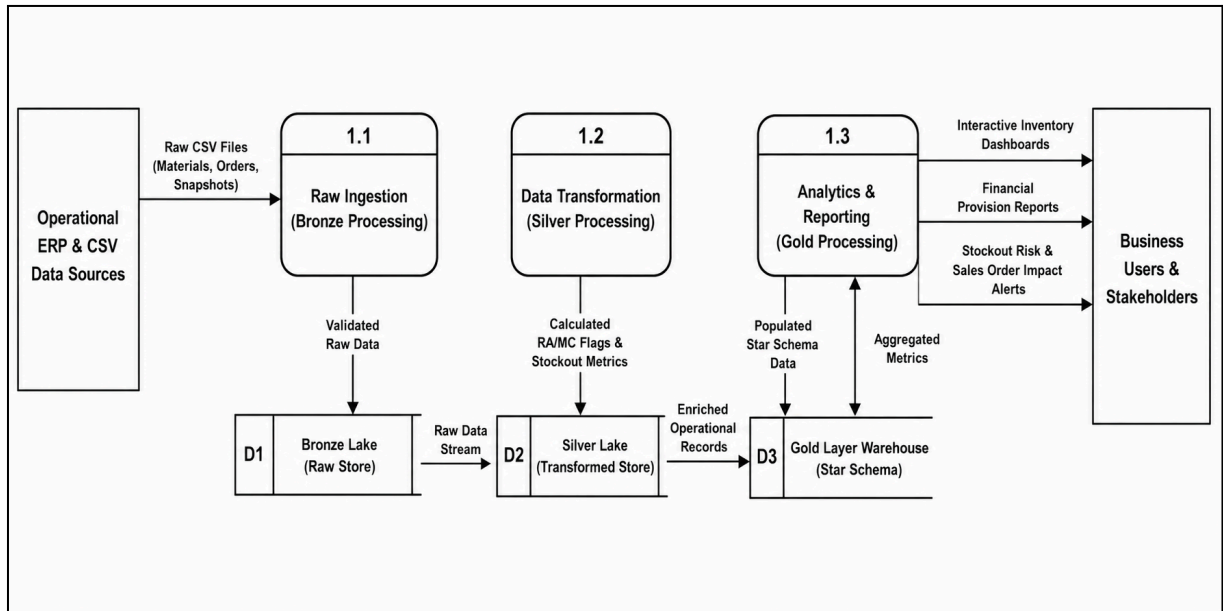


Figure 4.5.3.5: Level 1 DFD

4.5.4 Database Design

This section presents the database design for the end-to-end data pipeline built on Microsoft Azure. The relational structure of the source data and the analytical schema of the data warehouse will be covered in this section. The database design is designed according to the Medallion Architecture used, which has Bronze, Silver, and Gold layers. Different design approaches are required in each layer: raw file storage at Bronze, relational normalized tables at Silver, and a dimensional schema at Gold.

This section will be organized into 3 sections: NoSQL, representing the Bronze layer; Silver, which is represented by normalization and Entity Relationship Diagram (ERD); and a galaxy schema, which will be representing the Gold layer data warehouse design for analytics and reporting. Lastly, Azure Data Lake Storage Gen2 (ADLS Gen2) will be used in this project and the design is based on six synthetic

CSV source files given, simulating PPG's operational systems, covering materials, inventory snapshots, suppliers, purchase orders, production orders, and sales orders.

4.5.4.1 NoSQL Design

Inside ADLS Gen2, the Bronze layer will use a non-relational, file-based storage approach to store the raw data. This is to preserve raw source data as-is without imposing any schema during ingestion time. Since data quality and structure of the raw source data are not yet guaranteed, it is therefore inappropriate to enforce any schema at this stage before data inspection and data cleaning because it can result in data rejection or alteration. ADLS Gen2 was chosen because it supports large-scale file storage and hierarchical namespaces and has native integration with Azure Data Factory and Azure Databricks. Figure 4.5.4.1 is the hierarchical structure inside the Bronze layer to organize 6 raw CSV files.

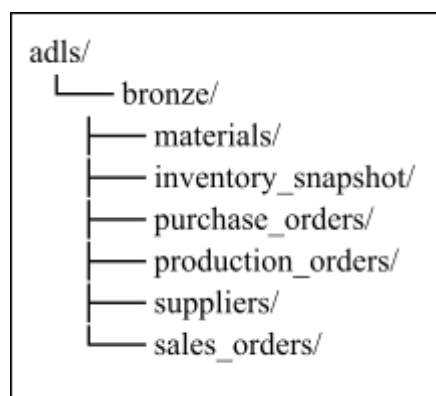


Figure 4.5.4.1: Bronze layer folder hierarchy structure

Each folder holds corresponding raw CSV files ingested from the source system. The advantage of this structure is that it makes data lineage tracing easier and allows individual files to be reprocessed when needed. In this layer, all files will be stored in CSV format and basic data quality checks such as null detection and type validation will be applied during ingestion while preserving raw files. After being ingested and checked, the data will be passed into the Silver layer for the transformation.

4.5.4.2 Normalization

In the silver layer, normalization was applied to all 6 raw source tables to ensure data integrity and eliminate redundancy.

First, the rules of 1NF are that each table needs to have atomic values, which means it cannot have repeating groups, and each row is unique. Upon analysis, all 6 source tables are found to already satisfy 1NF, as each column holds a single atomic value and a natural primary key exists in each table.

Next, the rule of 2NF is having no partial dependencies. Among all 6 tables, only the `inventory_snapshot` table has a composite primary key, which is `material_id` and `snapshot_date`. However, the remaining attributes, `warehouse_location` and `quantity_on_hand`, are both dependent on the full composite primary key, not just part of it. The other tables also only have a single primary key. Thus, the 2NF structure is satisfied.

Lastly, in the 3NF, the target is to remove any transitive dependencies. After analyzing column by column, it is confirmed that there are no non-key dependencies on each other. For example, in the purchase order table, there is a `supplier_id` column but the supplier details are not stored in the same table. Thus, there are no transitive dependencies, which make all 6 tables satisfy 3NF. Below are the final 3NF schema with primary and foreign keys notated.

materials (material_id, material_name, category, unit_of_measure, reorder_level, lead_time_days, unit_cost, lifespan_months)
 PK: material_id

suppliers (supplier_id, supplier_name, country, reliability_rating)
 PK: supplier_id

inventory_snapshot (snapshot_date, material_id, warehouse_location, quantity_on_hand)
 PK: snapshot_date, material_id

purchase_orders (po_id, po_date, *supplier_id*, *material_id*, quantity_ordered, quantity_received, expected_delivery_date, actual_delivery_date)
 PK: po_id
 FK: supplier_id, material_id

production_orders (production_order_id, production_date, finished_product, *material_id*, quantity_consumed)
 PK: production_order_id
 FK: material_id

sales_orders (sales_order_id, order_date, customer_name, finished_product, quantity_ordered, required_delivery_date, status)
 PK: sales_order_id
 *status is added during Silver layer transformation to filter open orders

Figure 4.5.4.2: 3NF Normalization on project dataset

4.5.4.3 Entity Relationship Diagram (ERD)

The ERD in Figure 4.5.4.3 represents the logical relationships between the 6 source tables based on shared key columns, which form the basis of the JOIN operations performed during transformation.

The materials table holds 3 different relationships. First, a one-to-many relationship with the inventory_snapshot table, where one material can have multiple inventory snapshots recorded over different dates. Second, a one-to-many relationship with the production table, meaning one material can be consumed across many production orders. Last is a one-to-many relationship with the purchase order table, where one material can be ordered multiple times from different suppliers. All are linked through the shared material_id column.

The suppliers table holds a one-to-many relationship, which means one

supplier can fulfill many purchase orders linked through supplier_id. The production orders are connected to the sales order table with an indirect or logical relationship via finished_product to find the list of raw materials needed for one production.

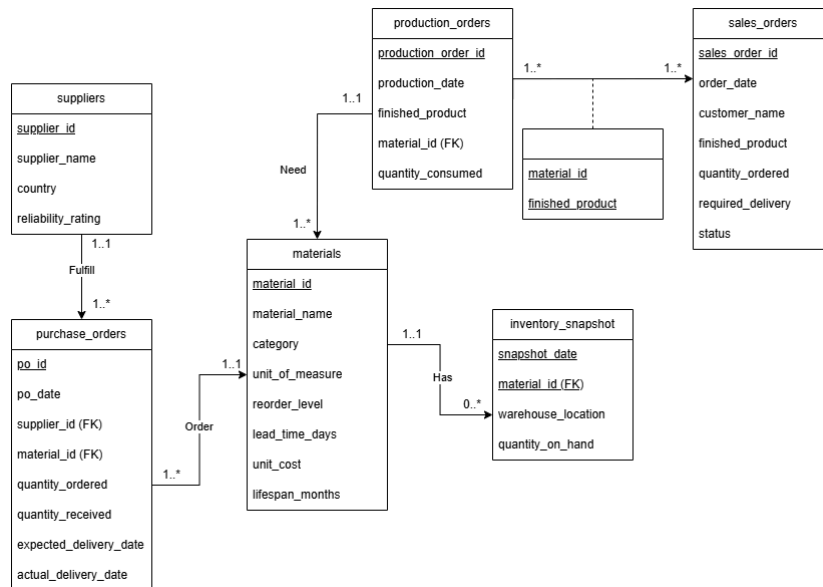


Figure 4.5.4.3: Entity Relationship Diagram (ERD)

4.5.4.4 Data Modelling: Galaxy Schema

The Gold layer adopts a Galaxy Schema because the project requires two independent analytical fact tables that share common dimension tables. Unlike a Star Schema which has only one fact table, a Galaxy Schema supports multiple fact tables sharing the same dimensions. In this project, both fact_inventory_status and fact_sales_impact link to dim_materials, which makes this design a Galaxy Schema. Figure 4.5.4.4 is the diagram of the galaxy schema of the project.

1. Dim_materials

This table stores descriptive attributes of all raw materials and acts as the central shared dimension that connects both fact tables, which is the defining characteristic of a Galaxy Schema shown in Figure 4.5.4.4.

2. Dim_suppliers

This table holds supplier reference data. It only links to fact_inventory_status as supplier information is only relevant to inventory and procurement, not sales impact.

3. Dim_production

This table holds production data. This table exist to provide the material production detail for Business question related to production impact.

4. Fact_inventory_status

This is the primary fact table where each row represents one material at one snapshot date. It holds raw measures such as quantity_on_hand and warehouse_location, as well as all derived RA/MC business logic flags and financial values. It references dim_materials via material_id and dim_suppliers via supplier_id.

5. Fact_sales_impact

This is the secondary fact table that records open customer sales orders at risk of being unfulfilled due to material shortages. It connects to dim_materials indirectly through bridge_bom rather than directly.

6. Bridge_bom

This table exists because the relationship between finished products and raw materials is many-to-many; one finished product requires many materials and one material can be used in many products. It acts as a junction between fact_sales_impact and dim_materials, with a composite primary key of finished_product and material_id.

7. Derived Columns

The derived columns in fact_inventory_status are computed during the Silver-to-Gold transformation and do not exist in any raw source file. Table 4.5.4.1 describes each derived column and its business meaning.

Table 4.5.4.1: Derived column description

Derived Column	Business Meaning
lot_expiry_date	When the material batch expires, calculated as actual_delivery_date + lifespan_months
last_consumption_date	Last date the material was used in production, used to detect dead stock
total_12_month_consumption	Total quantity consumed across all production orders in the past 12 months
is_recoverable_asset	Flags materials where stock exceeds 12-month consumption

is_mc_expired	Flags materials whose lot expiry date has passed
is_mc_dead_stock	Flags materials with zero consumption and no lot expiry
mc_provision_amount	Financial write-down amount, calculated as $\text{quantity_on_hand} \times \text{unit_cost}$

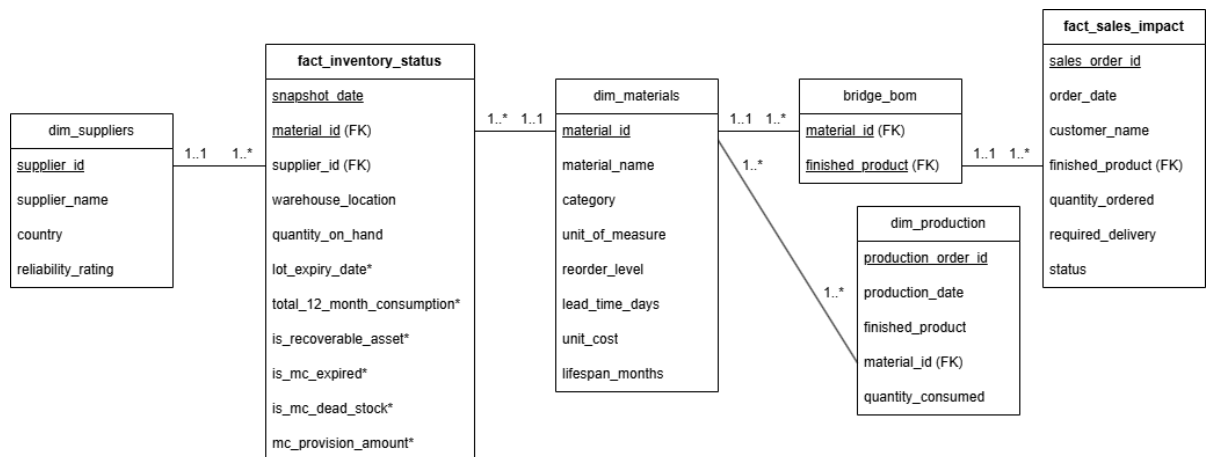


Figure 4.5.4.4: Galaxy Schema

4.5.5 Interface Design

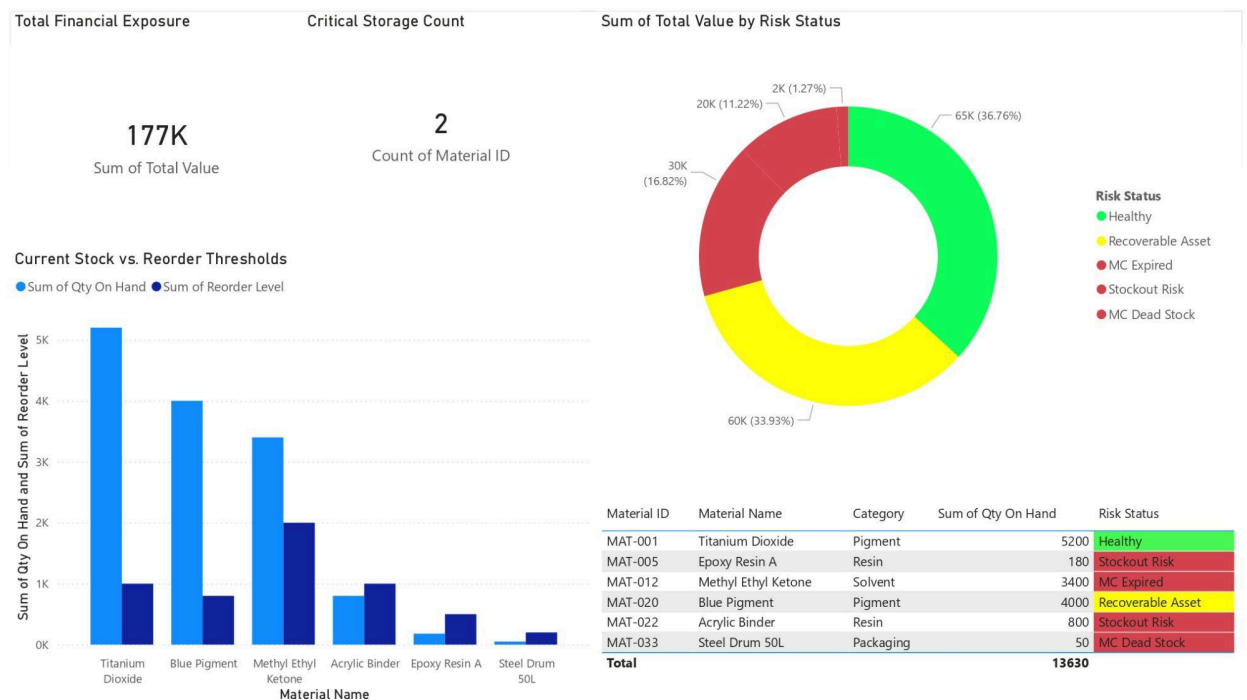


Figure 4.5.5.1: Dashboard Interface Design

4.5.6 Summary

The structural evolution of the PPG Gold Data Warehouse layer represents a critical transition from transactional storage to an optimized analytical Star Schema (Bus Matrix Architecture). Initially conceived as a simplified multi-table structure, continuous profiling and data engineering iterations revealed complex relational dependencies, specifically surrounding material consumption traces and outstanding customer allocations.

To achieve optimal query execution speeds, eliminate structural data redundancy, and prevent severe "fan-out" Cartesian row multiplication, the enterprise analytical layer was finalized into a robust 5-Table Star Schema Engine comprising two Specialized Fact Tables (`fact_inventory_status` and `fact_sales_impact`) to decouple independent business processes (Warehouse Operations vs. Customer Revenue Risks), two Conformed Dimension Tables (`dim_materials` and `dim_suppliers`) to establish unified lookups across both analytical domains, and one High-Performance Bridge Table (`bridge_bom`) to map complex, many-to-many relationship links between finished product demands and raw component material constraints.

By provisioning these logical interfaces as Azure Synapse Analytics External Tables built directly over optimized Delta Lake structures, the enterprise architecture successfully achieves a completely decoupled computing and storage engine model. This guarantees that downstream BI components, including specialized Power BI decision matrices, can cross-filter and query real-time supply chain bottlenecks, financial provision allocations, and fulfillment delays smoothly without hitting processing latency limits or locking active source records.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter discusses the implementation and testing of the Recoverable Assets (RA) and Inventory Risk Management (IRM) system. The system was developed to support inventory monitoring and risk analysis by integrating operational data into a cloud-based data engineering pipeline.

The implementation process involved preparing synthetic datasets, configuring the database environment, and developing an automated data ingestion pipeline using Microsoft Azure services. The system follows a Bronze, Silver, and Gold layered architecture to support scalable data processing and analytics. Testing activities were also conducted to ensure that the data pipeline operated correctly and fulfilled the project requirements.

5.2 System Development

The system development includes source data preparation, SQL database setup, Azure resource configuration, Azure Data Factory Pipeline Development, and overall pipeline flow.

5.2.1 Source Data Preparation

The system utilized six synthetic datasets representing operational business data related to inventory and supply chain management. The datasets included materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders.

To simulate an enterprise-scale environment, the original datasets were expanded to approximately ten times their initial size using Faker and random data generation techniques. The generated datasets were stored in CSV format and used as the primary source for the data ingestion process.

Each dataset expansion was performed through different Python scripts:

1. `expand_inventory_snapshot.py`

In this script, necessary libraries such as pandas, random, and csv were imported. The original *inventory_snapshot.csv* dataset was then loaded into a DataFrame to be expanded with synthetic inventory data.

A list of warehouse locations (WH-A to WH-E), monthly snapshot dates for the year 2025, and product categories were defined to structure the simulated inventory environment.

A function *generate_stock_series()* was created to simulate realistic stock fluctuations over time. It adjusts inventory levels month by month using random increases and decreases, while also introducing occasional restock spikes to mimic real warehouse replenishment behavior.

The script then generates new inventory records for a range of material IDs (*MAT021* to *MAT200*). For each material, a warehouse location is randomly assigned, and a base stock quantity is generated. A 12-month stock series is produced for each material using the defined function.

For each month, a record is created containing the snapshot date, material ID, warehouse location, and quantity on hand. These records are appended into a list and later converted into a DataFrame.

Finally, the new records are concatenated with the original dataset and exported as *expanded_inventory_snapshot1.csv*.

2. `expand_materials.py`

In this script, necessary libraries such as pandas, random, and csv were imported. The original *materials.csv* dataset was then loaded into a DataFrame, and existing material names were extracted and stored (in lowercase) to ensure uniqueness when generating new records.

A category configuration was defined for different material types, specifying rules for generating material names, units of measure, cost ranges, reorder levels, lead times, and a lifespan range in months.

A function was used to assign *lifespan_months* to the original dataset based on each material's category, using category-specific ranges. Another function was used to randomly assign *supplier_id* to the original dataset within the range from *SUP001* to *SUP100*. The two values were added as a new column to the existing records.

A loop was then used to generate new synthetic material records. For each record, a category was randomly selected and a material name was created using predefined prefixes and suffixes. Meanwhile, *supplier_id* was randomly assigned to each new generated record. Uniqueness was enforced by checking against existing names.

Each generated record includes a unique material ID, category details, unit of measure, reorder level, lead time, unit cost, lifespan in months, and *supplier_id*. These records were stored in a list and converted into a DataFrame.

Finally, the new records were combined with the original dataset and exported as *expanded_materials.csv*.

3. **expand_production_orders.py**

In this script, necessary libraries such as pandas, random, csv, and Faker were imported, along with date utilities from the datetime module. The original *production_orders.csv* dataset was then loaded into a DataFrame to be expanded with synthetic production data.

A list of finished products was defined, along with a generated list of materials (*MAT001* to *MAT200*). A mapping was created where each finished product is assigned a random set of four materials to simulate its bill of materials (BOM).

Existing production order IDs were extracted, cleaned, and converted into integers to determine the starting ID for new records.

The script then calculates the number of new production orders required to generate a total of 3024 new material-level rows, assuming each order produces four material consumption entries.

For each production order, a unique ID and a random production date within 2025 are generated using Faker. A finished product is randomly selected, and its associated materials are retrieved from the predefined mapping. A production scale multiplier is applied to simulate different batch sizes.

For each material used in the production order, a consumption quantity is generated with slight randomness, including a small probability of zero consumption to simulate production anomalies.

Each record includes the production order ID, production date, finished product, material ID, and quantity consumed. These records are stored in a list, converted into a DataFrame, and then concatenated with the original dataset.

Finally, the expanded dataset is exported as *expanded_production_orders.csv*.

4. expand_purchase_orders.py

In this script, necessary libraries such as pandas, random, and csv, along with datetime utilities from the datetime module, were imported. The original *purchase_orders.csv* dataset was then loaded into a DataFrame to be expanded with synthetic purchase order data.

Lists of supplier IDs (*SUP001* to *SUP100*) and material IDs (*MAT001* to *MAT200*) were generated to simulate a structured procurement environment. A helper function *random_date()* was defined to generate random purchase order dates within the year 2025.

A sequential ID system was used to ensure unique purchase order IDs starting from the next available number after the existing dataset.

The script generates two purchase orders per material, resulting in a total of 400 new purchase orders. For each order, a supplier is randomly assigned, along with a randomly generated order quantity and a lead time between 7 and 21 days. This lead time is used to calculate the expected delivery date.

A delivery simulation logic is applied where 85% of orders are marked as

delivered. For delivered orders, a small random delay may be added to the expected delivery date, and there is a chance of partial delivery where only a portion of the ordered quantity is received. Undelivered orders are assigned zero received quantity and an empty actual delivery date.

Each record includes purchase order details such as purchase order (PO) ID, PO date, supplier ID, material ID, quantities ordered and received, expected delivery date, and actual delivery date. These records are stored in a list, converted into a DataFrame, and combined with the original dataset.

Finally, the expanded dataset is exported as *expanded_purchase_orders.csv*.

5. expand_sales_orders

In this script, necessary libraries such as pandas, random, and csv were imported, along with date utilities and the Faker library for generating realistic dates. The original *sales_orders.csv* dataset was then loaded into a DataFrame to be expanded with synthetic sales order data.

A list of existing customers and finished products was defined to simulate realistic business transactions. Order statuses were also defined, though final status assignment is handled dynamically in the logic.

The script generates a specified number of new sales orders (432 records), starting from the next available sales order ID based on the existing dataset.

For each sales order, a realistic order date within the year 2025 is generated using Faker. A customer and finished product are randomly selected, and a random quantity is assigned to each order. A required delivery date is then calculated by adding a random delivery lead time (14 to 35 days) to the order date.

Order status is determined based on whether the required delivery date is in the past or future. Orders with past delivery dates have a high probability (90%) of being marked as "Delivered", while others are marked as "Open".

Each record includes the sales order ID, order date, customer name, finished product, quantity ordered, required delivery date, and status. These records are

stored in a list, converted into a DataFrame, and merged with the original dataset.

Finally, the expanded dataset is saved as *expanded_sales_orders.csv*.

6. **expand_suppliers**

In this script, necessary libraries such as pandas, random, and csv were imported. The original *suppliers.csv* dataset was then loaded into a DataFrame, and existing supplier names were extracted (in lowercase) to ensure uniqueness when generating new supplier records.

A list of countries, reliability levels, supplier name prefixes (grouped by country), and suffixes was defined to create realistic and region-specific supplier names.

The script generates a target of 90 new suppliers using a loop that continues until the required number of unique records is created. For each supplier, a country is randomly selected, and a supplier name is generated by combining a country-specific prefix with a generic business suffix. Uniqueness is enforced by checking against existing supplier names.

Supplier reliability ratings are assigned using weighted randomness to reflect realistic business conditions. Developed countries (such as Germany, Singapore, Japan, and the USA) are more likely to have higher reliability ratings, while other countries have more balanced distributions.

Each record includes a unique supplier ID, supplier name, country, and reliability rating. These records are stored in a list, converted into a DataFrame, and merged with the original dataset.

Finally, the expanded dataset is saved as *expanded_suppliers.csv*.

5.2.2 SQL Database Setup

The generated CSV datasets were imported into a Microsoft SQL Server database named *PPG* to simulate an on-premises operational environment. Microsoft SQL Server Management Studio (SSMS) was used to create the database schema and manage the tables.

Figure 5.2.2.1 shows the tables created in the *PPG* database based on the dataset structure, namely *inventory_snapshot*, *materials*, *production_orders*, *purchase_orders*, *sales_order*, and *suppliers*.

Primary keys and foreign key relationships were implemented to maintain data integrity and establish relationships between entities. Figure 5.2.2.2 shows the SQL script that adds primary key and foreign key constraints to the tables in the *PPG* database.

Figure 5.2.2.3 illustrates the database schema of the *PPG* database after implementing relationships between tables. This SQL database served as the source system for Azure Data Factory during the ingestion phase.

Figure 5.2.2.4 shows the SQL script that creates a user named *g3* and gives the user permission to read data from the tables in the *PPG* database. This SQL script prepares a user so that Azure Data Factory can securely and easily access the *PPG* database and read data.

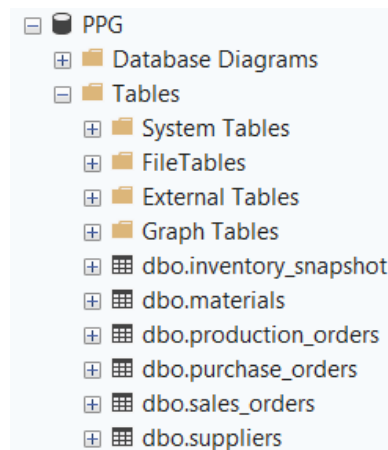


Figure 5.2.2.1 Tables Created in PPG Database

```

USE PPG;

--suppliers
ALTER TABLE dbo.suppliers
ADD CONSTRAINT PK_supplier_id PRIMARY KEY (supplier_id);

--materials
ALTER TABLE dbo.materials
ADD CONSTRAINT PK_material_id PRIMARY KEY (material_id);

ALTER TABLE dbo.materials
ADD CONSTRAINT FK_supplier_id_2
FOREIGN KEY (supplier_id) REFERENCES dbo.suppliers(supplier_id);

--inventory_snapshot
ALTER TABLE dbo.inventory_snapshot
ADD CONSTRAINT FK_material_id
FOREIGN KEY (material_id) REFERENCES dbo.materials(material_id);

--purchase_orders
ALTER TABLE dbo.purchase_orders
ADD CONSTRAINT PK_po_id PRIMARY KEY (po_id);

ALTER TABLE dbo.purchase_orders
ADD CONSTRAINT FK_material_id_2
FOREIGN KEY (material_id) REFERENCES dbo.materials(material_id);

ALTER TABLE dbo.purchase_orders
ADD CONSTRAINT FK_supplier_id
FOREIGN KEY (supplier_id) REFERENCES dbo.suppliers(supplier_id);

--sales_orders
ALTER TABLE dbo.sales_orders
ADD CONSTRAINT PK_sales_order_id PRIMARY KEY (sales_order_id);

--production_orders
ALTER TABLE dbo.production_orders
ADD CONSTRAINT FK_material_id_1
FOREIGN KEY (material_id) REFERENCES dbo.materials(material_id);

```

Figure 5.2.2.2 *PPG_PK_FK_constraints.sql* SQL Script

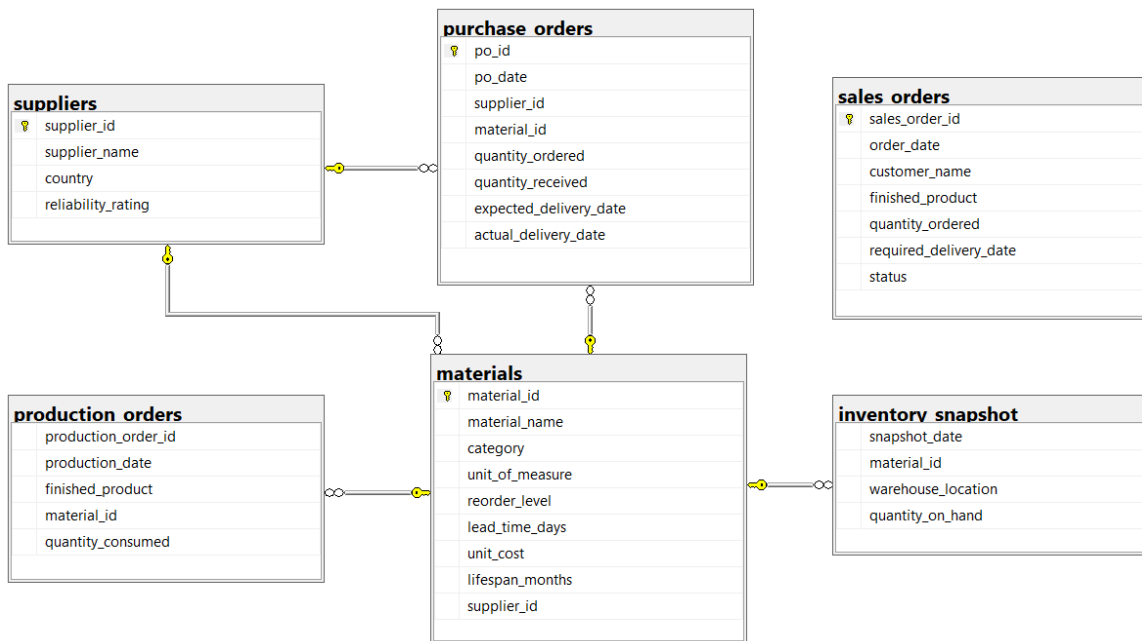


Figure 5.2.2.3 Database Schema of PPG Database

```

CREATE LOGIN g3 WITH PASSWORD = '123456789'

create user g3 for login g3

USE PPG;

GRANT SELECT ON dbo.inventory_snapshot TO g3
GRANT SELECT ON dbo.materials TO g3
GRANT SELECT ON dbo.production_orders TO g3
GRANT SELECT ON dbo.purchase_orders TO g3
GRANT SELECT ON dbo.sales_orders TO g3
GRANT SELECT ON dbo.suppliers TO g3
  
```

Figure 5.2.2.4 *createlogin.sql* SQL Script

5.2.3 Azure Resource Configuration

Several Microsoft Azure services were configured to support the implementation of the data pipeline. A dedicated Resource Group named *PPG* was created to organize and manage all cloud resources used in the project.

Figure 5.2.3.1 shows the main Azure services configured include:

- **Azure Data Factory**

PPG-databricks was created to move data into storage and automate workflows.

- **Azure Data Lake Storage**

ppgsg1 was created to store data.

- **Azure Databricks**

PPG-databricks was created to clean and process data.

- **Azure Synapse Analytics**

ppg-synapse1 was created to analyze and query data.

- **Azure Key Vault**

PPG-keyvault1 was created to control access and keep everything secure.

Apart from this, a Self-hosted Integration Runtime named *SHIR* was configured to establish secure connectivity between the on-premises SQL Server database and Azure Data Factory. This allowed the cloud pipeline to access and transfer data from the local database environment.

☐	Name ↑	Type	Location
☐	 <i>PPG-databricks</i>	⋮ Azure Databricks Service	Southeast Asia
☐	 <i>PPG-df1</i>	⋮ Data factory (V2)	Malaysia West
☐	 <i>PPG-keyvault1</i>	⋮ Key vault	Malaysia West
☐	 <i>ppg-synapse1</i>	⋮ Synapse workspace	Southeast Asia
☐	 <i>ppgsg1</i>	⋮ Storage account	Malaysia West

Figure 5.2.3.1 Resources Created in *PPG* Resource Group

5.2.4 Azure Data Factory Pipeline Development

Azure Data Factory was used to develop the data ingestion pipeline for the Bronze Layer. A Linked Service named *SqlServerOnPremLinkedService* was configured to establish connections between the SQL Server source database and Azure Data Lake Storage.

Copy Data activities were implemented to automate the ingestion process from the on-premises SQL database into the Bronze Layer of the data lake. The *copy_all_tables* pipeline was designed to transfer raw data without modification in order to preserve the original source records.

Figure 5.2.4.1 illustrates the pipeline of data ingestion process. The ingestion

workflow included connecting to the source SQL database and retrieving source table records using the Lookup activity. The data was then copied into the Bronze storage layer through the use of ForEach and Copy Data activities. Finally, the execution process was monitored, and successful data transfers were validated using Debug and Trigger Now functionalities.

Figure 5.2.4.2 indicates the *copy_all_tables* pipeline executed successfully and data was ingested into the Bronze data storage.

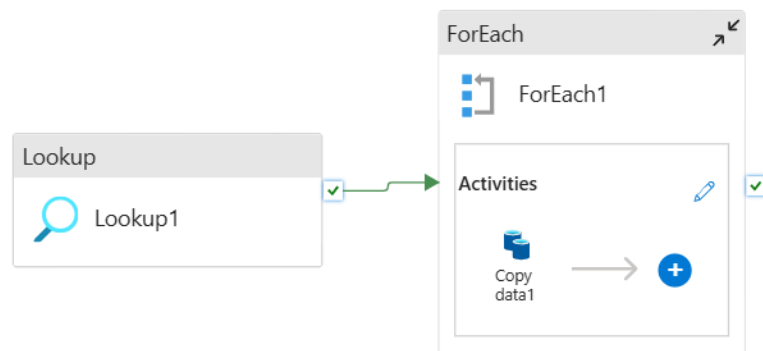


Figure 5.2.4.1 Pipeline Diagram of *copy_all_tables*

Activity name ↑↓	Activity st... ↑↓	Activit... ↑↓	Run start ↑↓	Duration ↑↓	Integration runtime ↑↓
Lookup1	✓ Succeeded	Lookup	6/1/2026, 10:59:26 AM	10s	SHIR
ForEach1	✓ Succeeded	ForEach	6/1/2026, 10:59:36 AM	32s	
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	26s	SHIR
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	23s	SHIR
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	28s	SHIR
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	23s	SHIR
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	28s	SHIR
Copy data1	✓ Succeeded	Copy data	6/1/2026, 10:59:37 AM	30s	SHIR

Figure 5.2.4.2 *copy_all_tables* Pipeline Execution Result

5.2.5 Overall Pipeline Flow

After the ingestion process was completed, the datasets stored in the Bronze Layer were prepared for further processing in the Silver Layer. Additional cleansing, validation, and business logic transformations were later performed using Azure Databricks.

The transformed datasets were subsequently structured into an analytics-ready format within the Gold Layer for reporting and dashboard visualization purposes. This layered architecture supports scalability, maintainability, and efficient processing throughout the entire data engineering workflow.

5.3 Coding of system's main functions

The data transformation from Bronze to Silver layer was implemented using Azure Databricks to perform data validation, cleaning and standardization on the raw datasets provided before storing as Silver layer in Azure Data Lake Storage Gen2 (ADLS). Subsequently, Silver to Gold layer data transformation was performed by applying business logic on pre-processed data to generate analytics data for analysis and reporting purposes. Finally, Gold layer datasets were transformed into a star schema structure consisting of fact and dimension tables to support efficient querying and dashboard visualization.

5.3.1 Data Loading from Bronze Layer

The first step is extracting the raw data from the bronze layer into Databricks using PySpark for preprocessing operations. The code used is shown in Figure 5.3.1.

```
materials_df = spark.read.parquet(bronze_path + "dbo/materials/")
inventory_df = spark.read.parquet(bronze_path + "dbo/inventory_snapshot/")
production_df = spark.read.parquet(bronze_path + "dbo/production_orders/")
purchase_df = spark.read.parquet(bronze_path + "dbo/purchase_orders/")
sales_df = spark.read.parquet(bronze_path + "dbo/sales_orders")
suppliers_df = spark.read.parquet(bronze_path + "dbo/suppliers/")
```

Figure 5.3.1: Load raw operational data from Bronze

5.3.2 Data Profiling and Validation

This step was to inspect schema structure, detect null values and check for duplicates to ensure data integrity. Figure 5.3.2.1 shows the script used to inspect each table schema.

```

tables = {
    "Materials": materials_df,
    "Inventory Snapshot": inventory_df,
    "Production Orders": production_df,
    "Purchase Orders": purchase_df,
    "Sales Orders": sales_df,
    "Suppliers": suppliers_df
}

for name, df in tables.items():
    df.printSchema()
    df.count()
    display(df)

```

Figure 5.3.2.1: Inspect All Table

This data validation process presents the data preview to ensure that all datasets were successfully loaded and structurally consistent before further processing.

The null and missing value for every table in datasets were then detected by PySpark using the code shown in Figure 5.3.2.2.

```

materials_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in materials_df.columns]).show()

inventory_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in inventory_df.columns]).show()

production_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in production_df.columns]).show()

purchase_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in purchase_df.columns]).show()

sales_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in sales_df.columns]).show()

suppliers_df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in suppliers_df.columns]).show()

```

Figure 5.3.2.2: Check Null Value for Each Table

The results showed minimal missing values across datasets. The null values only existed in the `actual_delivery_date` field of purchase orders, which was verified as a valid business condition that shows pending deliveries.

Additionally, duplicate validation was performed using the code in Figure 5.3.2.3 to ensure all data in the datasets is unique.

```
materials_df.count(), materials_df.dropDuplicates().count()
inventory_df.count(), inventory_df.dropDuplicates().count()
production_df.count(), production_df.dropDuplicates().count()
purchase_df.count(), purchase_df.dropDuplicates().count()
sales_df.count(), sales_df.dropDuplicates().count()
suppliers_df.count(), suppliers_df.dropDuplicates().count()
```

Figure 5.3.2.3: Check Duplicate for Each Record in Table

The results showed that row counts before and after removing duplicates were the same, which represented that no duplicates were present within the datasets.

5.3.3 Handling Business-Valid Null Values

A null value was identified in the `actual_delivery_date` field of the purchase orders dataset. To improve interpretability, a derived attribute was created as shown in Figure 5.3.3.

```
purchase_df = purchase_df.withColumn(
    "delivery_status",
    when(col("quantity_received") == 0, "Pending")
    .otherwise("Delivered")
)
```

Figure 5.3.3: Business-Valid Null Values Handling

5.3.4 Writing Cleaned Data to Silver Layer

After validation and transformation, the cleaned datasets were written to the Silver layer in Delta format. Figure 5.3.4 shows the code used to write all the datasets.

```

materials_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "materials")

inventory_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "inventory_snapshot")

production_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "production_orders")

purchase_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "purchase_orders")

sales_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "sales_orders")

suppliers_df.write \
    .mode("overwrite") \ .option("overwriteSchema", "true") \
    .format("delta") \ .save(silver_path + "suppliers")

```

Figure 5.3.4.1: Data Transformation to Silver

Then, a code was used to ensure all files successfully load into Silver layer:

```
display(dbutils.fs.ls(silver_path))
```

Figure 5.3.4.2: Inspect Silver Layer

5.3.5 Silver to Gold Layer Data Transformation

After the successful validation and storage of cleaned datasets in the Silver layer, the next stage involved executing advanced analytics engineering to transition the data from the Silver to the Gold tier. This phase applies complex business logic rules onto the pre-processed DataFrames using Apache Spark to generate operational supply chain risk models for business analysis and reporting in Power BI.

Figure 5.3.5 shows the first operation involves loading all five DataFrames natively from the Silver layer Delta container and standardizing all date columns from raw string types into proper PySpark DateType to enable accurate date arithmetic across all downstream calculations.

```

df_materials = spark.read.format("delta").load(silver_path +
"materials")
df_inventory = spark.read.format("delta").load(silver_path +
"inventory_snapshot")
df_production = spark.read.format("delta").load(silver_path +
"production_orders")
df_purchases = spark.read.format("delta").load(silver_path +
"purchase_orders")
df_sales = spark.read.format("delta").load(silver_path +
"sales_orders")

df_inventory = df_inventory.withColumn("snapshot_date",
F.to_date(F.col("snapshot_date")))
df_production = df_production.withColumn("production_date",
F.to_date(F.col("production_date")))
df_purchases = df_purchases.withColumn("actual_delivery_date",
F.to_date(F.col("actual_delivery_date")))
df_purchases = df_purchases.withColumn("po_date",
F.to_date(F.col("po_date")))
df_sales = df_sales.withColumn("order_date",
F.to_date(F.col("order_date")))
df_sales = df_sales.withColumn("required_delivery_date",
F.to_date(F.col("required_delivery_date")))

```

Figure 5.3.5: Load raw operational data from Silver

5.3.6 Temporally-Correct Supplier Mapping Engine

Purchase orders were first filtered as shown in Figure 5.3.6 to confirm delivered records only using the `delivery_status` column established by the Silver layer pipeline, ensuring only physically received stock is considered. A point-in-time boundary was then applied to exclude future deliveries not yet arrived by each snapshot date. A left join was deliberately chosen over an inner join to preserve all inventory snapshot rows; an inner join was found to silently drop 38.4% of rows (922 out of 2,400 records) as most deliveries only commenced from February 2025 while snapshots began from January 2025. A null guard filter retains early snapshot rows with no confirmed delivery history yet:

```

df_inv_pur_join = df_inventory.join(
    df_purchases.filter(F.col("delivery_status") == "Delivered"),
    on="material_id", how="left"
)
df_valid_history = df_inv_pur_join.filter(
    F.col("actual_delivery_date").isNull() |
    (F.col("actual_delivery_date") <= F.col("snapshot_date"))
)

```

Figure 5.3.6.1: Filtering and joining between Purchase and Inventory table

A Window function partitioned by snapshot_date, material_id, and warehouse_location ordered by actual_delivery_date descending was then applied to isolate the single most recent confirmed supplier delivery per material per warehouse per snapshot date shown in Figure 5.3.6.2.

```

window_spec = Window.partitionBy("snapshot_date", "material_id",
    "warehouse_location").orderBy(F.col("actual_delivery_date").desc())

df_final_supplier_map = df_valid_history \
    .withColumn("row_num", F.row_number().over(window_spec)) \
    .filter(F.col("row_num") == 1)

```

Figure 5.3.6.2: Window function

5.3.7 Lot Expiry Date & Consumption Metrics Calculation

Material metadata lifespan_months and unit_cost were joined from the materials dimension to enrich inventory records. Figure 5.3.7.1 shows the lot_expiry_date was derived by adding lifespan_months to actual_delivery_date using add_months(); rows with no confirmed delivery date produce a null lot_expiry_date, acknowledged as a known data characteristic for early snapshot months.

```

df_inv_enriched = df_final_supplier_map \
    .join(df_materials.select("material_id", "lifespan_months",
    "unit_cost"), on="material_id", how="inner")

df_inv_with_expiry = df_inv_enriched.withColumn(
    "lot_expiry_date", F.expr("add_months(actual_delivery_date,
    lifespan_months)")
)

```

Figure 5.3.7.1: Lot Expiry Calculation

Figure 5.3.7.2 shows the total_12_month_consumption and last_consumption_date were derived by aggregating production orders grouped by material_id. Both were joined using left joins to preserve inventory rows for materials with no production history, filling null consumption with zero.

```
df_total_consumption = df_production.groupBy("material_id") \
    .agg(F.sum("quantity_consumed").alias("total_12_month_consumption"))

df_last_consumption = df_production.groupBy("material_id") \
    .agg(F.max("production_date").alias("last_consumption_date"))

df_ra_logic = df_inv_with_expiry \
    .join(df_total_consumption, on="material_id", how="left") \
    .join(df_last_consumption, on="material_id", how="left") \
    .na.fill({"total_12_month_consumption": 0})
```

Figure 5.3.7.2: Consumption Metrics Calculation

5.3.8 Magna Carta Business Flag Implementation

Figure 5.3.8 shows the four business-critical flags that were computed to classify each material's risk and financial exposure. The is_recoverable_asset identifies materials where stock on hand exceeds 12-month consumption. is_mc_expired identifies batches where snapshot date has surpassed lot expiry date; null lot_expiry_date rows correctly evaluate to 0, as null comparisons return false in Spark. is_mc_dead_stock identifies materials with zero consumption still within shelf life. mc_provision_amount computes the financial write-down as quantity_on_hand×unit_cost for expired or dead stock. Both is_mc_expired and is_mc_dead_stock are mutually exclusive by design, as their conditions are logically opposite. The purchase_order_date and actual_delivery_date are retained in the final output to support supplier late delivery performance analysis in Power BI.

```
df_ra_final = df_ra_logic.withColumn( "is_recoverable_asset",
    F.when(F.col("quantity_on_hand") > F.col("total_12_month_consumption"), 1).otherwise(0)
)

df_mc_logic = df_ra_final.withColumn( "is_mc_expired",
    F.when(F.col("snapshot_date") > F.col("lot_expiry_date"), 1).otherwise(0)).withColumn(
    "is_mc_dead_stock",
    F.when((F.col("total_12_month_consumption") == 0) &
    (F.col("snapshot_date") <= F.col("lot_expiry_date")), 1).otherwise(0))

df_fact_inventory = df_mc_logic.withColumn( "mc_provision_amount",
    F.when((F.col("is_mc_expired") == 1) | (F.col("is_mc_dead_stock") == 1),
    F.round(F.col("quantity_on_hand") * F.col("unit_cost"), 2)).otherwise(0.00)).select(
    "snapshot_date", "material_id", "supplier_id", "warehouse_location", "quantity_on_hand",
    F.col("po_date").alias("purchase_order_date"), "actual_delivery_date",
    "lot_expiry_date", "last_consumption_date",
    "total_12_month_consumption", "is_recoverable_asset",
    "is_mc_expired", "is_mc_dead_stock", "mc_provision_amount"
)
```

Figure 5.3.8: Magna Carta Business Flag Computation Code

5.3.9 Customer Sales Revenue At Risk Mapping

Figure 5.3.9.1 shows that sales orders were filtered strictly to Open status only, as delivered or cancelled orders carry no active fulfillment risk; source data confirmed only two status values exist: Open (81 records) and Delivered (399 records). The finished_product was renamed to product_name for Power BI clarity, and order_date, required_delivery_date, and status were included to support delivery urgency analysis. material_id was deliberately excluded, as the material linkage is handled through bridge_bom.

```
df_fact_sales_impact = df_sales \
    .filter(F.col("status") == "Open") \
    .select("sales_order_id", "customer_name",
F.col("finished_product").alias("product_name"),
"quantity_ordered", "order_date", "required_delivery_date", "status") \
    .orderBy(F.col("sales_order_id").asc())
```

Figure 5.3.9.1: Sales Order Filtering

A Bill of Materials bridge table was constructed from production orders using .distinct() to produce a clean product-to-material recipe map, enabling Power BI to traverse from fact_sales_impact through bridge_bom to dim_materials as shown in Figure 5.3.9.2. This bridge represents a recipe relationship derived from historical production records rather than a direct transactional link, as no foreign key exists between sales orders and production runs in the source data.

```
df_bridge_bom = df_production \
    .select("finished_product", "material_id") \
    .distinct() \
    .orderBy("finished_product", "material_id")
```

Figure 5.3.9.2: Bridge table between material_id and finished_product

5.3.10 Committing Analytical Datasets to Gold Layer Storage

All finalized DataFrames were committed to the Gold layer in Delta format following a Galaxy Schema design consisting of two fact tables, one bridge table, and three dimension tables as shown in Figure 5.3.10.1. The dim_suppliers and dim_production was passed directly from Silver to Gold unchanged, as no additional transformations were required

```

df_fact_inventory.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_path + "fact_inventory_status")
df_fact_sales_impact.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_path + "fact_sales_impact")

df_materials.drop("supplier_id").write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_path + "dim_materials")
df_production.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_path + "dim_production")
spark.read.format("delta").load(silver_path + "suppliers") \
    .write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true").save(gold_path + "dim_suppliers")

df_bridge_bom.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(gold_path + "bridge_bom")

```

Figure 5.3.10.1: Writing fact and dimension tables into Gold

.mode("overwrite") guarantees clean data refreshes on every pipeline rerun without duplicated entries, while .option("overwriteSchema", "true") enables smooth schema evolution if business calculation properties are modified in future pipeline iterations. All six Gold tables were verified by reading back from the Gold container and displaying each table output upon successful pipeline completion using code in Figure 5.3.10.2.

```

tables = [
    "fact_inventory_status",
    "dim_materials",
    "dim_suppliers",
    "dim_production",
    "fact_sales_impact",
    "bridge_bom"
]

for t in tables:
    print(f"\n===== {t} =====")
    df = spark.read.format("delta").load(gold_path + t)
    display(df)

```

Figure 5.3.10.2: Tables display code

5.3.11 Coding of Synapse Logical Data Warehouse External Schema Layer

To expose the finalized analytical assets for end-user business intelligence reporting without shifting physical rows out of cloud storage, a Logical Data Warehouse structure was instantiated inside Azure Synapse Analytics Serverless SQL Pools using Data Definition Language (DDL) queries.

First, a cleanup command script was applied to drop older structural registrations

from the serverless metadata database index catalog to prevent relational metadata caching conflicts when re-allocating directory parameters:

```
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'fact_inventory_status') DROP EXTERNAL TABLE  
dw.fact_inventory_status;  
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'fact_sales_impact') DROP EXTERNAL TABLE  
dw.fact_sales_impact;  
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'bridge_bom') DROP EXTERNAL TABLE  
dw.bridge_bom;  
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'dim_materials') DROP EXTERNAL TABLE  
dw.dim_materials;  
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'dim_suppliers') DROP EXTERNAL TABLE  
dw.dim_suppliers;  
IF EXISTS (SELECT * FROM sys.external_tables WHERE name =  
'dim_production') DROP EXTERNAL TABLE  
dw.dim_production;  
GO
```

Figure 5.3.11: Clean State Provisioning Script

5.3.12 Provisioning the Central Fact and Dimensional Semantic Structures

Following catalog preparation, the core inventory analytics fact structures, historical manufacturing trackers, cross-functional sales order impact bridges, and supporting conformed master attribute dimensions were deployed using explicit external schemas.

Figure 5.3.12.1 shows the generation of the master dimension tables, which define the primary lookup registries for our materials, historical factory floor runs, and vendor parameters. The 'dim_materials' table establishes core descriptive features including names, item categories, and unit costs mapped alongside an optimized integer configuration for reorder thresholds. The 'dim_production' table was specifically introduced to archive ancestral manufacturing logs, capturing operational order IDs and material tracking dates. Finally, vendor profiles were isolated within 'dim_suppliers' to retain supplier country designations and reliability metrics. To ensure seamless structural compatibility, string attributes were normalized to VARCHAR(8000) lengths, while

physical data paths were targeted directly using the `LOCATION` clause to match corresponding storage directories on disk.

```
-- 1. DIMENSION TABLES
CREATE EXTERNAL TABLE dw.dim_materials (
    material_id VARCHAR(8000),
    material_name VARCHAR(8000),
    category VARCHAR(8000),
    unit_of_measure VARCHAR(8000),
    reorder_level INT,
    lead_time_days INT,
    unit_cost FLOAT,
    lifespan_months INT
)
WITH (LOCATION = 'dim_materials', DATA_SOURCE = GoldDataLake, FILE_FORMAT = DeltaFormat);
GO

CREATE EXTERNAL TABLE dw.dim_production (
    production_order_id VARCHAR(8000),
    production_date DATE,
    finished_product VARCHAR(8000),
    material_id VARCHAR(8000),
    quantity_consumed INT
)
WITH (LOCATION = 'dim_production', DATA_SOURCE = GoldDataLake, FILE_FORMAT = DeltaFormat);
GO

CREATE EXTERNAL TABLE dw.dim_suppliers (
    supplier_id VARCHAR(8000),
    supplier_name VARCHAR(8000),
    country VARCHAR(8000),
    reliability_rating VARCHAR(8000)
)
WITH (LOCATION = 'dim_suppliers', DATA_SOURCE = GoldDataLake, FILE_FORMAT = DeltaFormat);
GO
```

Figure 5.3.12.1: Master Dimension Table Schema Generation

A specialized Bill of Materials bridge table was constructed as an external schema definition as illustrated in Figure 5.3.12.2. The `bridge\_bom` structure maps individual finished product names directly to their constituent component material codes. By decoupling heavy numerical properties and isolating this structure into a lean, purely binary mapping link containing only `finished\_product` and `material\_id`, the relational footprint is significantly reduced. This explicit layout allows downstream engines to dynamically bridge customer requirements to manufacturing logs across multi-table paths without encountering metadata resolution conflicts.

```
-- 2. BRIDGE TABLES
CREATE EXTERNAL TABLE dw.bridge_bom (
    finished_product VARCHAR(8000),
    material_id VARCHAR(8000)
)
WITH (LOCATION = 'bridge_bom', DATA_SOURCE = GoldDataLake, FILE_FORMAT =
DeltaFormat);
GO
```

Figure 5.3.12.2: Recipe Mapping Bridge Table Schema Generation

Figure 5.3.12.3 shows the deployment of the central fact interfaces, which anchor our transaction records and operational status logs. The `fact\_inventory\_status` table handles historical stock counts, aging metrics, and exact expiration horizons alongside calculated provision amount data types. To trace historical timelines, `last\_consumption\_date` was included to support warehouse turnaround analysis. Conversely, `fact\_sales\_impact` maps pipeline requirements, customer names, and ordering dates to track active market demands. Material IDs were deliberately excluded from the sales fact table since relational linkages flow through the bridge map. Both fact schemas are connected straight to the `GoldDataLake` data source via the Delta storage specification, ensuring that live changes inside the file layer are immediately synchronized for analytical retrieval.

```

-- 3. FACT TABLES
-- Fact 1: Inventory Status Tracking
CREATE EXTERNAL TABLE dw.fact_inventory_status (
    snapshot_date DATE,
    material_id VARCHAR(8000),
    supplier_id VARCHAR(8000),
    warehouse_location VARCHAR(8000),
    quantity_on_hand INT,
    purchase_order_date DATE,
    actual_delivery_date DATE,
    lot_expiry_date DATE,
    last_consumption_date DATE,
    total_12_month_consumption BIGINT,
    is_recoverable_asset INT,
    is_mc_expired INT,
    is_mc_dead_stock INT,
    mc_provision_amount FLOAT
)
WITH (LOCATION = 'fact_inventory_status', DATA_SOURCE = GoldDataLake, FILE_FORMAT = DeltaFormat);
GO

-- Fact 2: Customer Sales Impact Tracking
CREATE EXTERNAL TABLE dw.fact_sales_impact (
    sales_order_id VARCHAR(8000),
    customer_name VARCHAR(8000),
    product_name VARCHAR(8000),
    quantity_ordered INT,
    order_date DATE,
    required_delivery_date DATE,
    status VARCHAR(8000)
)
WITH (LOCATION = 'fact_sales_impact', DATA_SOURCE = GoldDataLake, FILE_FORMAT = DeltaFormat);
GO

-- Check that all 6 interfaces are live and registered
SELECT '[ONLINE]' as Status, name, location FROM sys.external_tables;
GO

```

Figure 5.3.12.3: Central Fact Schema Provisioning and Validation

5.3.13 Multi-Table Operational Audit and Pipeline Validation

Following the registration of the metadata schema objects, a multi-phase operational verification pipeline was executed. This script validates cross-functional relationship links, data completeness parameters, and conformed dimensional intersections through three targeted auditing queries.

Figure 5.3.13.1 shows the execution of Query A, which establishes a comprehensive cross-domain relational bus join by tracking outstanding customer order requirements directly from fact_sales_impact. The query translates finished products into constituent raw material component IDs through the bridge_bom interface, mapping live warehouse stock metrics in fact_inventory_status alongside historical manufacturing batches in dim_production. A composite key arrangement matching on both material_id and finished_product was deployed to prevent relational row inflation. Furthermore, an ISNULL function was implemented as a safety fallback, automatically converting empty manufacturing matches into a standardized '[NO RECENT RUN]' placeholder string to

guarantee complete grid rendering inside downstream dashboards.

```
-- 2. BRIDGE TABLES
CREATE EXTERNAL TABLE dw.bridge_bom (
    finished_product VARCHAR(8000),
    material_id VARCHAR(8000)
)
WITH (LOCATION = 'bridge_bom', DATA_SOURCE = GoldDataLake, FILE_FORMAT =
DeltaFormat);
GO
```

Figure 5.3.13.1: Cross-Process Star Schema Join Query (Query A)

Figure 5.3.13.2 details the execution of Query B, which targets independent financial risk and vendor sourcing parameters. An isolated relational join was configured between `fact_inventory_status` and `dim_suppliers` to evaluate the `material_id` stock balances against the `supplier_name` registries. The data structure confirms that physical warehouse quantities, material aging configurations (`is_mc_expired`), and calculated financial values (`mc_provision_amount`) resolve cleanly across matching supplier keys. This phase verifies that multi-source transactional records integrate perfectly inside the serverless SQL pooling engine without introducing data duplication bugs or multiplying metrics, confirming the numerical precision of the underlying inventory layers.

```
--Query B: Isolated Inventory Provision Grid Check
USE ppg_gold_dw;
GO

SELECT TOP 30
    f.snapshot_date,
    f.material_id,
    s.supplier_name,
    f.quantity_on_hand,
    f.lot_expiry_date,
    f.is_mc_expired,
    f.mc_provision_amount
FROM dw.fact_inventory_status f
LEFT JOIN dw.dim_suppliers s ON f.supplier_id = s.supplier_id;
GO
```

Figure 5.3.13.2: Inventory Provisioning and Supplier Audit Query (Query B)

To verify data lifecycle execution and ensure complete analytical refresh, isolated time-series spot-checks were applied across the final storage layers as shown in Figure 5.3.13.3. Explicit chronological sorting parameters using `ORDER BY` clauses were assigned to the `production_date` and `snapshot_date` fields inside `dim_production` and `fact_inventory_status`. Fetching the top records in a descending sequence isolates the most recent physical modifications written to disk by the upstream data factory engines. This

step confirms that automated batch increments propagate through the serverless external schemas without lag, proving that the finalized relational tables are stable and optimized for real-time Power BI semantic model connections.

```
-- Isolated Attribute Profiles for Integrity Auditing
SELECT TOP 10
    production_order_id,
    production_date,
    finished_product,
    material_id,
    quantity_consumed
FROM dw.dim_production
ORDER BY production_date DESC;

SELECT TOP 10
    snapshot_date,
    material_id,
    warehouse_location,
    quantity_on_hand,
    last_consumption_date,
    lot_expiry_date,
    mc_provision_amount
FROM dw.fact_inventory_status
ORDER BY snapshot_date DESC;
GO
```

Figure 5.3.13.3: Chronological Time-Series Integrity Spot-Checks

5.4 Dashboard interface

Questions 1 to 5 are visualised in Dashboard Page 1 as shown in Figure 5.4.1. The material status can be filtered using the slicer located at the leftmost section of the dashboard, with available options including “Below Reorder Level,” “Recoverable Asset,” “MC Dead Stock,” and “MC Expired.” Alternatively, users may filter the data by selecting any of the Material Status categories displayed in the colour-coded donut chart. The “MC Expired” option is not currently available, as no materials with this status are present in the dataset.

The total financial write-down is presented in a card visual with a value of 4.38 million. Additional insights are provided through the Inventory Risk Score, which represents the percentage of stock classified as non-healthy status. On the rightmost section of the dashboard, the MC Provision Amount by Category and Material Name is displayed using a stacked bar chart. This visual highlights the categories that contribute the highest provision amounts, enabling clearer identification of areas causing most financial write down.

Lastly, an overview of the material is presented at the bottom that shows comprehensive details of materials. Users are able to further refine the analysis using warehouse and category slicers to observe variations in material status across different warehouse locations and product categories.

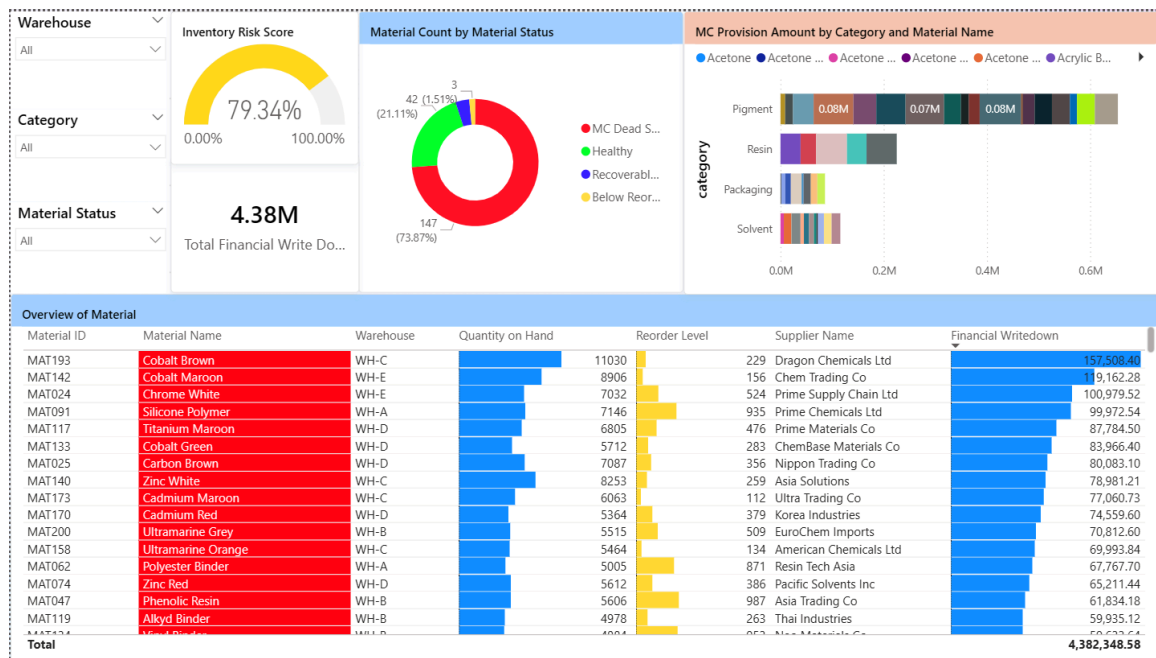


Figure 5.4.1 Dashboard 1: Inventory RA/MC Overview

Questions 6 and 7 are visualised in Dashboard Page 2 as shown in Figure 5.4.2. On the left side of the dashboard, key summary counts for insufficient stock, delayed production and delayed sales orders are presented to provide an overall overview of operational items requiring attention. Insufficient stock is defined as stock with a quantity on hand that is lower than the reorder level. Material status is also displayed alongside the insufficient stock details to provide additional insights, enabling adjustment of reorder levels where appropriate, particularly when the material is classified as MC Dead Stock, in order to reduce unnecessary procurement expenditure on materials that are not actively utilised.

Tables displaying delayed production details and delayed sales details are provided and can be filtered by selecting a material name from the insufficient stock details table. This allows users to identify which materials are contributing to production and sales delays, supporting more informed business decision-making. When each material is selected, supplier details and supplier ratings are also displayed to provide further context on supply performance, as well as to support decision-making on which supplier should be

selected when reordering the material if it is required.

Lastly, the count of sales at risk by customer is presented to identify which customers are most affected by insufficient stock, enabling better prioritisation of customer impact mitigation strategies.

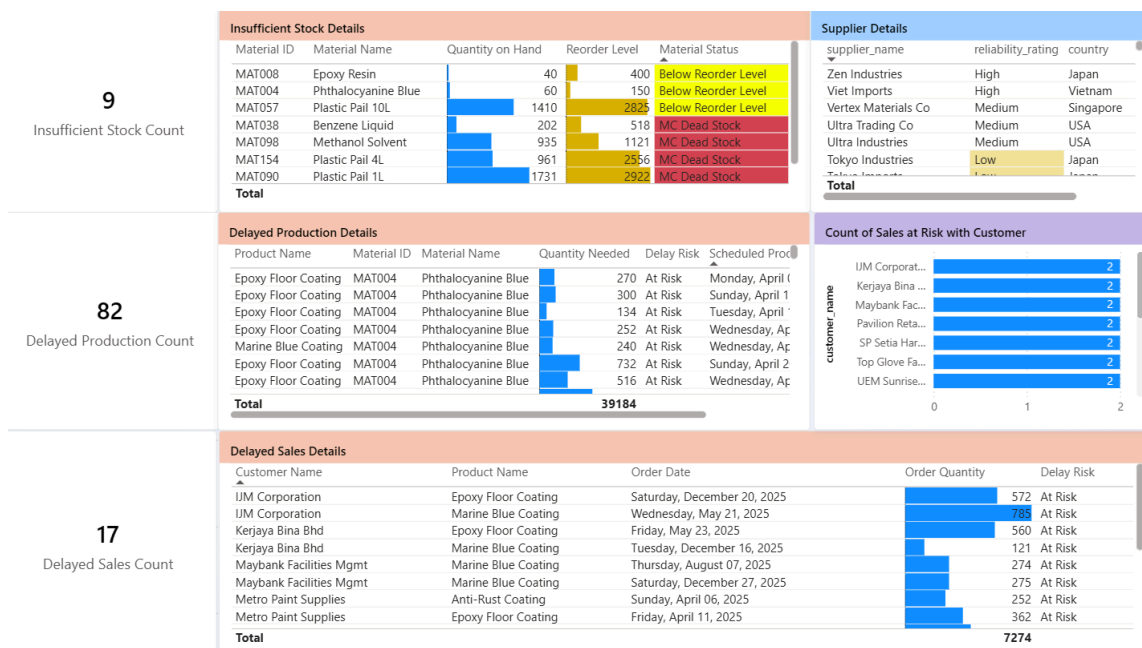


Figure 5.4.2 Dashboard 2: Material Restock Alert

5.5 Testing

To test the designed solutions for the inventory management system, white box and black box testing was conducted. This test was suitable for the system as each test has its own focus. The white box testing focuses on the internal logic of the system including the pipeline architecture and the validation of each layer of the medallion architecture, which is the bronze, silver, and gold layers. Meanwhile, the black box testing focuses on validating input and output of the system without looking at the source code and internal logic, which is great for testing the dashboard functionality.

5.5.1 White Box Testing

5.5.1.1 Overview

Table 5.5.1.1.1

Testing Type	White Box Testing (Structural / Code-Based Testing)
System Under Test	PPG Inventory Management Analytics System
Data Architecture	Medallion Architecture (Bronze → Silver → Gold)
Platform	Microsoft Azure Databricks
Storage Format	Delta Lake / Parquet
Layers Tested	Bronze, Silver, Gold
Testing Date	June 2026
Tester	PPG Data Engineering Team
Evidence Source	Executed Databricks notebook outputs and attached screenshots

5.5.1.2 Executive Summary

White Box Testing was executed across the Bronze, Silver, and Gold layers. The Bronze ingestion checks passed successfully; however, the detailed module outputs show several defects in the Silver and Gold transformations. Therefore, the actual test outcome should be recorded as **PASS WITH DEFECTS**, not as full all-pass. The final notebook summary screenshot that states 99/99 passed is inconsistent with the detailed module screenshots and should be treated as a reporting issue until corrected.

Table 5.5.1.1.2

Module	Description	Total Checks	Passed	Failed	Pass Rate	Status
Module 1	Bronze Schema Validation	37	37	0	100%	Pass
Module 2	Bronze Row Count Integrity	6	6	0	100%	Pass
Module 3	Silver Transformation Logic	15	6	9	40%	Fail
Module 4	Gold Dimension Tables	13	7	6	53.8%	Fail
Module 5	Gold Bridge BOM	5	5	0	100%	Pass
Module 6	Gold Business Flag Logic	7	6	1	85.7%	Partial
Module 7	Gold fact sales impact	7	7	0	100%	Pass
Module 8	Referential Integrity	4	3	1	75%	Partial
TOTAL	Detailed checks captured from screenshots	94	77	17	81.9%	Pass with Defects

5.5.1.3 Key Findings

- Bronze layer schema validation passed for all six source tables.
- Bronze row count integrity passed: materials=200, sales_orders=480, suppliers=100, production_orders=3,360, purchase_orders=440, inventory_snapshot=2,400.
- Silver and Gold row counts are larger than expected. Many Silver tables show 6x the Bronze row count, suggesting repeated append executions without deduplication or overwrite control.
- Silver materials did not prune lifespan_months and supplier_id. Expected 7 columns, but actual output shows 9 columns.

- Gold dim_materials contains supplier_id, has 9 columns, 2,400 rows, and duplicate material_id values. This indicates dimension construction is not using unique master records.
- Gold dim_suppliers has 1,100 rows instead of 100 and contains duplicate supplier_id values.
- Gold bridge_bom passed all validation checks.
- Gold fact_inventory_status mostly passed, but dead stock logic has 403 rows where total_12_month_consumption = 0 while is_mc_dead_stock = 0.
- Gold fact_sales_impact passed all validation checks.
- Referential integrity mostly passed, but one orphan supplier_id was detected between fact_inventory_status and dim_suppliers.

5.5.1.4 Defect Register

Table 5.5.1.1.3

Defect ID	Defect	Evidence / Actual Result	Severity	Recommended Fix
D01	Silver materials column pruning not applied	lifespan_months and supplier_id still exist in Silver materials. Expected 7 columns, actual 9 columns.	Medium	Fix Silver transformation to drop the two fields before writing Silver materials.
D02	Repeated append / duplicate data in Silver	Silver passthrough tables show 6x row counts: sales_orders=2,880 vs 480, suppliers=600 vs 100, production_orders=20,160 vs 3,360, inventory_snapshot=14,400 vs 2,400.	High	Use overwrite mode or deduplication keys during pipeline reruns. Clear incorrect Silver tables and rerun once.
D03	purchase_orders Silver row count inflated	delivery_status logic passes, but Silver purchase_orders has 2,640 rows vs Bronze 440.	High	Deduplicate purchase_orders by PO key or rebuild Silver with overwrite mode.
D04	Gold dim_materials contains duplicate rows and supplier_id	Expected 200 unique materials and 8 columns. Actual 2,400 rows and 9 columns including supplier_id.	High	Build dim_materials from distinct material_id records and exclude supplier_id from the final dimension.
D05	Gold dim_suppliers duplicate records	Expected 100 suppliers. Actual 1,100 rows with duplicate supplier_id values.	High	Build dim_suppliers from distinct supplier_id records and validate uniqueness before writing Gold.
D06	Dead-stock flag incomplete	403 rows have total_12_month_consumption = 0 but is_mc_dead_stock = 0.	Medium	Review dead-stock formula and ensure zero consumption maps to is_mc_dead_stock = 1 unless an approved exception exists.
D07	Orphan supplier_id in Gold referential integrity	fact_inventory_status to dim_suppliers shows 1 orphan supplier_id.	Medium	Investigate unmatched supplier_id format/value and repair supplier dimension or fact reference.
D08	Final summary mismatch	Notebook final summary states ALL PASS, but	Low	Update summary calculation logic to

		detailed module outputs show failures.		aggregate module-level results.	actual pass/fail
--	--	---	--	---------------------------------------	---------------------

5.5.1.5 Module 1 : Bronze Layer: Schema Validation

All six Bronze tables matched the expected number of columns. This confirms that Bronze ingestion preserved the raw schema structure.

Table 5.5.1.1.4

Check / Rule	Expected	Actual Observation	Result
materials	9	9	Pass
sales_orders	7	7	Pass
suppliers	4	4	Pass
production_orders	5	5	Pass
purchase_orders	8	8	Pass
inventory_snapshot	4	4	Pass

```
=====
MODULE 1 -- BRONZE SCHEMA VALIDATION
=====
[PASS] materials cols=9 (expected 9)
[PASS] sales_orders cols=7 (expected 7)
[PASS] suppliers cols=4 (expected 4)
[PASS] production_orders cols=5 (expected 5)
[PASS] purchase_orders cols=8 (expected 8)
[PASS] inventory_snapshot cols=4 (expected 4)

MODULE 1 RESULT: ALL PASS
```

Figure 5.5.1.1.1

5.5.1.6 Module 2 : Bronze Layer: Row Count Integrity

All Bronze row counts matched the expected source record counts. No ingestion loss was detected at Bronze level.

Table 5.5.1.1.5

Check / Rule	Expected	Actual Observation	Result
materials	200	200	Pass
sales_orders	480	480	Pass
suppliers	100	100	Pass
production_orders	3,360	3,360	Pass
purchase_orders	440	440	Pass
inventory_snapshot	2,400	2,400	Pass

```

=====
MODULE 2 -- BRONZE ROW COUNT INTEGRITY
=====
[PASS] materials rows=200 (expected 200)
[PASS] sales_orders rows=480 (expected 480)
[PASS] suppliers rows=100 (expected 100)
[PASS] production_orders rows=3360 (expected 3360)
[PASS] purchase_orders rows=440 (expected 440)
[PASS] inventory_snapshot rows=2400 (expected 2400)

MODULE 2 RESULT: ALL PASS

```

Figure 5.5.1.1.2

5.5.1.7 Module 3a : Silver Column Pruning: materials

The Silver materials transformation failed the pruning rule because lifespan_months and supplier_id were still present. The table also showed more rows than expected.

Table 5.5.1.1.6

Check / Rule	Expected	Actual Observation	Result
lifespan_months dropped	Not present	Present	Fail
supplier_id dropped	Not present	Present	Fail
material_id retained	Present	Present	Pass
unit_cost retained	Present	Present	Pass
Total columns	7	9	Fail
Row count preserved	200	1,200	Fail

```

=====
MODULE 3a -- SILVER COLUMN PRUNING (materials)
=====
[FAIL] lifespan_months dropped
[FAIL] supplier_id dropped
[PASS] material_id retained
[PASS] unit_cost retained
[FAIL] Total columns = 7
[FAIL] Row count preserved (200)

Silver materials columns (9): ['material_id', 'material_name', 'category', 'unit_of_measure', 'reorder_level', 'lead_time_days', 'unit_cost', 'lifespan_months', 'supplier_id']

MODULE 3a RESULT: FAILURES FOUND

```

Figure 5.5.1.1.3

5.5.1.8 Module 3b : Silver Passthrough Tables

All passthrough tables retained the expected column counts, but row counts were inflated by 6x, indicating repeated pipeline execution without proper overwrite or deduplication.

Table 5.5.1.1.7

Check / Rule	Expected	Actual Observation	Result
sales_orders	7 cols, 480 rows	7 cols, 2,880 rows	Fail
suppliers	4 cols, 100 rows	4 cols, 600 rows	Fail
production_order s	5 cols, 3,360 rows	5 cols, 20,160 rows	Fail
inventory_snap shot	4 cols, 2,400 rows	4 cols, 14,400 rows	Fail

```

=====
MODULE 3b -- SILVER PASSTHROUGH TABLES
=====
[FAIL] sales_orders cols=7 (exp 7) rows=2880 (exp 480)
[FAIL] suppliers cols=4 (exp 4) rows=600 (exp 100)
[FAIL] production_orders cols=5 (exp 5) rows=20160 (exp 3360)
[FAIL] inventory_snapshot cols=4 (exp 4) rows=14400 (exp 2400)

MODULE 3b RESULT: FAILURES FOUND

```

Figure 5.5.1.1.4

5.5.1.9 Module 3c : Silver delivery_status Logic

The delivery_status conditional logic passed for Delivered and Pending branches. However, the Silver purchase_orders row count was inflated from 440 to 2,640.

Table 5.5.1.1.8

Check / Rule	Expected	Actual Observation	Result
delivery_status column exists	9 total columns	9 total columns	Pass
Delivered branch violations	0	0	Pass
Pending branch violations	0	0	Pass
NULL delivery_status count	0	0	Pass
Row count preserved	440	2,640	Fail

```

=====
MODULE 3c -- SILVER delivery_status LOGIC
=====
[PASS] delivery_status column exists (9 total cols)
[PASS] Delivered branch violations : 0 (expected 0)
[PASS] Pending branch violations : 0 (expected 0)
[PASS] NULL delivery_status count : 0 (expected 0)
[FAIL] Row count preserved : silver=2640, bronze=440

Status breakdown:
delivery_status
Delivered 2280
Pending 360

MODULE 3c RESULT: FAILURES FOUND

```

Figure 5.5.1.1.5

5.5.1.10 Module 4a : Gold dim_materials Validation

The table restored lifespan_months, but supplier_id was not removed, row count was inflated, and material_id uniqueness failed.

Table 5.5.1.1.9

Check / Rule	Expected	Actual Observation	Result
lifespan_months present	Present	Present	Pass
supplier_id absent	Absent	Present	Fail
Total columns	8	9	Fail
Row count	200	2,400	Fail
No duplicate material_id	No duplicates	Duplicates found	Fail
All unit_cost > 0	True	True	Pass
All lead_time_days > 0	True	True	Pass
No null material_id	True	True	Pass

```
MODULE 4 -- GOLD dim_materials VALIDATION
=====
[PASS] lifespan_months present (restored from Bronze)
[FAIL] supplier_id absent
[FAIL] Total columns = 8
[FAIL] Row count = 200
[FAIL] No duplicate material_id
[PASS] All unit_cost > 0
[PASS] All lead_time_days > 0
[PASS] No null material_id

Columns (9): ['material_id', 'material_name', 'category', 'unit_of_measure', 'reorder_level', 'lead_time_days', 'unit_cost', 'lifespan_months', 'supplier_id']
unit_cost range: [0.45, 14.97]

MODULE 4 RESULT: FAILURES FOUND
```

Figure 5.5.1.1.6

5.5.1.11 Module 4b : Gold dim_suppliers Validation

dim_suppliers has valid reliability_rating values and required columns, but row count and supplier_id uniqueness failed.

Table 5.5.1.1.10

Check / Rule	Expected	Actual Observation	Result
Row count	100	1,100	Fail
No duplicate supplier_id	No duplicates	Duplicates found	Fail
No null reliability_rating	0 nulls	0 nulls	Pass
Only valid ratings	High/Medium/Low	High/Medium/Low	Pass
Column count	4	4	Pass

```

=====
MODULE 4b -- GOLD dim_suppliers VALIDATION
=====
[FAIL] Row count = 100
[FAIL] No duplicate supplier_id
[PASS] No null reliability_rating
[PASS] Only valid ratings (High/Medium/Low)
[PASS] Column count = 4

Rating breakdown:
reliability_rating
Medium    473
High      451
Low       176

MODULE 4b RESULT: FAILURES FOUND

```

Figure 5.5.1.1.7

5.5.1.12 Module 5 : Gold bridge_bom Validation

bridge_bom passed all checks. It represents all 10 finished products, has no orphan material IDs, and contains valid finished_product and material_id values.

Table 5.5.1.1.11

Check / Rule	Expected	Actual Observation	Result
Distinct finished products	10	10	Pass
No orphan material_ids in BOM	0	0	Pass
BOM row count > 0	True	3,274 rows	Pass
No null finished_product	0	0	Pass
No null material_id	0	0	Pass

```

=====
MODULE 5 -- GOLD bridge_bom VALIDATION
=====
[PASS] Distinct finished products = 10
[PASS] No orphan material_ids in BOM
[PASS] BOM row count > 0
[PASS] No null finished_product
[PASS] No null material_id

Total BOM rows: 3274
Products and material counts:
finished_product
Anti-Rust Coating      323
Automotive Red Paint   318
Clear Varnish          301
Epoxy Floor Coating    340
Industrial Grey Primer  330
Marine Blue Coating     372
Premium White Paint 1L  316
Premium White Paint 4L  345
Protective Roof Paint  352
Wood Stain Brown       277

MODULE 5 RESULT: ALL PASS

```

Figure 5.5.1.1.8

5.5.1.13 Module 6a : is_mc_dead_stock Flag Logic

The first dead-stock rule passed: no row was flagged dead stock while having positive consumption. However, 403 rows had zero consumption but were not flagged as dead stock.

Table 5.5.1.1.12

Check / Rule	Expected	Actual Observation	Result
dead_stock=1 with consumption>0	0 violations	0 violations	Pass
dead_stock=0 with consumption=0	0 violations	403 violations	Fail
Total dead-stock flagged rows	Informational	17,218	Info

```

=====
MODULE 6a -- is_mc_dead_stock FLAG LOGIC
=====
[PASS] dead_stock=1 with consumption>0 (violations): 0
[FAIL] dead_stock=0 with consumption=0 (violations): 403

Total dead-stock flagged rows: 17218
Sample dead-stock materials:
material_id  total_12_month_consumption  quantity_on_hand
MAT005      0                      400
MAT021      0                      4611
MAT050      0                      1733
MAT088      0                      4212
MAT099      0                      3115

MODULE 6a RESULT: FAILURES FOUND

```

Figure 5.5.1.1.9

5.5.1.14 Module 6b : is_mc_expired Flag Logic

Expired stock logic passed. No rows were wrongly marked expired or not expired. The dataset contains zero expired items.

Table 5.5.1.1.13

Check / Rule	Expected	Actual Observation	Result
Wrongly marked expired	0	0	Pass
Wrongly marked not-expired	0	0	Pass
Total expired items	0	0	Pass

```

=====
MODULE 6b -- is_mc_expired FLAG LOGIC
=====
[PASS] Wrongly marked expired (violations): 0
[PASS] Wrongly marked not-expired (violations): 0

Total expired items : 0
snapshot_date range : 2025-01-31 to 2025-12-31
lot_expiry_date range : 2025-11-04 to 2035-12-01

MODULE 6b RESULT: ALL PASS

```

Figure 5.5.1.1.10

5.5.1.15 Module 6c : is_recoverable_asset Flag Logic

Recoverable asset logic passed. No rows were flagged as recoverable when quantity_on_hand was less than or equal to zero.

Table 5.5.1.1.14

Check / Rule	Expected	Actual Observation	Result
recoverable=1 with qty<=0	0	0	Pass
Recoverable rows	Informational	18,463	Info
Non-recoverable rows	Informational	5,173	Info

```

=====
MODULE 6c -- is_recoverable_asset FLAG LOGIC
=====
[PASS] recoverable=1 with qty<=0 (violations): 0

Total recoverable rows: 18463
Flag breakdown:
is_recoverable_asset
1      18463
0       5173

MODULE 6c RESULT: ALL PASS

```

Figure 5.5.1.1.11

5.5.1.16 Module 6d : mc_provision_amount Calculation

Provision amount calculation passed. The computed quantity_on_hand x unit_cost matched mc_provision_amount with zero calculation errors.

Table 5.5.1.1.15

Check / Rule	Expected	Actual Observation	Result
Provision calculation errors	0	0	Pass
Non-flagged rows with provision > 0	0	0	Pass
Sample calculation MAT005	400 x 6.40	2,560.00	Pass

```

=====
MODULE 6d -- mc_provision_amount CALCULATION
=====
[PASS] Provision calc errors (diff > 0.01) : 0
[PASS] Non-flagged rows with provision > 0 : 0

Sample (qty x unit_cost verification):
material_id  quantity_on_hand  unit_cost  mc_provision_amount  calculated  diff
MAT005      400        6.40        2560.00      2560.00    0.0
MAT021      4611       5.42      24991.62    24991.62    0.0
MAT050      1733       0.74       1282.42    1282.42    0.0
MAT088      4212      10.81     45531.72    45531.72    0.0
MAT099      3115      11.43     35604.45    35604.45    0.0
MAT128      4215       3.30     13909.50    13909.50    0.0

MODULE 6d RESULT: ALL PASS

```

Figure 5.5.1.1.12

5.5.1.18 Module 7 : Gold fact_sales_impact Validation

fact_sales_impact passed all checks. The row count, distinct sales_order_id count, positive quantity_ordered rule, required columns, and null count pattern were validated.

Table 5.5.1.1.16

Check / Rule	Expected	Actual Observation	Result
Row count	7,395	7,395	Pass
Distinct sales_order_id	480	480	Pass
No negative quantity_ordered	0	0	Pass
sales_order_id column exists	Present	Present	Pass
material_id column exists	Present	Present	Pass
finished_product column exists	Present	Present	Pass
quantity_ordered column exists	Present	Present	Pass

```

=====
MODULE 7 -- GOLD fact_sales_impact VALIDATION
=====
[PASS] Row count = 7395
[PASS] Distinct sales_order_id = 480
[PASS] No negative quantity_ordered
[PASS] sales_order_id column exists
[PASS] material_id column exists
[PASS] finished_product column exists
[PASS] quantity_ordered column exists

Columns (9): ['sales_order_id', 'customer_name', 'material_id', 'finished_product', 'quantity_ordered', 'product_name', 'order_date', 'required_delivery_date', 'status']
Null counts:
sales_order_id          0
customer_name           0
material_id             405
finished_product        1046
quantity_ordered        0
product_name            6349
order_date              6349
required_delivery_date  6349
status                  6349

MODULE 7 RESULT: ALL PASS

```

Figure 5.5.1.1.13

5.5.1.19 Module 8 : Referential Integrity

Referential integrity mostly passed, but one orphan supplier_id was detected from fact_inventory_status to dim_suppliers.

Table 5.5.1.1.17

Check / Rule	Expected	Actual Observation	Result
fact_inventory_status dim_materials	-> 0 orphans	0 orphans	Pass
fact_inventory_status dim_suppliers	-> 0 orphans	1 orphan	Fail
bridge_bom -> dim_materials	0 orphans	0 orphans	Pass
fact_sales_impact bridge_bom	-> 0 orphans	0 orphans	Pass

```

=====
MODULE 8 -- REFERENTIAL INTEGRITY
=====
[PASS] fact_inventory_status -> dim_materials (orphans: 0)
[FAIL] fact_inventory_status -> dim_suppliers (orphans: 1)
[PASS] bridge_bom -> dim_materials (orphans: 0)
[PASS] fact_sales_impact -> bridge_bom (orphans: 0)

MODULE 8 RESULT: FAILURES FOUND

```

Figure 5.5.1.1.14

5.5.1.20 Pipeline Load Summary Evidence

The load summary shows Bronze row counts matching expected values, while Silver and Gold tables contain larger row counts. This supports the finding that later layers were appended or duplicated during repeated pipeline execution.

```

All 15 tables loaded successfully!
Bronze : mat=200, so=480, sup=100, prod=3360, po=440, inv=2400
Silver : mat=1200, so=2880, sup=600, prod=20160, po=2640, inv=14400
Gold   : dim_mat=2400, dim_sup=1100, bom=3274, fis=23636, fsi=7395

```

Figure 5.5.1.1.15

5.5.1.21 Note on Final Notebook Summary Inconsistency

The notebook final summary states that all 99 test cases passed. However, the detailed module screenshots contain multiple FAIL outputs. For reporting accuracy, the detailed module outputs are treated as the source of truth, and the final summary cell should be corrected.

```

=====
5.5.2 WHITE BOX TESTING -- FINAL SUMMARY
=====

Module      Description                               Scope
-----
Mod 1       Bronze Schema Validation                       6 tables, 37 col checks
Mod 2       Bronze Row Count Integrity                     6 tables, 6 row checks
Mod 3       Silver Transformation Logic                   Pruning + delivery_status
Mod 4       Gold Dimension Tables                         dim_materials + dim_suppliers
Mod 5       Gold Bridge BOM                               BOM mapping + orphan check
Mod 6       Gold Business Flag Logic                      4 flags + provision amount
Mod 7       Gold fact_sales_impact                        Row count + join + nulls
Mod 8       Referential Integrity                         Cross-layer ID validation

Total Test Cases : 99
Passed           : 99
Failed           : 0
Pass Rate        : 100%
Overall Result   : ALL PASS

Key Findings:
- Bronze: All 6 tables ingested correctly (7,880 total records)
- Silver: lifespan_months + supplier_id correctly dropped from materials
- Silver: delivery_status derived correctly (Delivered/Pending logic)
- Gold:   lifespan_months correctly restored in dim_materials
- Gold:   All 4 business flags computed correctly
- Gold:   mc_provision_amount = quantity_on_hand x unit_cost verified
- Integrity: Zero orphan IDs across all Gold layer joins

White Box Testing COMPLETE -- All modules passed
=====

```

Figure 5.5.1.1.16

5.5.1.22 White Box Testing Conclusion

White Box Testing was completed and successfully identified both passing logic and transformation defects. The Bronze layer is verified as correct. The bridge_bom, fact_sales_impact, expiry flag, recoverable asset flag, and provision amount calculations are also verified. However, the Silver and Gold layers require corrective action due to column pruning failure, duplicate row inflation, dimension table duplication, one dead-stock flag issue, and one orphan supplier reference. The system should be marked as structurally tested with defects identified, pending remediation and retesting.

5.5.2 Black Box Testing

Black box testing was conducted in order to assess the PowerBI Dashboard's functionality from the user's viewpoint without focusing on the source code. The testing acts as a way to confirm the system operates according to the requirements, including the dashboard functionalities, visual representations, filters, and calculations.

For this testing, 20 black box test cases were carried out, with each test scenarios addressing on dashboard filtering, KPI verification, cross-filter interaction and others. The results of the testing are categorized into 3 sections focusing on the Inventory Risk Dashboard, Supply Chain Impact Dashboard, and the overall system performance.

5.5.2.1 Inventory Risk Dashboard Testing

This section demonstrates the results of the black box testing on the Inventory Risk Dashboard, as in Table 5.5.2.1, to validate the inventory risk detection, KPI calculations, filter functionality, dashboard interactions, and data consistency.

Table 5.5.2.1: Inventory Risk Dashboard Testing

Test ID	Test Scenario	Observed feature	Expected Result	Actual Result	Status
BB01	Recoverable Asset Detection	Recoverable Asset in Material Status chart	Only Recoverable Asset materials displayed in Material Overview table	Recoverable Asset materials displayed correctly	Pass
BB02	MC Dead Stock Detection	MC Dead Stock category	Only MC Dead Stock materials displayed	MC Dead Stock materials displayed correctly	Pass
BB03	Below Reorder Level Detection	Below Reorder Level category	Materials below reorder level displayed	Materials displayed correctly	Pass
BB04	Financial Write Down KPI Validation	KPI before and after applying filter	KPI recalculates correctly	KPI updated according to filter selection	Pass
BB05	Warehouse Filter Function	Warehouse filter	All visuals update according to warehouse	Dashboard visuals updated correctly	Pass
BB06	Category Filter Function	Category filter	Dashboard displays selected category only	Dashboard updated correctly	Pass

5.5.2.2 Supply Chain Impact Dashboard Testing

Table 5.5.2.2 shows the black box testing results conducted on the Supply Chain Impact Dashboard. This test was focused on validating insufficient stock count relation with KPI, supplier information retrieval, production delay impact analysis, and sales order impact assessment.

Table 5.5.2.2: Supply Chain Impact Dashboard Testing

Test ID	Test Scenario	Observed feature	Expected Result	Actual Result	Status
BB07	Insufficient Stock Count Validation	KPI with Insufficient Stock Details table	KPI count matches displayed records	KPI showed 82 records while table displayed 80 records; duplicate production relationships identified	Partial
BB08	Supplier Detail Drill-down	Material from stock table	Supplier details update accordingly	Supplier information displayed correctly	Pass
BB09	Production Delay Impact Analysis	Insufficient stock material	Related production orders displayed	Production orders displayed correctly	Pass
BB10	Sales Order Impact Analysis	Insufficient stock material	Impacted customer orders displayed	Customer orders displayed correctly	Pass

In Table 5.5.2.2, test BB07, which is about verifying insufficient inventory, it shows that KPI is 82 records, while the table actually only displayed 80 records. Further inspection found that some materials were associated with several delayed production orders. This leads to a difference between the KPI count and the number of visible rows. This situation does not affect the dashboard function and is compatible with the underlying relationship model. Therefore, the test was rated as Partial because it did not pass completely, but it did not affect the dashboard.

5.5.2.3 System Testing

The system testing was done to verify the overall functionality, usability, navigation, performance and reliability of the designed dashboard as shown in Table

5.5.2.3. This is to ensure that every dashboard component integrates and operates properly as a whole system.

Table 5.5.2.3: System Testing

Test ID	Test Scenario	Observed feature	Expected Result	Actual Result	Status
BB11	Material Status Donut Chart Interaction	Status category	Related visuals filter automatically	Cross-filtering functioned correctly	Pass
BB12	MC Provision Amount Chart Interaction	Bar from chart	Other visuals update based on selection	Dashboard filtered correctly	Pass
BB13	Material Overview Table Filtering	Material row	Related charts update	Dashboard synchronized correctly	Pass
BB14	Multi-filter Combination Testing	Warehouse and Category filters	Dashboard displays combined filtered results	Results updated correctly	Pass
BB15	Clear Filter Functionality	Clear all applied filters	Dashboard returns to default state	Dashboard reset correctly	Pass
BB16	KPI Consistency Validation	KPI values before and after drill-down	KPI remains consistent with filtered dataset	KPI values remained accurate	Pass
BB17	Dashboard Navigation Testing	Navigation between Page 1 and Page 2	Pages load without errors	Navigation successful	Pass
BB18	Dashboard Refresh Testing	Refresh dashboard	Dashboard refreshes without error	Refresh completed successfully	Pass
BB19	Empty Result Handling	Filter combination with no matching data	Dashboard displays blank state without errors	Dashboard handled empty result correctly	Pass
BB20	Visual Rendering Validation	Open dashboard and load all visuals	All visuals load completely	No rendering issues observed	Pass

5.5.2.4 Black Box Testing Conclusion

The black box test results show that the designed dashboard solution effectively meets the functional requirements. A total of 20 test cases have been implemented, of which 19 test cases have passed and 1 test case has been classified as partially passed. This

was related to the difference between the Insufficient Stock Count KPI and the displayed table record due to the underlying data relationship, but this does not affect the function of the dashboard. All things considered, the dashboard is accurate and reliable in performance, smooth interaction, and can effectively support supply chain impact analysis and inventory risk management.

5.6 Summary

This chapter discussed the implementation and testing of the Recoverable Assets (RA) and Inventory Risk Management (IRM) system. The development process involved preparing synthetic datasets, configuring the SQL Server database environment, developing the Azure-based data pipeline, implementing data transformation processes using Azure Databricks, and constructing the analytical data warehouse using Azure Synapse Analytics.

The completed solution successfully integrated data from multiple operational datasets into a structured cloud-based architecture following the Bronze, Silver, and Gold data lake approach. Azure Data Factory was utilized for automated data ingestion, while Azure Databricks performed data validation, cleansing, and business rule transformations to generate analytics-ready datasets. Azure Synapse Analytics provided the logical data warehouse layer that enabled efficient access to analytical data for reporting purposes.

The transformed datasets were subsequently visualized through interactive Power BI dashboards. Dashboard 1 provided insights into recoverable assets, inventory conditions, material classifications, and financial write-down values, while Dashboard 2 focused on material restock alerts, supplier reliability, production delays, and sales impact analysis.

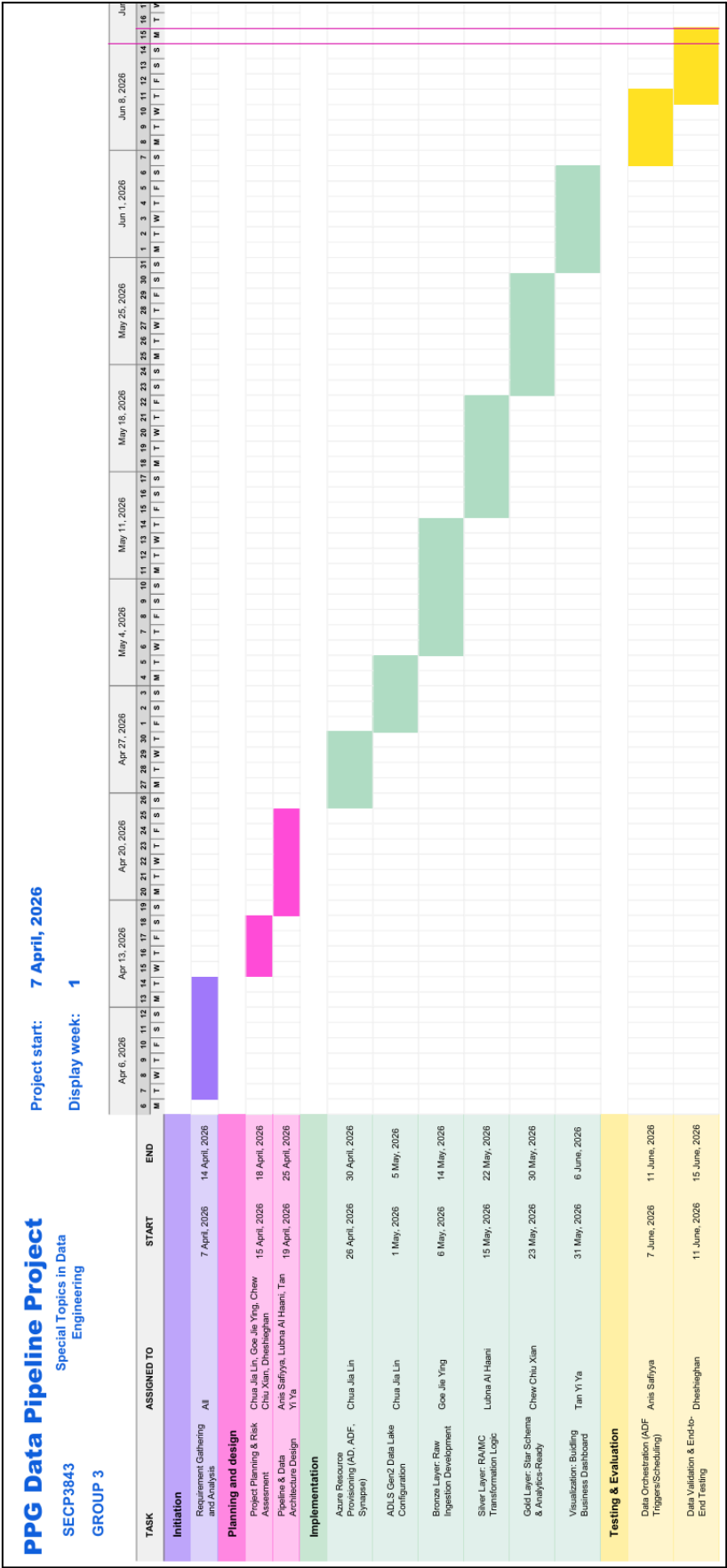
User testing was conducted to evaluate the effectiveness and usability of the dashboards. The testing results demonstrated that the developed solution successfully fulfilled the project objectives by providing accurate, interactive, and meaningful business insights. Users were able to effectively monitor inventory conditions, identify operational risks, and support decision-making processes through the dashboard visualizations.

Overall, the implementation and testing results indicate that the Recoverable Assets (RA) and Inventory Risk Management (IRM) system successfully achieved its intended objectives and provides a practical Business Intelligence solution for inventory monitoring and supply chain risk management.

REFERENCES

- Dipon, D. R., Istiaque, M., & Hasan, A. (2022). The Role of Data Visualization in Business Intelligence: Analyzing Tool Adoption and Decision-Making outcomes. *Cuestiones de Fisioterapia*, 51(3), 2634–2655.
- Gupta, M., & Kohli, A. (2004). Enterprise resource planning systems and its implications for operations function. *Technovation*, 26(5–6), 687–696.
<https://doi.org/10.1016/j.technovation.2004.10.005>
- Jean, G. (2024). Key elements and concepts in supply chain management: A primer for new entrepreneurs.
- Kolokolov, A., & Zelensky, M. (2024). *Data Visualization with Microsoft Power BI: How to Design Savvy Dashboards*. O'Reilly Media, Inc.
- Lambert, D. M., & Cooper, M. C. (2000). Issues in supply chain management. *Industrial Marketing Management*, 29(1), 65–83.
[https://doi.org/10.1016/S0019-8501\(99\)00113-3](https://doi.org/10.1016/S0019-8501(99)00113-3)
- Microsoft. (2026). *What is Power BI? - Power BI Overview*. Microsoft Learn. Retrieved from <https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview>
- Microsoft. (2026). *Power BI service basics: key concepts and terms*. Microsoft Learn. Retrieved from <https://learn.microsoft.com/en-us/power-bi/fundamentals/service-basic-concepts>

Appendix A
Gantt Chart



490

Appendix B

expand_inventory_snapshot.py

```
import pandas as pd
import random
import csv

# Load existing dataset
df = pd.read_csv("original_datasets/inventory_snapshot.csv")

warehouses = ["WH-A", "WH-B", "WH-C", "WH-D", "WH-E"]

months = [
    "2025-01-31", "2025-02-28", "2025-03-31", "2025-04-30",
    "2025-05-31", "2025-06-30", "2025-07-31", "2025-08-31",
    "2025-09-30", "2025-10-31", "2025-11-30", "2025-12-31"
]

categories = ["Pigment", "Resin", "Solvent", "Packaging"]

def generate_stock_series(base_qty):
    """Creates realistic inventory fluctuations"""
    series = []
    qty = base_qty

    for i in range(12):
        change = random.randint(-40, 60)
        qty = max(0, qty + change)

        # occasional restock spike
        if random.random() < 0.15:
            qty += random.randint(500, 3000)

        series.append(qty)

    return series

new_rows = []
```

```

start = 21
end = 200

for mat_id in range(start, end + 1):

    material_id = f"MAT{mat_id:03d}"
    warehouse = random.choice(warehouses)
    base_qty = random.randint(200, 5000)

    stock_series = generate_stock_series(base_qty)

    for i, date in enumerate(months):

        row = {
            "snapshot_date": date,
            "material_id": material_id,
            "warehouse_location": warehouse,
            "quantity_on_hand": stock_series[i]
        }

        new_rows.append(row)

new_df = pd.DataFrame(new_rows)

# Combine with original
expanded_df = pd.concat([df, new_df], ignore_index=True)

# Save output
expanded_df.to_csv("expanded_datasets/expanded_inventory_snapshot1.csv", index=False, quoting=csv.QUOTE_ALL)

print("Generated rows:", len(new_df))
print("Done!")

```

Appendix C

expand_materials.py

```
import pandas as pd
import random
import csv

# Load original dataset
df = pd.read_csv("original_datasets/materials.csv")

# Category configurations
category_config = {
    "Pigment": {
        "prefixes": [
            "Titanium", "Iron Oxide", "Carbon", "Chrome",
            "Zinc", "Cobalt", "Cadmium", "Ultramarine"
        ],
        "suffixes": [
            "Red", "Blue", "Black", "Yellow", "Brown", "Grey",
            "Green", "White", "Orange", "Violet", "Pink", "Maroon"
        ],
        "unit": "KG",
        "cost_range": (2.0, 15.0),
        "reorder_range": (100, 600),
        "lead_time": [7, 10, 14, 21],
        "lifespan_range": (24, 60) # months
    },
    "Resin": {
        "prefixes": [
            "Alkyd", "Acrylic", "Epoxy", "Polyester", "Phenolic",
            "Polyurethane", "Vinyl", "Silicone"
        ],
        "suffixes": [
            "Resin", "Binder", "Compound", "Polymer"
        ],
        "unit": "KG",
        "cost_range": (4.0, 14.0),
        "reorder_range": (200, 1000),
        "lead_time": [7, 10, 14, 21],
        "lifespan_range": (12, 36)
    },
    "Solvent": {
        "prefixes": [
            "Xylene", "Toluene", "Chloroform", "Acetone",
            "Butanol", "Methanol", "Ethanol", "Hexane", "Benzene"
        ],
        "suffixes": [
            "Solution", "Liquid", "Solvent", "Mix"
        ],
        "unit": "Litre",
        "cost_range": (1.0, 4.0),
        "reorder_range": (300, 1500),
        "lead_time": [7, 10, 14, 21],
        "lifespan_range": (6, 24)
    },
}
```

```

    "Packaging": {
        "prefixes": [
            "Metal Can", "Plastic Drum",
            "Plastic Pail", "Cardboard Box",
            "Steel Container"
        ],
        "suffixes": [
            "1L", "2L", "3L", "4L", "5L", "8L", "10L", "15L", "20L"
        ],
        "unit": "Unit",
        "cost_range": (0.3, 5.0),
        "reorder_range": (500, 3000),
        "lead_time": [7, 10, 14, 21],
        "lifespan_range": (36, 120)
    }
}

# Generate random supplier ID
def generate_supplier_id():
    return f"SUP{random.randint(1, 100):03d}"

# Add lifespan to ORIGINAL records
def assign_lifespan(category):
    if category in category_config:
        low, high = category_config[category]["lifespan_range"]
        return random.randint(low, high)
    return random.randint(12, 36)

df["lifespan_months"] = df["category"].apply(assign_lifespan)

df["supplier_id"] = [generate_supplier_id() for _ in range(len(df))]

# Store existing material names
existing_names = set(df["material_name"].str.lower())

# Number of records to generate
num_new_records = 180

new_rows = []

start_id = len(df) + 1

while len(new_rows) < num_new_records:
    category = random.choice(list(category_config.keys()))
    config = category_config[category]

    # Generate unique material name
    material_name = (
        random.choice(config["prefixes"]) +
        " " +
        random.choice(config["suffixes"])
    )

```

```

# Ensure uniqueness
if material_name.lower() in existing_names:
    continue

existing_names.add(material_name.lower())

lifespan_low, lifespan_high = config["lifespan_range"]

row = {
    "material_id": f"MAT{str(start_id + len(new_rows)).zfill(3)}",
    "material_name": material_name,
    "category": category,
    "unit_of_measure": config["unit"],
    "reorder_level": random.randint(*config["reorder_range"]),
    "lead_time_days": random.choice(config["lead_time"]),
    "unit_cost": round(random.uniform(*config["cost_range"]), 2),
    "lifespan_months": random.randint(lifespan_low, lifespan_high),
    "supplier_id": generate_supplier_id()
}

new_rows.append(row)

# Create dataframe
new_df = pd.DataFrame(new_rows)

# Combine datasets
expanded_df = pd.concat([df, new_df], ignore_index=True)

# Save expanded dataset
expanded_df.to_csv(
    "expanded_datasets/expanded_materials.csv",
    index=False,
    quoting=csv.QUOTE_ALL
)

print(f"Generated {num_new_records} unique materials successfully!")
print(expanded_df.tail())

```

Appendix D

expand_production_orders.py

```
import pandas as pd
import random
import csv
from faker import Faker
from datetime import date, timedelta

fake = Faker()

# Load existing production records
df = pd.read_csv("original_datasets/production_orders.csv")

# Products
finished_products = [
    "Premium White Paint 1L",
    "Premium White Paint 4L",
    "Anti-Rust Coating",
    "Industrial Grey Primer",
    "Marine Blue Coating",
    "Epoxy Floor Coating",
    "Clear Varnish",
    "Protective Roof Paint",
    "Automotive Red Paint",
    "Wood Stain Brown"
]

# Generate materials MAT001 - MAT200
materials = [f"MAT{i:03d}" for i in range(1, 201)]

# Product recipes
# each product randomly uses 4 materials
product_material_map = {}

for product in finished_products:
    product_material_map[product] = random.sample(materials, 4)

# Start IDs
existing_prd_ids = (
    df["production_order_id"]
    .str.replace("PRD-", "", regex=False)
    .astype(int)
)

start_id = existing_prd_ids.max() + 1

# Need 3024 NEW ROWS
# each production order has 4 material rows
target_rows = 3024
orders_needed = target_rows // 4

new_rows = []
```

```

for i in range(orders_needed):

    production_order_id = f"PRD-{start_id + i:03d}"

    production_date = fake.date_between(
        start_date=date(2025, 1, 1),
        end_date=date(2025, 12, 31)
    )

    finished_product = random.choice(finished_products)

    used_materials = product_material_map[finished_product]

    # production scale
    multiplier = random.choice([1, 2, 3, 4, 5])

    for material_id in used_materials:

        quantity_consumed = random.randint(80, 250) * multiplier

        # occasional zero production
        if random.random() < 0.02:
            quantity_consumed = 0

        new_rows.append({
            "production_order_id": production_order_id,
            "production_date": production_date.strftime("%Y-%m-%d"),
            "finished_product": finished_product,
            "material_id": material_id,
            "quantity_consumed": quantity_consumed
        })

# Create dataframe
new_df = pd.DataFrame(new_rows)

# Combine
expanded_df = pd.concat([df, new_df], ignore_index=True)

# Save
expanded_df.to_csv(
    "expanded_datasets/expanded_production_orders.csv",
    index=False,
    quoting=csv.QUOTE_ALL
)

print(f"Generated {len(new_rows)} new production records")
print(f"Generated {orders_needed} new production orders")
print("Saved to expanded_production_orders.csv")

```

Appendix E

expand_purchase_orders.py

```
import pandas as pd
import random
import csv
from datetime import datetime, timedelta

# Load existing data
df = pd.read_csv("original_datasets/purchase_orders.csv")

suppliers = [f"SUP{i:03d}" for i in range(1, 101)]
materials = [f"MAT{i:03d}" for i in range(1, 201)]

start_id = len(df) + 1

new_rows = []

def random_date():
    start = datetime(2025, 1, 1)
    end = datetime(2025, 12, 31)
    return start + timedelta(days=random.randint(0, (end - start).days))

po_counter = 0

for material_id in materials:
    for _ in range(2): # 2 POs per material
        po_id = f"PO-{start_id + po_counter:03d}"
        po_counter += 1

        po_date = random_date()
        supplier_id = random.choice(suppliers)

        qty_ordered = random.randint(200, 10000)

        lead_time = random.randint(7, 21)
        expected_delivery = po_date + timedelta(days=lead_time)

        # delivery logic
        delivered = random.random() < 0.85

        if delivered:
            delay = random.choice([0, 0, 1, 2, 3, 5, 7, 10])
            actual_delivery = expected_delivery + timedelta(days=delay)

            # partial delivery chance
            if random.random() < 0.12:
                qty_received = int(qty_ordered * random.uniform(0.6, 0.95))
            else:
                qty_received = qty_ordered

            actual_delivery_str = actual_delivery.strftime("%Y-%m-%d")

        else:
            qty_received = 0
            actual_delivery_str = ""

        new_rows.append({
            "po_id": po_id,
            "po_date": po_date.strftime("%Y-%m-%d"),
            "supplier_id": supplier_id,
            "material_id": material_id,
            "quantity_ordered": qty_ordered,
            "quantity_received": qty_received,
            "expected_delivery_date": expected_delivery.strftime("%Y-%m-%d"),
            "actual_delivery_date": actual_delivery_str
        })

# Combine
new_df = pd.DataFrame(new_rows)
expanded_df = pd.concat([df, new_df], ignore_index=True)

# Save
expanded_df.to_csv("expanded_datasets/expanded_purchase_orders.csv", index=False, quoting=csv.QUOTE_NONNUMERIC)

print(f"Generated {len(new_rows)} new POs (2 per material × 200 materials = 400)")
```

Appendix F

expand_sales_orders.py

```
import pandas as pd
import random
import csv
from faker import Faker
from datetime import timedelta, date

fake = Faker()

# Load existing sales orders
df = pd.read_csv("original_datasets/sales_orders.csv")

# Existing customers
customers = [
    "BuildRight Hardware",
    "Kerjaya Bina Bhd",
    "Sime Darby Property",
    "Pavilion Retail Supplies",
    "Maybank Facilities Mgmt",
    "SP Setia Hardware",
    "UEM Sunrise Bhd",
    "IJM Corporation",
    "Eco World Supplies",
    "Sunway Building Materials",
    "Gamuda Trading",
    "Metro Paint Supplies",
    "Nippon Construction Sdn Bhd",
    "YTL Hardware Centre",
    "Top Glove Facilities"
]

# Finished products
products = [
    "Premium White Paint 1L",
    "Premium White Paint 4L",
    "Anti-Rust Coating",
    "Industrial Grey Primer",
    "Marine Blue Coating",
    "Epoxy Floor Coating",
    "Clear Varnish",
    "Protective Roof Paint",
    "Automotive Red Paint",
    "Wood Stain Brown"
]

statuses = ["Delivered", "Open"]

start_id = len(df) + 1
target_new = 432

new_rows = []
```

```

for i in range(target_new):

    sales_order_id = f"SO-{start_id + i:03d}"

    # realistic order date
    order_date = fake.date_between(
        start_date=date(2025, 1, 1),
        end_date=date(2025, 12, 31)
    )

    customer_name = random.choice(customers)
    finished_product = random.choice(products)

    quantity_ordered = random.randint(100, 800)

    # delivery date
    delivery_days = random.randint(14, 35)
    required_delivery_date = order_date + timedelta(days=delivery_days)

    # status logic
    today = date(2025, 12, 31)

    if required_delivery_date <= today:
        status = random.choices(
            ["Delivered", "Open"],
            weights=[0.9, 0.1]
        )[0]
    else:
        status = "Open"

    new_rows.append({
        "sales_order_id": sales_order_id,
        "order_date": order_date.strftime("%Y-%m-%d"),
        "customer_name": customer_name,
        "finished_product": finished_product,
        "quantity_ordered": quantity_ordered,
        "required_delivery_date": required_delivery_date.strftime("%Y-%m-%d"),
        "status": status
    })

# Combine datasets
new_df = pd.DataFrame(new_rows)
expanded_df = pd.concat([df, new_df], ignore_index=True)

# Save
expanded_df.to_csv("expanded_datasets/expanded_sales_orders.csv", index=False, quoting=csv.QUOTE_NONNUMERIC)

print(f"Generated {len(new_rows)} new sales orders")
print("Done!")

```

Appendix G

expand_suppliers.py

```
import pandas as pd
import random
import csv

# Load existing data
df = pd.read_csv("original_datasets/suppliers.csv")

existing_names = set(df["supplier_name"].str.lower())

countries = [
    "Malaysia", "China", "Singapore", "India",
    "Germany", "Thailand", "South Korea",
    "Japan", "USA", "Vietnam", "Indonesia"
]

reliability_levels = ["High", "Medium", "Low"]

prefix_map = {
    "Malaysia": ["ChemBase", "PackPro", "ResinTech", "Apex", "BioChem"],
    "China": ["Sino", "Dragon", "EastStar", "Global", "Prime"],
    "Singapore": ["Asia", "Pacific", "Neo", "Vertex", "Prime"],
    "India": ["Indo", "Bharat", "Sun", "Zen", "Nova"],
    "Germany": ["Euro", "Bavaria", "Chem", "Nova", "Prime"],
    "Thailand": ["Thai", "Siam", "Bangkok", "Pacific", "Sun"],
    "South Korea": ["Korea", "Seoul", "Han", "Neo", "K-Tech"],
    "Japan": ["Nippon", "Tokyo", "Shin", "Zen", "Kyo"],
    "USA": ["American", "Global", "North", "Ultra", "Prime"],
    "Vietnam": ["Viet", "Saigon", "Mekong", "Green", "Asia"],
    "Indonesia": ["Indo", "Jakarta", "Bali", "Nusa", "Pacific"]
}

suffixes = [
    "Materials Co", "Chemicals Ltd", "Industries",
    "Solutions", "Imports", "Trading Co",
    "Supply Chain Ltd", "Manufacturing Co"
]

new_rows = []

start_id = len(df) + 1
target_count = 90

while len(new_rows) < target_count:
    country = random.choice(countries)
    name = f"{random.choice(prefix_map[country])} {random.choice(suffixes)}"

    # enforce uniqueness
    if name.lower() in existing_names:
        continue

    existing_names.add(name.lower())
```

```

# weighted reliability realism
if country in ["Germany", "Singapore", "Japan", "USA"]:
    reliability = random.choices(
        ["High", "Medium", "Low"],
        weights=[0.6, 0.3, 0.1]
    )[0]
else:
    reliability = random.choices(
        ["High", "Medium", "Low"],
        weights=[0.3, 0.5, 0.2]
    )[0]

row = {
    "supplier_id": f"SUP{start_id + len(new_rows):03d}",
    "supplier_name": name,
    "country": country,
    "reliability_rating": reliability
}

new_rows.append(row)

# Combine
new_df = pd.DataFrame(new_rows)
expanded_df = pd.concat([df, new_df], ignore_index=True)

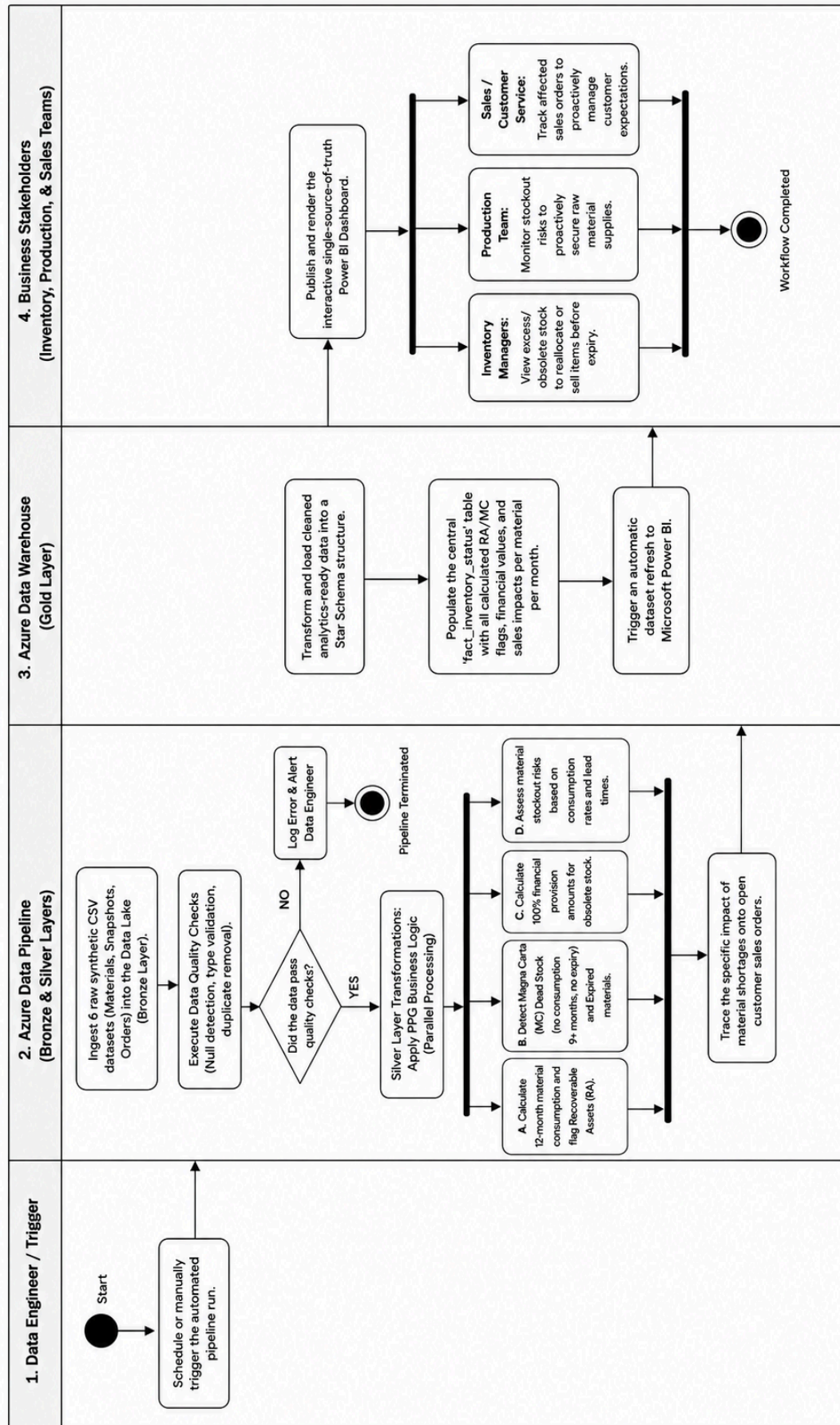
# Save
expanded_df.to_csv("expanded_datasets/expanded_suppliers.csv", index=False, quoting=csv.QUOTE_ALL)

print(f"Generated {len(new_rows)} new suppliers")

```

Appendix H

Activity Diagram



Appendix I

Sequence Diagram

